

Product Vision Document

SentinelCut — AI-Powered Audio/Video Profanity Censorship Platform

Document Version: 1.0 **Prepared For:** Investors, Engineering Team, Academic Reviewers **Document Owner:** Product Management **Status:** Draft for Review

1. Executive Summary

SentinelCut is an AI-driven SaaS platform that automatically detects and censors profanity, abusive language, and offensive content in audio and video files across English, Hindi, Hinglish, and mixed-language speech. It replaces manual, time-consuming audio bleeping — traditionally done frame-by-frame in tools like Adobe Premiere or Audacity — with an automated, editable, multilingual pipeline that produces broadcast-ready output in minutes instead of hours.

This document defines the product's purpose, target users, market opportunity, scope, and success criteria. It is the foundational reference that all subsequent documents (PRD, SRS, Architecture, Roadmap) will build upon.

Note on naming: "AI Profanity Video & Audio Censorship Platform" is descriptive but not brandable. We recommend **SentinelCut** (alternates considered: *CleanCast*, *MuteAI*, *CensorAI*, *PurgeTrack*) as a working product name for documentation and demo purposes. This can be finalized later without affecting technical scope.

2. Problem Statement

2.1 The Core Problem

Content creators, broadcasters, podcast editors, educational platforms, and enterprises that publish audio/video content routinely need to remove profanity or offensive language to meet:

- Platform community guidelines (YouTube, Instagram, OTT platforms)
- Broadcast/censorship regulations (India's CBFC-style norms, FCC in the US)
- Workplace/educational content standards
- Regional language sensitivities (a word innocuous in one dialect may be offensive in another)

2.2 Why Existing Solutions Fall Short

Current Approach	Limitation
Manual editing in Premiere/Audacity	Extremely slow (hours per file), requires skilled editors, doesn't scale
Existing Western censorship tools (e.g., basic beep-editors)	English-only, dictionary-based, no support for Hindi/Hinglish/code-mixed speech
Generic transcription tools	Detect speech but have no profanity-awareness, slang detection, or censorship-specific workflow
Manual review by moderation teams	High cost, inconsistent judgment, not scalable, slow turnaround

2.3 The Gap We Address

There is **no accessible, affordable, self-serve platform** that combines:

1. Multilingual speech recognition (English + Hindi + Hinglish + code-mixed)
2. Context-aware profanity detection (not just dictionary matching — handles slang, spelling variations like "fuuuuck," abbreviations like "bc," "mc," "bkl," and leetspeak like "m@darchod")
3. An editable timeline UI for human-in-the-loop correction
4. Automated, synchronized audio/video re-generation

This gap is especially pronounced for **Indian and South Asian content creators**, who make up one of the largest and fastest-growing content markets globally but are underserved by English-first tools.

3. Target Users

3.1 Primary Personas

1. Independent Content Creator ("Rohan, 24, YouTuber/Podcaster")

- Publishes vlogs/podcasts in Hinglish, needs quick "clean version" for sponsors/YouTube monetization policies.
- Pain: manually bleeping takes 2-3 hours per episode.

2. OTT/Broadcast Post-Production House ("Priya, 32, Video Editor at a media studio")

- Needs to produce a "TV-safe" cut of web-first content for broadcast.
- Pain: compliance deadlines, high editor labor cost, inconsistency across editors.

3. EdTech / Corporate L&D Team ("Ankit, 29, Learning Content Manager")

- Publishes recorded lectures/webinars with informal speaker language that needs cleanup before internal/external distribution.

4. Podcast Network / Aggregator

- Needs to produce "clean" versions of episodes at scale for platforms (e.g., Spotify) requiring explicit-content tagging accuracy or clean alternates.

3.2 Secondary Personas

- Parents/educators wanting to sanitize downloaded content for children.
 - Independent film/short-film makers preparing censor-board submission cuts.
 - Enterprises with internal recorded-meeting archives requiring redaction of inappropriate language before wider distribution.
-

4. Goals

4.1 Business Goals

- Establish a working, demonstrable MVP suitable for academic evaluation and portfolio/investor presentation within one semester.
- Position the platform as a foundation for a real SaaS product post-graduation (freemium credit-based model).
- Build a differentiated moat around **Hindi/Hinglish profanity detection**, an underserved NLP niche.

4.2 User Goals

- Upload a file and receive a censored, synchronized output with minimal manual effort.
- Retain full control via a review/edit step before final export (avoid "black box" AI mistakes).
- Support both fully automatic and fully custom (user-defined word list) workflows.

4.3 Technical Goals

- Achieve high transcription and detection accuracy on code-mixed Hindi-English speech, a domain where most commercial ASR tools underperform.
 - Maintain frame-accurate audio/video sync after censorship edits.
 - Design a modular pipeline (ASR → Language ID → Profanity Detection → Audio Edit → Mux) so components (e.g., ASR engine) can be swapped as better models emerge.
-

5. Vision Statement

"To make every voice broadcast-ready — instantly, accurately, and in the language people actually speak."

SentinelCut envisions a future where content creators across every language, especially underserved multilingual markets like India, can publish clean, compliant content without sacrificing speed, cost, or creative control. Just as spell-check made grammar accessible to everyone, SentinelCut aims to make **content compliance** a one-click operation.

6. Value Proposition

6.1 For Content Creators

"Upload once, get a broadcast-safe version in minutes — not hours — with a transcript-based editor that puts you in control of every edit."

6.2 Key Differentiators

Differentiator	Why It Matters
Multilingual-first design (English + Hindi + Hinglish + code-mixed)	Most competitors are English-only; this is our core wedge into an underserved market
Editable timeline, not a black box	Builds trust — users can review/override every AI decision before export
Hybrid detection (dictionary + context-aware model)	Catches slang, misspellings, leetspeak, and repeated-letter variants (e.g., "fuuuuck") that pure dictionary tools miss
Multiple censorship modes (beep, mute, custom sound, future AI voice replacement)	Flexibility for different content types (kids' content vs. broadcast vs. casual)
Self-serve SaaS workflow	No need to hire an editor or use complex NLE software

7. Competitive Analysis

Product	Category	Strengths	Weaknesses (vs. SentinelCut)
Adobe Podcast / Premiere manual workflow	Professional NLE	High precision, full creative control	Fully manual, no automated detection, steep learning curve
Descript	AI transcript-based editing	Great transcript-driven editing UX, filler-word removal	No profanity-specific detection, weak Hindi/Hinglish support, not censorship-focused
Auphonic	Audio post-processing	Good audio cleanup automation	No language/profanity detection at all
YouTube auto-captions + manual bleep	Free, built-in	Free	No censorship automation, English-biased captions, poor Hindi accuracy
Generic "profanity filter" text APIs (e.g., text-only moderation APIs)	Text moderation	Fast, cheap for text	Not built for audio/video, no timestamp/audio editing capability
Regional/manual dubbing studios	Full-service	High quality	Expensive, slow, not self-serve, not scalable for small creators

7.1 Competitive Positioning Summary

No direct competitor combines **automated multilingual ASR + profanity-specific NLP + human-editable timeline + synchronized re-render** in a self-serve product. SentinelCut's primary competitive moat is **Hindi/Hinglish/code-mixed profanity detection**, a capability that is largely absent from Western-built tools.

8. Assumptions

- Users have legal rights to the audio/video content they upload.
- Internet connectivity is available for upload/processing (cloud-based MVP; offline/on-prem is a future consideration).
- A "good enough" MVP accuracy target (e.g., 85–90% detection recall) is acceptable for v1; 100% precision is not required at launch, since the timeline editor allows

human correction.

- Free-tier cloud/ML resources (as available to students, e.g., free GPU credits, open-source models) are sufficient to build and demo the MVP.
- Users are willing to review AI-flagged content before final export at this stage of product maturity (human-in-the-loop is a feature, not a limitation).

9. Constraints

Type	Constraint
Academic/Time	Must be deliverable within a college project timeline (single/double semester)
Budget	Must primarily use free-tier/open-source tools (open-source ASR models, free-tier cloud hosting, free-tier storage)
Compute	Limited or no dedicated GPU infrastructure; must design for CPU-feasible inference where possible, with GPU as an optimization path
Team Size	Small team (student-scale), requiring a modular, incrementally buildable architecture
Data	No large proprietary Hindi/Hinglish profanity-labeled dataset exists off-the-shelf; must rely on open datasets, custom dictionaries, and augmentation
Legal	Must avoid becoming a tool for content misuse (e.g., must not enable non-consensual voice cloning); future AI voice-replacement features must include safeguards

10. Scope

10.1 In Scope (MVP and Near-Term)

- Audio/video upload (common formats: MP4, MOV, MP3, WAV, etc.)
- Automatic speech recognition with language detection (English/Hindi/Hinglish)
- Automatic + custom + combined profanity detection modes
- Beep/mute/custom-sound censorship
- Editable transcript timeline (add/remove/adjust detections)
- Synchronized final render with download (video/audio/subtitle/transcript/JSON)
- Basic authentication, dashboard, history, and (for demo purposes) a basic admin panel

10.2 Out of Scope (MVP)

- Real-time/live-stream censorship (Zoom, OBS, live broadcast)
- AI voice cloning for replacement words
- Mobile native apps
- Browser/Chrome extension, Discord/Zoom integrations
- Multi-tenant enterprise billing/roles beyond basic RBAC

(These are documented in the Future Scope section as the long-term product roadmap.)

11. Success Metrics

11.1 Product/Academic Success Metrics

- **Detection Recall:** $\geq 85\%$ of profane/offensive words correctly flagged in mixed Hindi-English test audio.
- **Detection Precision:** $\geq 80\%$ (minimize false positives that annoy users).
- **Sync Accuracy:** Final output audio/video sync drift $< 100\text{ms}$ after censorship edits.
- **Processing Time:** $\leq 2\times$ real-time duration for a standard-definition file (e.g., a 10-minute video processed in ≤ 20 minutes) on available infrastructure.
- **Task Completion Rate:** $\geq 90\%$ of test users able to complete upload $\rightarrow$ review $\rightarrow$ export flow without external help.

11.2 Business/Demo Success Metrics (if pursued beyond academics)

- Number of files processed during demo/beta period.
 - User retention across repeat uploads (proxy for usefulness).
 - Qualitative feedback score from evaluators/faculty/investors on UX and technical depth.
-

12. Risks

Risk	Impact	Likelihood	Mitigation
Hindi/Hinglish ASR accuracy insufficient	High	Medium	Use best-in-class multilingual ASR (evaluated in Tech Stack doc); allow manual transcript correction in timeline editor
Profanity detection misses slang/spelling variants	Medium	High	Hybrid approach: fuzzy-matching + normalization (handle repeated letters, leetspeak) + context model
Audio/video sync breaks after edits	High	Medium	Use frame-accurate FFmpeg-based processing; extensive sync testing
Compute cost/GPU unavailability	Medium	High	Design pipeline to run on CPU-feasible open-source models for MVP; treat GPU as a performance upgrade, not a hard dependency
False positives frustrate users	Medium	Medium	Human-editable timeline before final export mitigates this by design
Scope creep (very large feature list in original brief)	High	High	Strict MVP-first phased roadmap (documented in Development Roadmap)
Misuse potential (future voice cloning)	High	Low (future feature only)	Defer AI voice cloning to future phase with explicit consent/safeguard design

13. Future Vision (3–5 Year Horizon)

SentinelCut's long-term vision extends beyond file-based censorship into a **real-time, multi-platform content compliance layer**:

- **Real-time censorship** for live streaming, video calls (Zoom/OBS integration), and podcasts recorded live.
- **Platform integrations**: YouTube, Discord, and browser extensions for in-the-moment content moderation.
- **Expanded language coverage**: regional Indian languages (Tamil, Telugu, Bengali, Marathi, Punjabi) and other code-mixed language pairs globally.
- **AI voice replacement**: natural-sounding word substitution (not just beeping) using consent-safe, non-cloning voice synthesis.

- **Enterprise content moderation dashboard:** for media companies and platforms needing bulk compliance workflows, analytics, and audit trails.
- **Developer API:** allowing third-party platforms to embed censorship-as-a-service into their own upload pipelines.
- **On-device/offline processing:** for privacy-sensitive or low-connectivity use cases.

The long-term ambition is for SentinelCut to become the **default compliance layer for multilingual audio/video content**, in the same way spell-checkers and grammar tools became default layers for text.

14. Document Approval & Next Steps

This Product Vision document forms the foundation for:

1. **Product Requirements Document (PRD)** — detailed feature requirements
2. **Software Requirements Specification (SRS)** — functional/non-functional requirements
3. **Tech Stack Justification** — detailed comparison and selection of ASR, NLP, backend, frontend, and infrastructure

Next Document: Product Requirements Document (PRD)

End of Product Vision Document

Product Requirements Document (PRD)

SentinelCut — AI-Powered Audio/Video Profanity Censorship Platform

Document Version: 1.0 **Status:** Draft for Engineering Review **Owner:** Product Management **Based On:** Product Vision v1.0

1. Purpose of This Document

This PRD translates the Product Vision into concrete, buildable requirements. It defines **what** the product must do (features, user stories, acceptance criteria), **how well** it must do it (non-functional requirements), and **in what order** it gets built (prioritization and release plan). It is the primary reference for Engineering, Design, and QA during implementation.

2. Product Summary

SentinelCut lets a user upload an audio/video file, automatically detects profane/offensive words in English, Hindi, Hinglish, and mixed-language speech, and produces a censored output (beep/mute/custom sound) with a human-editable review timeline before final export.

3. User Personas (Reference)

Persona	Primary Need
Independent Creator (Rohan)	Fast, self-serve censorship for YouTube/podcast uploads
Post-Production Editor (Priya)	Bulk, reliable censorship with fine-grained manual override
L&D/EdTech Manager (Ankit)	Clean up recorded lecture/webinar audio before distribution
Guest/Evaluator	Try the product with zero signup friction (demo purposes)

4. User Stories & Functional Requirements

Each feature area lists **user stories**, **functional requirements (FR)**, and **acceptance criteria (AC)**.

4.1 Authentication & Account

User Stories

- As a new user, I want to sign up with email/password or Google so I can save my work.
- As a returning user, I want to log in and see my past files.

- As a first-time visitor, I want to try the product as a guest without creating an account.
- As a user who forgot my password, I want to reset it via email.

Functional Requirements

ID	Requirement
FR-1.1	System shall support email/password signup and login
FR-1.2	System shall support Google OAuth login
FR-1.3	System shall support "Forgot Password" via emailed reset link
FR-1.4	System shall support Guest Mode with session-limited access (no persistent history)
FR-1.5	System shall enforce password strength rules (min 8 chars, 1 number, 1 special char)

Acceptance Criteria

- AC-1: User can complete signup → verify email → login in under 2 minutes.
- AC-2: Guest sessions expire after a configurable time (e.g., 24 hours) and clearly warn the user their files are temporary.
- AC-3: Password reset link expires after 30 minutes for security.

4.2 Dashboard

User Stories

- As a logged-in user, I want to see my recent files, processing status, and remaining credits at a glance.

Functional Requirements

ID	Requirement
FR-2.1	Dashboard shall display Recent Files (last N uploads with status)
FR-2.2	Dashboard shall display History with search/filter by date, status, filename
FR-2.3	Dashboard shall display Credits/Usage remaining
FR-2.4	Dashboard shall display Storage Usage (used vs. limit)
FR-2.5	Dashboard shall display live Processing Queue status for in-progress jobs

Acceptance Criteria

- AC-1: Dashboard loads within 2 seconds for a user with up to 100 historical files.
- AC-2: Processing status updates in near real-time (polling or websocket, ≤5s latency).

4.3 Upload

User Stories

- As a user, I want to drag-and-drop or browse to upload my audio/video file.
- As a user, I want clear feedback if my file is too large or an unsupported format.

Functional Requirements

ID	Requirement
FR-3.1	System shall accept video formats: MP4, MOV, MKV, AVI
FR-3.2	System shall accept audio formats: MP3, WAV, AAC, FLAC, M4A
FR-3.3	System shall support drag-and-drop and click-to-browse upload
FR-3.4	System shall enforce a configurable max file size (e.g., 500MB for MVP, tunable per plan)
FR-3.5	System shall show upload progress (%) with the ability to cancel
FR-3.6	System shall validate file integrity/format server-side before queuing for processing

Acceptance Criteria

- AC-1: Unsupported file types are rejected with a clear, specific error message before upload starts.
- AC-2: Uploads over the size limit are blocked client-side with an explanatory message.
- AC-3: A failed/interrupted upload can be retried without re-selecting the file (resumable upload preferred, not mandatory for MVP).

4.4 Detection Mode Selection

User Stories

- As a user, I want to choose automatic detection using a built-in profanity database.
- As a user, I want to define my own custom word list.
- As a user, I want to combine both automatic and custom detection.

Functional Requirements

ID	Requirement
FR-4.1	System shall offer three detection modes: Automatic, Custom, Combined
FR-4.2	Custom mode shall accept a comma/line-separated list of words/phrases
FR-4.3	Combined mode shall merge automatic-database hits with custom-list hits, de-duplicated
FR-4.4	System shall allow saving a custom word list as a reusable preset per user

Acceptance Criteria

- AC-1: Custom words are matched case-insensitively and against common spelling variants.
- AC-2: A user can save, edit, and delete custom word-list presets.

4.5 Language Detection & Speech Recognition

User Stories

- As a user speaking Hinglish, I want the system to correctly transcribe and detect profanity without me specifying the language manually.

Functional Requirements

ID	Requirement
FR-5.1	System shall auto-detect spoken language(s): English, Hindi, Hinglish, mixed
FR-5.2	System shall select the appropriate ASR model/config based on detected language
FR-5.3	System shall handle code-switching within a single file (e.g., sentence starts in English, ends in Hindi)
FR-5.4	System shall allow manual language override if auto-detection is incorrect

Acceptance Criteria

- AC-1: Language auto-detection accuracy $\geq 85\%$ on a labeled test set (English/Hindi/Hinglish).

- AC-2: User can override detected language from a dropdown, triggering re-transcription.
-

4.6 Profanity Detection

User Stories

- As a user, I want the system to catch not just exact profane words but also slang, misspellings, and disguised variants (e.g., "fuuuuck," "m@darchod," "bc").

Functional Requirements

ID	Requirement
FR-6.1	System shall detect profanity via a hybrid approach (dictionary + normalization + context model — see AI Pipeline doc)
FR-6.2	System shall normalize repeated letters (e.g., "fuuuuck" → "fuck") before matching
FR-6.3	System shall detect common abbreviations/slang (bc, mc, bkl, etc.) via curated slang dictionary
FR-6.4	System shall detect leetspeak/symbol substitutions (e.g., "m@darchod") via normalization rules
FR-6.5	System shall assign a confidence score to each detection
FR-6.6	System shall allow filtering detections by confidence threshold in the UI

Acceptance Criteria

- AC-1: Detection recall $\geq 85\%$, precision $\geq 80\%$ on the internal test dataset (per Product Vision success metrics).
 - AC-2: Each detected instance displays word, timestamp, confidence score, and detection source (dictionary/custom/model).
-

4.7 Timeline Editor (Review & Edit)

User Stories

- As a user, I want to see a transcript with detected words highlighted so I can review before finalizing.
- As a user, I want to add a missed detection or remove a false positive.

- As a user, I want to change the replacement type per detection (beep vs. mute vs. custom sound).

Functional Requirements

ID	Requirement
FR-7.1	System shall display full transcript with timestamps aligned to audio/video playback
FR-7.2	System shall visually highlight all detected words/phrases in the transcript
FR-7.3	User shall be able to delete a detection (mark as not profane)
FR-7.4	User shall be able to add a new detection manually by selecting transcript text or a timestamp range
FR-7.5	User shall be able to change replacement mode per individual detection
FR-7.6	User shall be able to adjust the start/end timestamp of a detection (fine-tuning)
FR-7.7	System shall provide inline audio/video playback synced to the transcript cursor
FR-7.8	System shall support "Regenerate Output" after edits without requiring a full re-upload

Acceptance Criteria

- AC-1: Edits made in the timeline are reflected in a live preview before final export.
- AC-2: Regeneration after edits only reprocesses the affected audio segments where feasible (not a full re-transcription), to reduce processing time.
- AC-3: Undo/redo supported for at least the last 10 edit actions.

4.8 Replacement / Censorship Modes

Functional Requirements

ID	Requirement
FR-8.1	System shall support Beep replacement (configurable tone/frequency)
FR-8.2	System shall support Mute (silence) replacement
FR-8.3	System shall support Custom uploaded audio sound as replacement
FR-8.4	System shall reserve extension points for future AI voice-word replacement and voice cloning (not built in MVP)

Acceptance Criteria

- AC-1: Replacement audio duration matches or gracefully fits the original detected word's duration without causing audible desync.

4.9 Export

Functional Requirements

ID	Requirement
FR-9.1	System shall export censored video (matching original container/format where feasible)
FR-9.2	System shall export censored audio-only file
FR-9.3	System shall export subtitle file (.srt) with censored word markers
FR-9.4	System shall export plain transcript (.txt)
FR-9.5	System shall export JSON metadata (detections, timestamps, confidence, actions taken)

Acceptance Criteria

- AC-1: Exported video preserves original resolution, frame rate, and metadata (per Product Vision constraints).
- AC-2: Audio/video sync drift in final export < 100ms (per success metrics).

4.10 Admin Panel (MVP-lite)

Functional Requirements

ID	Requirement
FR-10.1	Admin shall be able to view/manage registered users
FR-10.2	Admin shall be able to manage the global profanity database (add/remove/edit entries)
FR-10.3	Admin shall be able to view uploaded files and processing logs
FR-10.4	Admin shall be able to view basic analytics (uploads/day, avg. processing time, detection stats)
FR-10.5	Admin shall be able to view user feedback submissions

Acceptance Criteria

- AC-1: Admin actions (e.g., editing the profanity database) are logged with admin ID and timestamp.

5. Non-Functional Requirements (NFR)

Category	Requirement
Performance	Processing time $\leq 2\times$ real-time duration for standard-definition files (per Product Vision)
Scalability	Backend processing shall run via a queue system to support concurrent jobs without blocking the API
Availability	Target 99% uptime for the web application (academic/demo-scale target, not enterprise SLA)
Security	All traffic over HTTPS; JWT-based session auth; input validation on all uploads; virus/malware scanning on uploaded files
Data Privacy	Uploaded files and transcripts are private to the uploading user by default; guest files auto-deleted after expiry
Usability	A first-time user should be able to complete upload $\rightarrow$ detect $\rightarrow$ export without external instructions (per Product Vision task completion metric)
Accessibility	UI shall meet baseline WCAG 2.1 AA contrast/keyboard-navigation guidelines where feasible
Compatibility	Web app shall be responsive across desktop and tablet screen sizes at minimum
Maintainability	Codebase shall follow modular/repository-pattern architecture (per Coding Standards doc) to allow swapping ASR/NLP models
Observability	System shall log processing pipeline stages and errors for debugging and admin visibility

6. User Journeys

6.1 Primary Journey — Registered User, Automatic Detection

1. User logs in $\rightarrow$ lands on Dashboard.
2. Clicks "New Upload" $\rightarrow$ drags in an MP4 file.
3. Selects Detection Mode: Automatic.
4. Selects Output Mode: Beep.
5. Clicks "Start Processing" $\rightarrow$ sees job enter Processing Queue.
6. Receives notification / sees status change to "Ready for Review."

7. Opens Timeline Editor → reviews highlighted detections, removes 1 false positive, adds 1 missed word.
8. Clicks "Regenerate."
9. Previews final output inline.
10. Downloads censored video + transcript.

6.2 Secondary Journey — Guest User, Custom Word List

1. Visitor lands on landing page → clicks "Try as Guest."
2. Uploads a short audio clip.
3. Selects Detection Mode: Custom → enters 3 custom words.
4. Selects Output Mode: Mute.
5. Reviews detections (limited timeline features for guest, e.g., no saved presets).
6. Downloads output before session expires; prompted to sign up to save history.

6.3 Edge Journey — Failed Detection Correction

1. User uploads a file with heavy background noise.
 2. ASR transcription has multiple errors; several profane words are missed.
 3. User manually adds detections by selecting text/timestamps in the editor.
 4. User regenerates and verifies via inline preview before export.
-

7. Edge Cases

Edge Case	Expected Behavior
File with no speech (music only)	System completes transcription with empty/near-empty transcript; no false detections; user notified "No speech detected"
Extremely long file (e.g., 3-hour lecture)	System should chunk processing and show incremental progress; may require async/background job with email notification on completion
Overlapping speakers	Detection still attempted per audio segment; UI should indicate lower confidence where overlap is detected (future: speaker diarization)
Profane word at the very start/end of a clip (partial word cut off)	System should not crash; boundary detection padded by a safety margin (e.g., 100–150ms)
User uploads a corrupted/unsupported codec file	Reject with clear error before entering the processing queue
Same word appears both as profanity and normal word in different context (context-dependent)	Confidence score reflects context-model judgment; user can override via timeline editor
User edits timeline, closes browser mid-edit	Auto-save draft state periodically so work isn't lost
Guest mode session expires before download	Clear warning shown before expiry with a "sign up to save" prompt
Duplicate/near-duplicate custom words entered	De-duplicated automatically, case-insensitive
Non-Latin script custom word input (Devanagari)	Supported; matching pipeline must handle Unicode normalization

8. Assumptions & Constraints

(Inherited from Product Vision; restated here in PRD context)

- MVP targets a reasonable accuracy bar ($\geq 85\%$ recall), not perfection — the human-editable timeline is the safety net.

- Free-tier/open-source infrastructure is used for the academic build; architecture should still be "production-shaped" for future scaling.
 - No proprietary large-scale Hindi/Hinglish profanity dataset exists; detection quality depends on curated dictionaries + open datasets + normalization rules, which is an accepted MVP limitation.
 - Real-time/live censorship, AI voice cloning, and third-party platform integrations are explicitly out of scope for this PRD (see Future Scope in Product Vision).
-

9. Feature Prioritization (MoSCoW)

Must Have (MVP — Release 1)

- Signup/Login (email + Google), Guest Mode
- Upload (drag-and-drop, format/size validation)
- Automatic Detection Mode (dictionary + normalization)
- English + Hindi + Hinglish language auto-detection
- Beep and Mute replacement modes
- Timeline Editor: view transcript, highlight detections, delete/add detection, regenerate
- Export: censored video/audio, transcript (.txt), JSON metadata
- Basic Dashboard: recent files, processing status

Should Have (Release 2)

- Custom Detection Mode + Combined Mode
- Custom word-list presets (save/reuse)
- Custom Sound replacement mode
- Subtitle (.srt) export
- History with search/filter
- Confidence-score filtering in timeline editor
- Basic Admin Panel (user + profanity DB management)

Could Have (Release 3)

- Storage usage & credits system UI
- Undo/redo in timeline editor
- Analytics dashboard for admin
- Forgot password flow polish, rate limiting, virus scan integration
- Dark mode / light mode toggle

Won't Have (This Release — Documented as Future Scope)

- Real-time/live-stream censorship
- AI voice cloning / custom spoken word replacement

- Mobile native apps, browser extensions
- Third-party integrations (Zoom, Discord, OBS, YouTube)
- Multi-tenant enterprise billing

10. Release Planning

Release	Theme	Key Deliverables	Target Milestone
R1 — MVP	Core censorship pipeline works end-to-end	Auth, Upload, Automatic Detection, Beep/Mute, Timeline Editor (basic), Export (video/audio/transcript/JSON)	Mid-semester demo
R2 — Usability & Control	User control and customization	Custom/Combined detection, presets, custom sound, subtitles, history/search	Pre-final demo
R3 — Polish & Admin	Production-shaped polish	Admin panel, analytics, credits/storage UI, UX polish (dark mode, undo/redo)	Final submission/demo
R4+ — Future Scope	Beyond academic scope	Real-time censorship, integrations, voice cloning, mobile app	Post-graduation roadmap

11. Open Questions for Engineering (to resolve in SRS/Architecture phase)

- Exact ASR model selection and its offline/CPU feasibility (addressed in Tech Stack Justification doc).
 - Whether "Regenerate Output" reprocesses only edited segments or the full file for MVP simplicity vs. R2 optimization.
 - Storage retention policy specifics (how long are registered-user files retained by default).
-

12. Next Document

Document 3: Software Requirements Specification (SRS) — formalizing functional and non-functional requirements into IEEE-style specification format, including detailed interface requirements and system constraints.

End of Product Requirements Document

Software Requirements Specification (SRS)

SentinelCut — AI-Powered Audio/Video Profanity Censorship Platform

Document Version: 1.0 **Standard Followed:** IEEE 830-1998 / ISO/IEC/IEEE 29148:2018 (hybrid structure) **Status:** Implementation-Ready Draft **Based On:** Product Vision v1.0, PRD v1.0

1. Introduction

1.1 Purpose

This Software Requirements Specification (SRS) defines the complete functional and non-functional requirements for **SentinelCut**, an AI-powered platform that detects and censors profanity in audio and video files across English, Hindi, Hinglish, and mixed-language content. This document is written for use by the development team, QA team, and project evaluators as the authoritative technical requirements reference. It is intended to be sufficiently detailed that a developer can begin implementation without requiring further requirement clarification.

1.2 Scope

The system, SentinelCut, shall:

- Accept audio/video uploads from users.
- Transcribe speech using automatic speech recognition (ASR) with multilingual support.
- Detect profane/offensive words using a hybrid dictionary + context-aware NLP approach.
- Allow users to review and edit detections via a timeline interface.
- Generate a censored output file (beep/mute/custom sound) with preserved audio-video synchronization.
- Provide user account management, a dashboard, and a lightweight admin panel.

The system shall **not**, in this version, provide: real-time/live-stream censorship, AI voice cloning, mobile native apps, or third-party platform integrations (Zoom/Discord/OBS/YouTube). These are documented as future scope in the Product Vision document.

1.3 Definitions, Acronyms, and Abbreviations

Term	Definition
ASR	Automatic Speech Recognition
NLP	Natural Language Processing
FR	Functional Requirement
NFR	Non-Functional Requirement
MVP	Minimum Viable Product
RBAC	Role-Based Access Control
JWT	JSON Web Token
API	Application Programming Interface
Hinglish	Code-mixed Hindi-English spoken/written language
Code-switching	Alternating between two or more languages within a single conversation/sentence
Profanity Detection Engine (PDE)	The subsystem responsible for identifying offensive words/phrases from transcribed text
Timeline Editor	The UI component allowing users to review/edit detected profanity instances
Detection Instance	A single flagged word/phrase with associated timestamp, confidence score, and source
Mux/Muxing	The process of merging processed audio back into a video container
SRS	Software Requirements Specification

1.4 References

- Product Vision Document v1.0 (SentinelCut)
- Product Requirements Document (PRD) v1.0 (SentinelCut)
- IEEE 830-1998, Recommended Practice for Software Requirements Specifications
- ISO/IEC/IEEE 29148:2018, Systems and Software Engineering — Requirements Engineering

1.5 Overview of Document

Section 2 provides an overall description of the product, users, and constraints. Section 3 details system features and functional requirements with unique IDs. Section 4 specifies external interface requirements. Section 5 specifies non-functional requirements (performance, security, usability, etc.). Section 6 covers appendices, including traceability notes.

2. Overall Description

2.1 Product Perspective

SentinelCut is a new, standalone, cloud-hosted SaaS web application. It is not a replacement for professional NLE (non-linear editing) software but a complementary automation layer for a specific task: profanity detection and censorship. The system architecture consists of four logical subsystems:

1. **Frontend Web Application** — user-facing UI (upload, dashboard, timeline editor, admin panel).
2. **Backend API Service** — handles auth, business logic, job orchestration, database access.
3. **AI Processing Pipeline** — ASR, language detection, profanity detection, audio/video processing (runs as background/queued jobs).
4. **Storage & Database Layer** — persistent storage for files, transcripts, user data, and metadata.

2.2 Product Functions (Summary)

- User registration/authentication (including guest mode)
- File upload with format/size validation
- Automatic, Custom, and Combined profanity detection modes
- Multilingual ASR and language auto-detection
- Timeline-based manual review and editing of detections
- Audio/video censorship rendering with sync preservation
- Multi-format export (video, audio, subtitle, transcript, JSON)
- Admin management of users, profanity database, and logs

2.3 User Characteristics

User Class	Technical Proficiency	Frequency of Use	Notes
Guest User	Low-Medium	One-time/occasional	No account; limited feature access; session-based
Registered Creator	Low-Medium	Regular (weekly)	Primary user base; expects simple, fast workflow
Professional Editor	Medium-High	Frequent (daily/weekly)	Values precision, control, and export flexibility
Admin	High	Occasional	Manages system-level data and monitors platform health

2.4 Assumptions and Dependencies

Assumptions

- Users have legal rights to uploaded content.
- Users have a stable internet connection for upload/download.
- A reasonable accuracy bar ($\geq 85\%$ recall) is acceptable for MVP; the timeline editor compensates for residual errors.

Dependencies

- Availability of a suitable multilingual open-source or free-tier ASR engine (finalized in Tech Stack Justification document).
- Availability of FFmpeg for audio/video processing.
- Third-party OAuth provider (Google) uptime for social login.
- Cloud hosting/storage free-tier availability for academic deployment.

2.5 Constraints

- Must be buildable within an academic project timeline using primarily free/open-source tools.
- Must operate acceptably on CPU-only infrastructure, with GPU as an optional performance enhancement, not a hard requirement.
- No large proprietary Hindi/Hinglish profanity-labeled dataset is available; the system relies on curated dictionaries, open datasets, and normalization heuristics.

3. System Features and Functional Requirements

Each feature is documented with a unique ID pattern: **SRS-FR-[Module]-[Number]**.

3.1 Module: Authentication (AUTH)

Description: Enables user registration, login, and session management, including guest access.

Req ID	Requirement	Priority
SRS-FR-AUTH-01	The system shall allow a user to register using email and password.	Must
SRS-FR-AUTH-02	The system shall allow a user to log in using Google OAuth 2.0.	Must
SRS-FR-AUTH-03	The system shall allow password reset via a time-limited emailed link (expires in 30 minutes).	Must
SRS-FR-AUTH-04	The system shall support Guest Mode, issuing a temporary session token without requiring registration.	Must
SRS-FR-AUTH-05	The system shall enforce password complexity (minimum 8 characters, at least 1 number, 1 special character).	Must
SRS-FR-AUTH-06	The system shall issue a JWT access token upon successful authentication, valid for a configurable duration (default 60 minutes), with refresh token support.	Must
SRS-FR-AUTH-07	The system shall expire guest sessions and associated temporary files after a configurable period (default 24 hours).	Should

3.2 Module: File Upload (UPLD)

Req ID	Requirement	Priority
SRS-FR-UPLD-01	The system shall accept video files in formats: MP4, MOV, MKV, AVI.	Must
SRS-FR-UPLD-02	The system shall accept audio files in formats: MP3, WAV, AAC, FLAC, M4A.	Must
SRS-FR-UPLD-03	The system shall support drag-and-drop and manual file browser upload.	Must
SRS-FR-UPLD-04	The system shall reject files exceeding a configurable maximum size (default 500MB) with a descriptive error.	Must
SRS-FR-UPLD-05	The system shall validate file format server-side (not relying solely on file extension) prior to queuing.	Must
SRS-FR-UPLD-06	The system shall display real-time upload progress as a percentage.	Should
SRS-FR-UPLD-07	The system shall scan uploaded files for malware prior to processing.	Should

3.3 Module: Detection Configuration (DTCT-CFG)

Req ID	Requirement	Priority
SRS-FR-DTCT-CFG-01	The system shall provide three detection modes: Automatic, Custom, Combined.	Must
SRS-FR-DTCT-CFG-02	In Custom mode, the system shall accept a user-supplied list of words/phrases (comma or newline separated).	Must
SRS-FR-DTCT-CFG-03	In Combined mode, the system shall merge automatic-database results with custom-list results, removing duplicates.	Must
SRS-FR-DTCT-CFG-04	The system shall allow registered users to save and reuse custom word lists as named presets.	Should
SRS-FR-DTCT-CFG-05	The system shall support Unicode (Devanagari script) input for custom word lists.	Must

3.4 Module: Language Detection & ASR (LANG-ASR)

Req ID	Requirement	Priority
SRS-FR-LANG-01	The system shall automatically detect the primary spoken language(s) in an uploaded file: English, Hindi, Hinglish, or mixed.	Must
SRS-FR-LANG-02	The system shall select an ASR configuration appropriate to the detected language(s).	Must
SRS-FR-LANG-03	The system shall support code-switching detection within a single file/sentence.	Should
SRS-FR-LANG-04	The system shall allow the user to manually override the detected language, triggering re-transcription.	Should
SRS-FR-LANG-05	The system shall generate a time-aligned transcript (word-level or segment-level timestamps) from the ASR output.	Must

3.5 Module: Profanity Detection Engine (PDE)

Req ID	Requirement	Priority
SRS-FR-PDE-01	The system shall detect profane/offensive words using a hybrid dictionary-matching plus context-aware model approach.	Must
SRS-FR-PDE-02	The system shall normalize repeated-character variants (e.g., "fuuuuck" → "fuck") prior to matching.	Must
SRS-FR-PDE-03	The system shall detect common transliterated/abbreviated slang terms via a curated slang dictionary.	Must
SRS-FR-PDE-04	The system shall normalize leetspeak/symbol-substituted text (e.g., "@a" → "a") prior to matching.	Must
SRS-FR-PDE-05	The system shall assign a confidence score (0.0-1.0) to each detection instance.	Must
SRS-FR-PDE-06	The system shall allow filtering of displayed detections by a user-configurable confidence threshold.	Should
SRS-FR-PDE-07	The system shall tag each detection with its source (dictionary, custom-list, or model-inferred).	Should
SRS-FR-PDE-08	The profanity dictionary shall be extensible by administrators without requiring a code deployment.	Should

3.6 Module: Timeline Editor (TLE)

Req ID	Requirement	Priority
SRS-FR-TLE-01	The system shall display the full transcript with detected instances visually highlighted.	Must
SRS-FR-TLE-02	The system shall synchronize transcript display with audio/video playback position.	Must
SRS-FR-TLE-03	The system shall allow the user to delete (dismiss) a detection instance.	Must
SRS-FR-TLE-04	The system shall allow the user to manually add a new detection instance by selecting text or a timestamp range.	Must
SRS-FR-TLE-05	The system shall allow the user to change the replacement mode for an individual detection instance.	Must
SRS-FR-TLE-06	The system shall allow the user to fine-tune the start/end timestamp of a detection instance.	Should
SRS-FR-TLE-07	The system shall support "Regenerate Output" that reprocesses the file based on current edits.	Must
SRS-FR-TLE-08	The system shall auto-save timeline edit drafts periodically (e.g., every 30 seconds) to prevent data loss.	Should
SRS-FR-TLE-09	The system shall support undo/redo for at least the last 10 edit actions.	Could

3.7 Module: Replacement / Censorship (REPL)

Req ID	Requirement	Priority
SRS-FR-REPL-01	The system shall support Beep replacement with configurable tone frequency.	Must
SRS-FR-REPL-02	The system shall support Mute (silence) replacement.	Must
SRS-FR-REPL-03	The system shall support Custom uploaded audio-clip replacement.	Should
SRS-FR-REPL-04	The system shall reserve an extensible interface for future AI voice-word replacement (not implemented in current release).	Could
SRS-FR-REPL-05	The system shall ensure replacement audio duration matches the original detected segment duration (± 50 ms tolerance).	Must

3.8 Module: Export (EXPT)

Req ID	Requirement	Priority
SRS-FR-EXPT-01	The system shall export a censored video file preserving original container format where feasible.	Must
SRS-FR-EXPT-02	The system shall export a censored audio-only file.	Must
SRS-FR-EXPT-03	The system shall export a subtitle file in .srt format reflecting censorship markers.	Should
SRS-FR-EXPT-04	The system shall export a plain-text transcript (.txt).	Must
SRS-FR-EXPT-05	The system shall export a JSON metadata file containing all detection instances, timestamps, confidence scores, and user actions taken.	Must

3.9 Module: Dashboard (DASH)

Req ID	Requirement	Priority
SRS-FR-DASH-01	The system shall display a list of the user's recent files with processing status.	Must
SRS-FR-DASH-02	The system shall provide searchable/filterable file history.	Should
SRS-FR-DASH-03	The system shall display remaining credits/usage quota.	Should
SRS-FR-DASH-04	The system shall display current storage usage against the user's limit.	Should
SRS-FR-DASH-05	The system shall display live status of the processing queue for in-progress jobs.	Must

3.10 Module: Admin Panel (ADM)

Req ID	Requirement	Priority
SRS-FR-ADM-01	The system shall allow administrators to view and manage registered users (view, disable, delete).	Should
SRS-FR-ADM-02	The system shall allow administrators to add, edit, or remove entries in the global profanity dictionary.	Should
SRS-FR-ADM-03	The system shall allow administrators to view uploaded file records and processing logs.	Should
SRS-FR-ADM-04	The system shall present basic analytics: uploads per day, average processing time, detection statistics.	Could
SRS-FR-ADM-05	The system shall log all admin actions with admin identity and timestamp for auditability.	Should
SRS-FR-ADM-06	The system shall allow administrators to view user-submitted feedback.	Could

4. External Interface Requirements

4.1 User Interfaces

- Responsive web application (desktop and tablet breakpoints minimum) built with a component-based framework.

- Key screens: Landing Page, Signup/Login, Dashboard, Upload, Detection/Output Configuration, Processing Status, Timeline Editor, Export/Download, Settings/Profile, Admin Dashboard.
- Support for Light Mode and Dark Mode (Should priority).

4.2 Hardware Interfaces

- No direct hardware interface required. Optional GPU utilization on the backend/AI processing server for accelerated inference, abstracted from the end user.

4.3 Software Interfaces

Interface	Purpose
ASR Engine (final selection documented in Tech Stack Justification)	Speech-to-text transcription
FFmpeg	Audio extraction, audio/video processing, muxing, format conversion
Google OAuth 2.0 API	Social login
Email Service (transactional email provider)	Password reset, notifications
Object Storage Service	Storing uploaded/processed media files
Relational Database	Storing user, file, and detection metadata
Malware Scanning Service/Library	Uploaded file safety validation

4.4 Communication Interfaces

- RESTful API over HTTPS between frontend and backend.
- WebSocket or polling-based mechanism for real-time processing status updates.
- Secure, authenticated signed URLs for file upload/download to/from object storage.

5. Non-Functional Requirements

5.1 Performance Requirements

Req ID	Requirement
SRS-NFR-PERF-01	The system shall process a standard-definition file in no more than 2x its real-time duration under normal load.
SRS-NFR-PERF-02	The dashboard shall load within 2 seconds for a user account with up to 100 historical files.
SRS-NFR-PERF-03	Processing status updates shall propagate to the UI within 5 seconds of a state change.
SRS-NFR-PERF-04	The system shall support at least [N, e.g., 5] concurrent processing jobs via a queue without degrading API responsiveness (target scaled to available academic infrastructure).

5.2 Security Requirements

Req ID	Requirement
SRS-NFR-SEC-01	All client-server communication shall occur over HTTPS/TLS.
SRS-NFR-SEC-02	The system shall use JWT-based authentication with short-lived access tokens and refresh tokens.
SRS-NFR-SEC-03	The system shall validate and sanitize all user inputs (file uploads, custom word lists, form fields) to prevent injection attacks.
SRS-NFR-SEC-04	The system shall enforce Role-Based Access Control (RBAC) distinguishing Guest, Registered User, and Admin roles.
SRS-NFR-SEC-05	The system shall encrypt sensitive data at rest (e.g., password hashes using a strong algorithm such as bcrypt/argon2).
SRS-NFR-SEC-06	The system shall implement rate limiting on authentication and upload endpoints to mitigate abuse.
SRS-NFR-SEC-07	The system shall scan uploaded files for malware prior to processing.
SRS-NFR-SEC-08	The system shall follow OWASP Top 10 mitigation practices for web application security.

5.3 Usability Requirements

Req ID	Requirement
SRS-NFR-USE-01	A first-time user shall be able to complete the upload → detect → export flow without external documentation, in a usability test with representative users.
SRS-NFR-USE-02	The UI shall meet baseline WCAG 2.1 AA contrast and keyboard-navigation standards where feasible.
SRS-NFR-USE-03	Error messages shall be specific and actionable (e.g., naming the exact unsupported format rather than a generic error).

5.4 Reliability & Availability Requirements

Req ID	Requirement
SRS-NFR-REL-01	The system shall target 99% uptime for the web application (academic/demo-scale target).
SRS-NFR-REL-02	Failed processing jobs shall be retried automatically up to a configurable retry count before being marked as failed and surfaced to the user.
SRS-NFR-REL-03	Partial timeline edits shall be preserved (auto-saved) in the event of a client-side crash or disconnect.

5.5 Maintainability Requirements

Req ID	Requirement
SRS-NFR-MAINT-01	The codebase shall follow a modular, layered architecture (Repository Pattern for data access, service layer for business logic) to enable independent updates to the ASR/NLP components.
SRS-NFR-MAINT-02	All modules shall include inline documentation/comments and adhere to a documented coding standard (see Coding Standards document).
SRS-NFR-MAINT-03	The system shall externalize configuration (e.g., max file size, token expiry, confidence thresholds) rather than hardcoding values.

5.6 Portability Requirements

Req ID	Requirement
SRS-NFR-PORT-01	The backend and AI processing services shall be containerized (Docker) to ensure consistent deployment across environments.
SRS-NFR-PORT-02	The system shall avoid platform-specific dependencies that would prevent deployment across common cloud providers or local infrastructure.

5.7 Legal & Compliance Requirements

Req ID	Requirement
SRS-NFR-LEGAL-01	The system shall include a Privacy Policy and Terms & Conditions accepted at signup.
SRS-NFR-LEGAL-02	The system shall provide users the ability to delete their account and associated data.
SRS-NFR-LEGAL-03	The system shall not retain guest-mode files beyond the configured expiry window.

6. Appendices

6.1 Requirement Priority Legend

Priority	Meaning
Must	Required for MVP; system is non-functional without it
Should	Important, targeted for near-term release, not blocking MVP
Could	Desirable enhancement, lowest priority

6.2 Traceability Note

Each functional requirement in Section 3 traces back to a corresponding user story/feature in the PRD (v1.0) and supports one or more goals defined in the Product Vision (v1.0). A full Requirements Traceability Matrix (RTM) mapping Vision → PRD → SRS → Test Cases is provided as a separate deliverable in the documentation package.

6.3 Open Items for Architecture Phase

- Final ASR engine selection and justification (Tech Stack Justification document).
- Detailed database schema (Database Design document).

- API endpoint specification (API Design document).
- Confirmed queueing/background job technology choice (System Architecture document).

6.4 Document Revision History

Version	Date	Change
1.0	Current	Initial implementation-ready SRS derived from Product Vision and PRD

7. Next Document

Document 4: Tech Stack Justification — comparative evaluation and final recommendation for ASR engine, profanity detection architecture, backend framework, frontend framework, database, storage, and deployment platforms, directly informing the System Architecture document that follows.

End of Software Requirements Specification

Tech Stack Justification Document

SentinelCut — AI-Powered Audio/Video Profanity Censorship Platform

Document Version: 1.0 **Status:** Final Recommendation for Architecture Phase **Based On:** Product Vision v1.0, PRD v1.0, SRS v1.0

1. Purpose

This document evaluates the major technology choices required to build SentinelCut and provides a final, justified recommendation for each layer of the stack: speech recognition, profanity detection architecture, audio/video processing, backend framework, frontend framework, database, storage, authentication, and deployment. Every recommendation is made against the specific constraints established in the SRS: **free/open-source-first, CPU-feasible, Hindi/Hinglish-capable, and buildable within an academic timeline.**

2. Evaluation Criteria (Applied Throughout)

Criterion	Why It Matters for This Project
Accuracy (esp. Hindi/Hinglish)	Core product differentiator per Product Vision
Offline/CPU feasibility	No guaranteed GPU budget; free-tier academic constraint
Cost	Must be free or near-free for MVP
Ease of implementation	Small team, limited timeline
Community support/documentation	Reduces implementation risk
Scalability path	Should not be a dead end if the project continues post-graduation

3. Speech Recognition (ASR) Engine

3.1 Options Compared

Option	Accuracy	Hindi/Hinglish Support	Offline	Speed	GPU Need	Cost	Ease of Use
OpenAI Whisper (open-source)	High	Good (multilingual training incl. Hindi)	Yes	Moderate (slower on CPU)	Optional (faster with GPU)	Free	Easy (pip install, well documented)
WhisperX	High + word-level timestamps	Good (built on Whisper)	Yes	Faster than base Whisper (batched inference + forced alignment)	Optional	Free	Moderate (extra alignment dependencies)
Faster-Whisper (CTranslate2 reimplementation)	Same as Whisper, faster	Good	Yes	Significantly faster on CPU than original Whisper	Optional	Free	Easy
Vosk	Moderate	Weaker on code-mixed/Hinglish	Yes	Fast, lightweight	No	Free	Easy but lower accuracy
NVIDIA NeMo	High	Requires custom Hindi model training/fine-tuning	Yes	Fast on GPU	Strongly GPU-recommended	Free (compute cost)	Complex, steep learning curve
Google Speech-to-Text API	Very High	Good Hindi support	No (cloud only)	Fast	N/A (cloud)	Paid beyond free tier	Very easy (no call)
Azure Speech	Very High	Good Hindi support	No (cloud only)	Fast	N/A (cloud)	Paid beyond free tier	Very easy
Deepgram	High	Limited Hindi/Hinglish maturity vs. Whisper family	No	Very fast	N/A (cloud)	Paid beyond free tier	Very easy
AssemblyAI	High	Limited Hindi/Hinglish maturity	No	Fast	N/A (cloud)	Paid beyond free tier	Very easy

3.2 Key Trade-offs

- **Cloud APIs (Google, Azure, Deepgram, AssemblyAI)** offer the best raw accuracy and zero setup effort, but they violate two hard constraints: they require ongoing per-minute billing beyond limited free tiers, and they are not viable for an "offline-capable, cost-free academic MVP." They also create a hard external dependency that risks demo failure if free-tier quotas are exhausted.
- **Vosk** is attractive for being lightweight and fully offline, but its Hindi and especially Hinglish/code-mixed accuracy is noticeably weaker than Whisper-family models, which directly undermines the project's core differentiator.
- **NVIDIA NeMo** can achieve excellent results but requires either a pretrained Hindi checkpoint of uncertain quality or custom fine-tuning — both of which add significant time/compute risk that is disproportionate for an academic timeline.
- **The Whisper family (Whisper / WhisperX / Faster-Whisper)** consistently offers the best balance: strong multilingual training (including Hindi and reasonable handling of code-switched speech), fully offline/open-source (no API cost), and an active community with abundant implementation resources.

3.3 Recommendation: Faster-Whisper as the primary ASR engine, with WhisperX-style forced alignment for word-level timestamps

Why:

1. **Accuracy:** Inherits Whisper's multilingual training, which performs meaningfully better on Hindi and mixed-language speech than Vosk or untrained NeMo checkpoints.
2. **Speed on CPU:** Faster-Whisper (built on CTranslate2) is significantly faster than vanilla Whisper on CPU-only infrastructure — critical given the "no guaranteed GPU" constraint from the SRS.
3. **Offline & free:** No per-minute API cost, no external dependency risk during a live demo.
4. **Word-level timestamps:** The Timeline Editor (SRS-FR-TLE) requires precise timestamps per detected word. Whisper's native segment-level timestamps are too coarse; incorporating a **forced-alignment step** (the technique WhisperX popularizes, e.g., via wav2vec2-based alignment or Faster-Whisper's built-in word-timestamp option) is necessary to meet FR-TLE-01/02 and the <100ms sync requirement in the Product Vision.
5. **GPU as upgrade path, not dependency:** If GPU becomes available (e.g., free Colab/Kaggle credits during development, or paid infra post-graduation), the same model scales up in speed without an architecture change.

Fallback design note: The ASR engine should sit behind an internal `SpeechRecognitionService` interface (per Repository/Service pattern in Coding Standards) so it can be swapped for a cloud API (e.g., Google Speech-to-Text) in a future paid tier without touching the rest of the pipeline.

4. Profanity Detection Architecture

4.1 Options Compared

Approach	Accuracy on Slang/Variants	Multilingual Handling	Explainability	Implementation Effort	Data Requirement
Pure Dictionary Matching	Poor (misses variants, spelling tricks)	Requires manual per-language dictionaries	High	Low	Curated word lists only
Regex-based matching	Moderate (handles some patterns like repeated letters)	Same as dictionary, plus pattern rules	High	Low-Medium	Curated word lists + patterns
Context-aware Transformer (BERT/roBERTa)	High (understands context, disambiguates)	Requires multilingual or fine-tuned model	Medium	Medium-High	Labeled training data (limited availability for Hindi/Hinglish profanity)
IndicBERT / MuRIL (India-focused multilingual BERT variants)	High for Indian languages	Purpose-built for Hindi/Indic + code-mixed text	Medium	Medium	Some public Indic NLP resources available; profanity-specific labels still limited
XLM-R (cross-lingual roBERTa)	High, broad multilingual	Strong general multilingual, less India-specific tuning than IndicBERT/MuRIL	Medium	Medium-High	Similar data scarcity issue
LLM Classification (zero/few-shot via an LLM)	High, flexible, handles novel slang via reasoning	Strong multilingual understanding out of the box	Low (harder to audit exact rule)	Low (prompt-based) but ongoing inference cost	No training data required, but adds per-request cost/latency
Sentence Transformers (embedding similarity to profanity examples)	Moderate-High	Depends on base multilingual embedding model	Medium	Medium	Needs a seed set of example profane phrases per language

4.2 Key Trade-offs

- **Pure dictionary/regex** is fast, explainable, and requires zero training data — but on its own it cannot reliably catch creative obfuscation (e.g., "fuuuuck," "m@darchod," novel slang), which the Product Vision explicitly requires the system to handle.
- **Pure transformer/LLM classification** is powerful but has two problems for this project: (1) labeled Hindi/Hinglish profanity datasets are scarce, making fine-tuning a full context-aware classifier from scratch a significant undertaking within an academic timeline, and (2) LLM-based classification for every transcript segment adds latency and, if cloud-hosted, cost — conflicting with the offline/free-tier constraint.
- **IndicBERT/MuRIL** are the most relevant pretrained multilingual models for this specific problem (India-focused, code-mixed aware) but still benefit from being combined with a rule-based layer rather than used in isolation, given limited profanity-specific labeled data.

4.3 Recommendation: Hybrid Architecture — Normalization + Dictionary/Regex Layer + Lightweight Context Model (fine-tuned/adapted multilingual embedding or IndicBERT-based classifier) as a confidence booster

Why a hybrid, not a single method:

This directly answers the Product Vision requirement to "explain whether the model should be Rule-Based, Machine Learning, Deep Learning, or Hybrid" — **Hybrid is the correct choice** for this project specifically, because:

1. **No single approach satisfies both requirements simultaneously:** catching obvious/known profanity with high explainability (favors rule-based) *and* catching context-dependent, disguised, or novel slang (favors ML/DL). A hybrid captures both strengths.
2. **Data scarcity is the binding constraint.** A pure deep-learning classifier needs substantial labeled Hindi/Hinglish profanity data that does not readily exist; relying on it alone is high-risk for an academic timeline. A hybrid lets the dictionary/rule layer carry most of the load on day one while the ML layer is incrementally improved.
3. **Explainability matters for user trust.** Because every detection is shown to the user in the Timeline Editor with a confidence score and source tag (SRS-FR-PDE-05/07), a hybrid system can clearly label "dictionary match" vs. "model-inferred," which a black-box-only classifier cannot do as cleanly.

Recommended pipeline layers (in order):

1. **Text Normalization Layer** — lowercasing, Unicode normalization (for Devanagari), repeated-character collapsing ("fuuuuck" → "fuck"), leetspeak/symbol substitution mapping ("m@darchod" → "madarchod").
2. **Dictionary + Regex Matching Layer** — curated multilingual profanity dictionary (English + Hindi + common Hinglish/abbreviated slang such as "bc," "mc," "bkl") matched against normalized text; assigns high-confidence detections instantly.
3. **Context-Aware Scoring Layer** — a lightweight multilingual sentence embedding model (e.g., a MuRIL/IndicBERT-based or general multilingual sentence-

transformer model) computes similarity between each transcript segment and a seed bank of profane example phrases, used to (a) catch terms the dictionary missed and (b) adjust confidence for borderline/context-dependent cases.

4. **Confidence Aggregation** — final confidence score for each detection instance is a combination of match-source reliability (dictionary > regex > embedding-similarity) surfaced to the user per SRS-FR-PDE-05.

This design also directly satisfies the Product Vision's requirement for "**support for future learning**" — the embedding-similarity layer can be improved over time simply by expanding the seed example bank, without retraining a full classifier from scratch, and the dictionary is designed to be admin-editable at runtime (SRS-FR-PDE-08).

5. Audio & Video Processing Libraries

5.1 Options Compared

Library	Purpose	Strengths	Weaknesses
FFmpeg	Industry-standard audio/video processing	Extremely fast, handles virtually every codec/container, precise frame/timestamp control, muxing	Command-line/subprocess-based; steeper learning curve than Python-native libraries
MoviePy	Python video editing wrapper (uses FFmpeg internally)	Pythonic API, good for prototyping edits	Slower for large files, less precise frame-accurate control, adds an abstraction layer over FFmpeg that can obscure performance issues
PyDub	Python audio manipulation	Simple API for audio slicing/overlaying (ideal for beep/mute insertion)	Audio-only; relies on FFmpeg/libav under the hood anyway; not suited for video muxing
Librosa	Audio analysis (not editing)	Excellent for audio feature extraction/analysis (e.g., silence detection, spectral features)	Not designed for writing/exporting production audio; analysis-focused, not an editing tool
Torchaudio	PyTorch-native audio I/O and transforms	Good if audio processing needs to integrate directly with a PyTorch ML pipeline (e.g., feeding ASR models)	Not designed for general-purpose muxing/export workflows

5.2 Recommendation: FFmpeg as the core processing engine, with PyDub for audio segment manipulation (beep/mute/custom-sound insertion) at the Python service layer, and Librosa reserved for any future audio-analysis features (e.g., speaker diarization)

Why:

- **FFmpeg** is non-negotiable for the final mux/export step: it is the only option here with the precision, format coverage, and speed required to satisfy SRS-NFR-PERF-01 ($\leq 2\times$ real-time processing) and the Product Vision's requirement to preserve video quality, frame rate, resolution, and metadata.
- **PyDub** provides the most direct, readable API for the specific operation of "replace a time-bounded audio segment with a beep tone / silence / custom clip," which is the exact operation the Replacement module (SRS-FR-REPL) needs — and PyDub itself shells out to FFmpeg, so there is no redundant dependency.
- **MoviePy** is deliberately not used as the primary engine: its abstraction over FFmpeg tends to be slower and less precise for the frame-accurate, timestamp-critical edits this project requires, though it remains a reasonable option for quick prototyping during early development.
- **Librosa** and **TorchAudio** are not part of the core censorship pipeline but are the correct tools to reach for if/when the project adds analysis-heavy future features like speaker diarization or emotion detection (see Future Scope).

Processing flow: Extract audio (FFmpeg) → run ASR + detection on extracted audio → apply beep/mute/custom-sound edits to the audio track (PyDub) → re-mux edited audio back into the original video container (FFmpeg), preserving original video stream untouched to guarantee resolution/frame-rate/metadata preservation.

6. Backend Framework

6.1 Options Compared

Framework	Language	Async Support	ML/AI Integration	Ecosystem Fit	Learning Curve
FastAPI	Python	Native async, high performance	Excellent — same language as the AI pipeline (Whisper, PyTorch, etc.)	Auto-generated OpenAPI docs, strong typing via Pydantic	Low-Medium
Flask	Python	Limited (needs extensions for async)	Good, same-language benefit	Simple, minimal, very flexible	Low
Django	Python	Limited native async (improving)	Good, same-language benefit	Batteries-included (admin panel, ORM) but heavier than needed	Medium
Express	Node.js/JavaScript	Native async	Requires cross-language calls to Python AI services (extra complexity)	Huge ecosystem	Low-Medium
NestJS	Node.js/TypeScript	Native async	Same cross-language issue as Express	Structured, enterprise-style architecture	Medium-High
Go	Go	Native, very high performance	Requires cross-language calls to Python AI services	Excellent performance, smaller AI/ML ecosystem	Medium-High

6.2 Recommendation: FastAPI

Why:

1. **Same-language pipeline.** The entire AI stack (Whisper/Faster-Whisper, the profanity detection layer, PyDub) is Python. Using FastAPI avoids the significant added complexity of cross-language service calls (e.g., a Node.js backend calling out to a separate Python microservice) that Express, NestJS, or Go would require — a meaningful simplification for a small academic team.
2. **Async performance.** FastAPI's native async support is well suited to a system with a mix of I/O-bound work (file upload/download, DB queries) and long-running background jobs (AI processing), which need to be offloaded to a queue rather than block the request thread.
3. **Automatic API documentation** (OpenAPI/Swagger) directly supports the API Design and API Documentation deliverables required later in the project.
4. **Pydantic-based validation** enforces clean request/response schemas, aligning with SRS-NFR-SEC-03 (input validation/sanitization).
5. Compared to Flask, FastAPI provides async and validation out of the box rather than requiring extensions; compared to Django, FastAPI avoids unnecessary weight (a full ORM/admin/templating stack) that is largely redundant given the project already plans a separate frontend and a lightweight custom admin panel.

7. Frontend Framework

7.1 Options Compared

Framework	Rendering	SEO (Landing Page)	Component Ecosystem	Fit for Rich Interactive UI (Timeline Editor)	Learning Curve
Next.js (React-based)	SSR + CSR hybrid	Strong (SSR for landing/marketing pages)	Very large (shares React ecosystem)	Excellent, with mature libraries for waveform/timeline UIs	Medium
React (CRA/Vite, no SSR)	CSR only	Weaker without extra SSR setup	Very large	Excellent	Medium
Vue	SSR possible (Nuxt)	Strong with Nuxt	Large, slightly smaller than React	Good, fewer ready-made timeline/waveform libraries than React	Low-Medium
Angular	SSR possible (Angular Universal)	Strong with setup	Large but more opinionated/heavier	Good but higher boilerplate for custom interactive components	High

7.2 Recommendation: Next.js

Why:

1. **Hybrid rendering fits the product's dual nature:** SentinelCut needs a marketing/landing page (benefits from SSR for load speed and shareability) *and* a highly interactive authenticated app (Dashboard, Timeline Editor) that behaves like a client-rendered SPA once logged in. Next.js supports both patterns natively in one framework.
2. **Largest ecosystem for the hardest UI piece.** The Timeline Editor is the most technically demanding UI component in the product (synced transcript + audio/video playback + waveform-like highlighting). React's ecosystem has the most mature libraries and community examples for building this kind of media-timeline interface, which reduces implementation risk more than Vue or Angular.
3. **Tailwind CSS integration** (mandated by the SRS/PRD) is first-class and extremely well documented in the Next.js ecosystem.
4. **Deployment simplicity:** Next.js apps deploy seamlessly to free-tier hosting (e.g., Vercel), directly supporting the "recommend free services for a college project" requirement.

8. Database

8.1 Options Compared

Database	Data Model Fit	Free Tier Availability	Query Flexibility	Ops Overhead
PostgreSQL	Excellent for structured, relational data (users, files, detections, jobs) with JSON columns available for flexible metadata	Good (via Supabase/Neon/Railway free tiers)	Full SQL + JSONB flexibility	Low with managed hosting
MongoDB	Good for flexible/nested documents (e.g., detection lists) but weaker for strictly relational data (users↔files↔credits↔roles)	Good (MongoDB Atlas free tier)	Flexible but weaker joins/relational integrity	Low with managed hosting
Supabase (Postgres + BaaS layer)	Same underlying model as PostgreSQL, plus built-in auth/storage/realtime	Generous free tier	Full SQL	Very low (managed)
Firebase (Firestore)	Document-based, real-time sync strength	Generous free tier	Limited complex querying/joins	Very low (managed)
SQLite	Simple relational, file-based	N/A (local file, no hosting needed)	Full SQL but not built for concurrent multi-user production load	Very low, but not production-appropriate for a multi-user SaaS

8.2 Recommendation: PostgreSQL (via Supabase-managed hosting)

Why:

- 1. **Data shape is inherently relational.** Users, files, detection instances, credits, roles, and admin logs have clear relationships (foreign keys, joins) that map cleanly onto a relational schema — the Database Design document (next in the documentation pipeline) will define entities such as `users`, `files`, `detections`, `presets`, `jobs`, and `admin_logs`, which is a textbook relational use case.
- 2. **JSONB support** in PostgreSQL still allows flexible storage where needed (e.g., storing the full JSON detection metadata blob per file) without giving up relational integrity elsewhere — getting the best of both the SQL and document-store worlds.
- 3. **Supabase specifically** is recommended as the hosting layer because it bundles Postgres with authentication and storage in one free-tier-friendly service, directly simplifying the "Deployment: recommend free services for college project" requirement (see Section 10), while still being "just Postgres" underneath — avoiding vendor lock-in risk associated with Firebase's proprietary Firestore query model.
- 4. Compared to MongoDB/Firestore, Postgres avoids the well-known difficulty of expressing relational analytics queries (e.g., admin analytics: "average processing time per user per week") in a document store.
- 5. SQLite is excluded for production/demo hosting despite its simplicity, since it is not designed for the concurrent multi-user access a deployed SaaS demo requires — though it remains useful for local unit testing.

9. Storage

9.1 Options Compared

Storage Option	Media File Fit	Free Tier	Integration Effort
Cloudinary	Strong for images/video with built-in transformations	Limited free tier for video-heavy use	Easy SDKs
AWS S3	Industry standard for large media files	Free tier limited in duration (12 months) and volume	Moderate (IAM setup)
Supabase Storage	Good for general file storage, integrates directly with Supabase Auth/DB (signed URLs, row-level security)	Generous free tier	Very easy, same ecosystem as chosen DB/Auth
Local Storage (server disk)	Simplest for a pure academic demo	Free (no cloud dependency)	Very easy but not scalable, risk of data loss on redeploy, not production-shaped

9.2 Recommendation: Supabase Storage (primary), with Local Storage as a fallback for early local development/testing only

Why:

1. **Ecosystem consistency.** Since PostgreSQL is already hosted via Supabase, using Supabase Storage for uploaded/processed media files means authentication, database, and storage share one consistent permissions model (Row-Level Security policies can directly govern who can access which files) — reducing integration complexity versus stitching together a separate AWS S3 account with its own IAM configuration.
 2. **Free-tier friendliness** directly satisfies the constraint of using free services for a college project, while still being a "real" production-grade object storage service (not a throwaway local-disk hack), keeping the architecture credible to both academic evaluators and future investors.
 3. **AWS S3** remains the recommended upgrade path if the project scales post-graduation and needs finer-grained storage-class/cost optimization, but it is not necessary for MVP.
 4. **Cloudinary** is deprioritized here because its core value proposition (image/video transformation pipeline) overlaps with, rather than adds to, the custom FFmpeg-based processing pipeline this project already needs to build — it would be a partially redundant dependency.
-

10. Authentication

10.1 Options Compared

Option	Custom Auth	OAuth Support	Integration Effort	Fit
JWT (self-managed)	Full control	Requires manual OAuth wiring	Higher	Good if team wants full control, more code to maintain
OAuth (standalone, e.g., Google directly)	N/A	Yes	Moderate	Needed regardless, for Google Login requirement
NextAuth (Auth.js)	Good	Excellent, many providers pre-built	Low	Strong fit specifically because the frontend is Next.js
Clerk	Good, hosted UI components	Excellent	Very low	Strong DX but adds a third-party auth vendor dependency/cost at scale
Supabase Auth	Good	Good (Google OAuth supported)	Low, same ecosystem as DB/Storage	Strong fit given Supabase is already the DB/Storage choice

10.2 Recommendation: Supabase Auth (JWT-based under the hood) for the primary user/session management, satisfying both the "JWT" and "OAuth" requirements simultaneously

Why:

1. **Single-vendor coherence.** Since Postgres (DB) and Storage are both hosted on Supabase, using Supabase Auth as well means a single set of credentials and a single Row-Level Security model governs data access across all three layers — this is a meaningfully simpler architecture for a small team than integrating NextAuth/Clerk as a fourth separate service on top.
2. **Satisfies FR-AUTH requirements directly:** Supabase Auth natively issues JWTs (SRS-FR-AUTH-06) and supports Google OAuth (SRS-FR-AUTH-02) and email/password with reset flows (SRS-FR-AUTH-01/03) out of the box.
3. **Guest Mode** (SRS-FR-AUTH-04) is implemented as an application-level pattern (an anonymous/temporary record with a scoped, time-limited token) layered on top of Supabase Auth rather than requiring a separate system.
4. **NextAuth** remains a very reasonable alternative if the team later decouples from Supabase for auth specifically (e.g., adds more OAuth providers), and **Clerk** is noted as the best pure "developer experience" option but is deprioritized here to

avoid adding a paid third-party vendor dependency beyond what's needed for MVP.

11. Deployment Platforms (Free-Tier Recommendation for Academic Project)

Layer	Recommended Service	Why
Frontend (Next.js)	Vercel (free tier)	Purpose-built for Next.js, zero-config deploys, generous free tier, automatic HTTPS
Backend (FastAPI)	Render or Railway (free/hobby tier)	Simple Docker-based deploys for Python services, supports background workers for the processing queue
AI Processing Workers	Same host as backend (Render/Railway background worker) for MVP; Google Colab/Kaggle free GPU credits usable during development/testing for heavier experimentation	Keeps infra simple for MVP; avoids managing a separate GPU cluster
Database	Supabase (managed Postgres, free tier)	Already justified in Section 8
Storage	Supabase Storage (free tier)	Already justified in Section 9
Queue/Background Jobs	Redis (free tier via Upstash) + a Python task queue (e.g., Celery or RQ)	Needed to satisfy SRS-FR-DASH-05 (live processing queue) and NFR-PERF-04 (concurrent job handling) without blocking the API
Email (password reset)	A transactional email free-tier provider (e.g., Resend or similar)	Needed for SRS-FR-AUTH-03

Note: Exact free-tier limits and specific provider names change frequently; the pattern recommended here (SSR-frontend host + containerized Python backend host + managed Postgres/Storage/Auth + lightweight Redis-backed queue) is the architecturally important decision, and the specific vendor can be swapped without redesigning the system if a given free tier changes or is discontinued.

12. Consolidated Tech Stack Summary

Layer	Final Recommendation
ASR Engine	Faster-Whisper (+ forced alignment for word-level timestamps)
Profanity Detection	Hybrid: Normalization → Dictionary/Regex → Multilingual embedding/IndicBERT-style context scoring
Audio/Video Processing	FFmpeg (core), PyDub (segment-level beep/mute/custom-sound edits)
Backend Framework	FastAPI (Python)
Frontend Framework	Next.js (React, Tailwind CSS)
Database	PostgreSQL (via Supabase)
Storage	Supabase Storage
Authentication	Supabase Auth (JWT + Google OAuth)
Queue/Background Jobs	Redis + Celery/RQ
Deployment	Vercel (frontend) + Render/Railway (backend & workers) + Supabase (DB/Storage/Auth)

This stack is intentionally **Python-unified on the AI/backend side** (simplifying integration between FastAPI and the Whisper/PyDub/FFmpeg pipeline), **Supabase-unified on the data side** (simplifying Auth/DB/Storage integration and free-tier hosting), and **React/Next.js on the frontend** (best-fit ecosystem for the Timeline Editor's interactive requirements) — directly satisfying every constraint set out in the SRS: free-tier feasibility, CPU-first operation, Hindi/Hinglish capability, and buildability within an academic timeline, while remaining a credible, scalable foundation if the project continues beyond graduation.

13. Next Document

Document 5: System Architecture — translating this tech stack into a complete architecture with component diagrams, sequence diagrams, data flow diagrams, and deployment diagrams, building directly on the recommendations above.

End of Tech Stack Justification Document

System Architecture Document

SentinelCut — AI-Powered Audio/Video Profanity Censorship Platform

Document Version: 1.0 **Status:** Final for Implementation **Based On:** Product Vision v1.0, PRD v1.0, SRS v1.0, Tech Stack Justification v1.0

Diagram Notation: All diagrams below are written in Mermaid syntax. They render natively on GitHub, GitLab, Notion, and most modern Markdown viewers. If viewing in a plain-text editor, paste any block into mermaid.live to visualize it.

1. Architectural Style & Principles

SentinelCut follows a **layered, service-oriented monolith with an asynchronous background-processing subsystem** — not a full microservices split. This is a deliberate choice, justified below.

1.1 Why Not Microservices (for MVP)

Consideration	Decision Rationale
Team size	Small academic team; microservices add operational overhead (service discovery, inter-service auth, distributed tracing) disproportionate to team capacity
Language unity	Backend + AI pipeline are both Python (per Tech Stack doc); no cross-language boundary forces a service split
Deployment constraint	Free-tier hosting (Render/Railway) favors fewer, simpler deployable units
Future-proofing	The internal architecture is still modular (service layer + repository pattern + queue-based workers), so it can be decomposed into real microservices later (e.g., splitting the AI worker into its own scalable service) without a rewrite

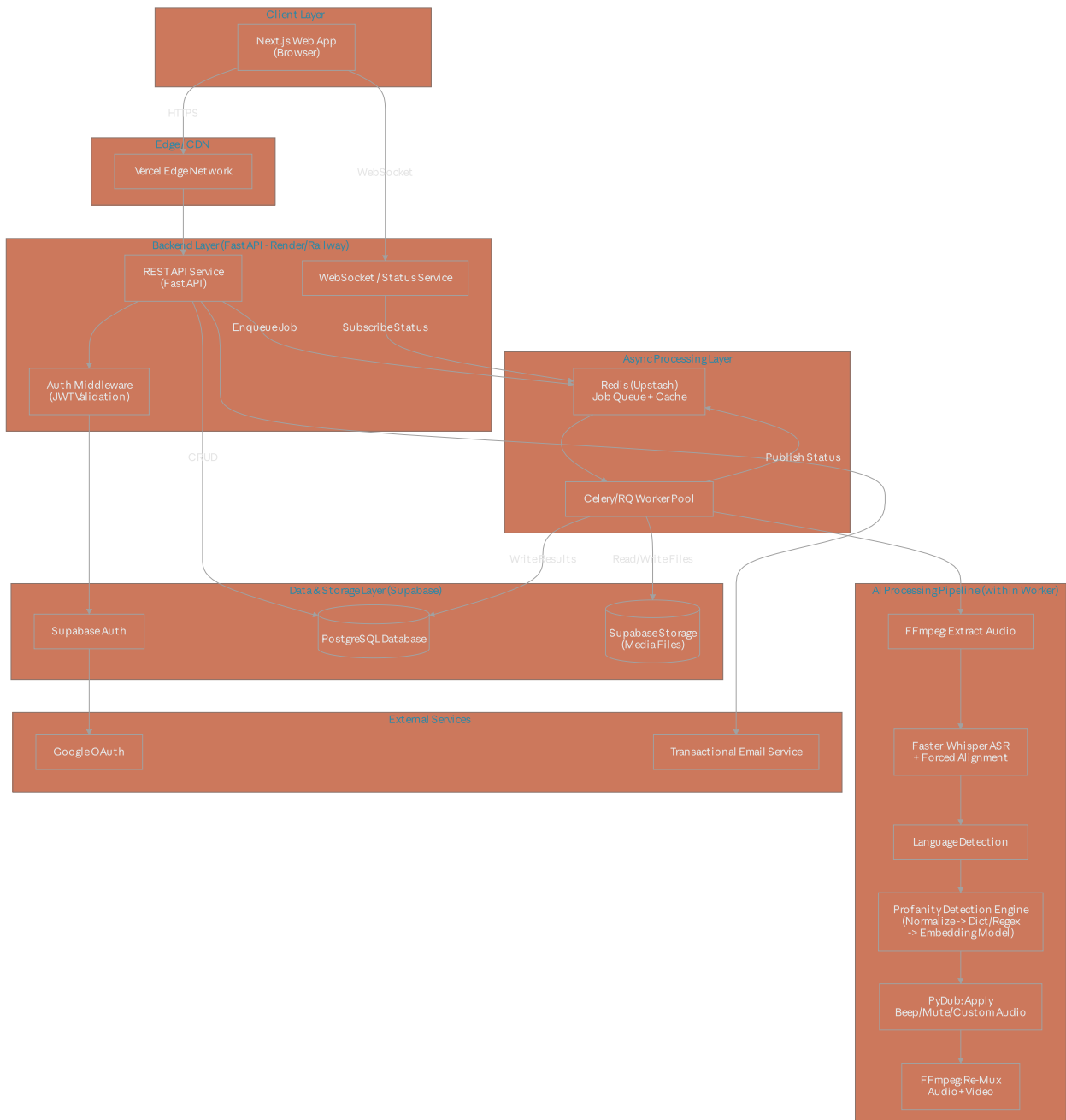
1.2 Core Architectural Principles

- Separation of Concerns:** API layer, business logic (services), data access (repositories), and AI processing are distinct, independently testable layers.
- Asynchronous Processing:** Any operation exceeding a few seconds (ASR, detection, rendering) runs as a background job via a queue — the API never blocks on AI processing.
- Interface-Driven AI Components:** The ASR engine and profanity detection engine are accessed through internal service interfaces

((SpeechRecognitionService), (ProfanityDetectionService)), allowing model swaps without touching calling code (per SRS-NFR-MAINT-01).

- **Single Source of Truth for Auth/Data/Storage:** Supabase (Postgres + Auth + Storage) unifies these three concerns, as justified in the Tech Stack document.
- **Idempotent, Resumable Jobs:** Processing jobs are designed so a retry (SRS-NFR-REL-02) does not duplicate side effects.

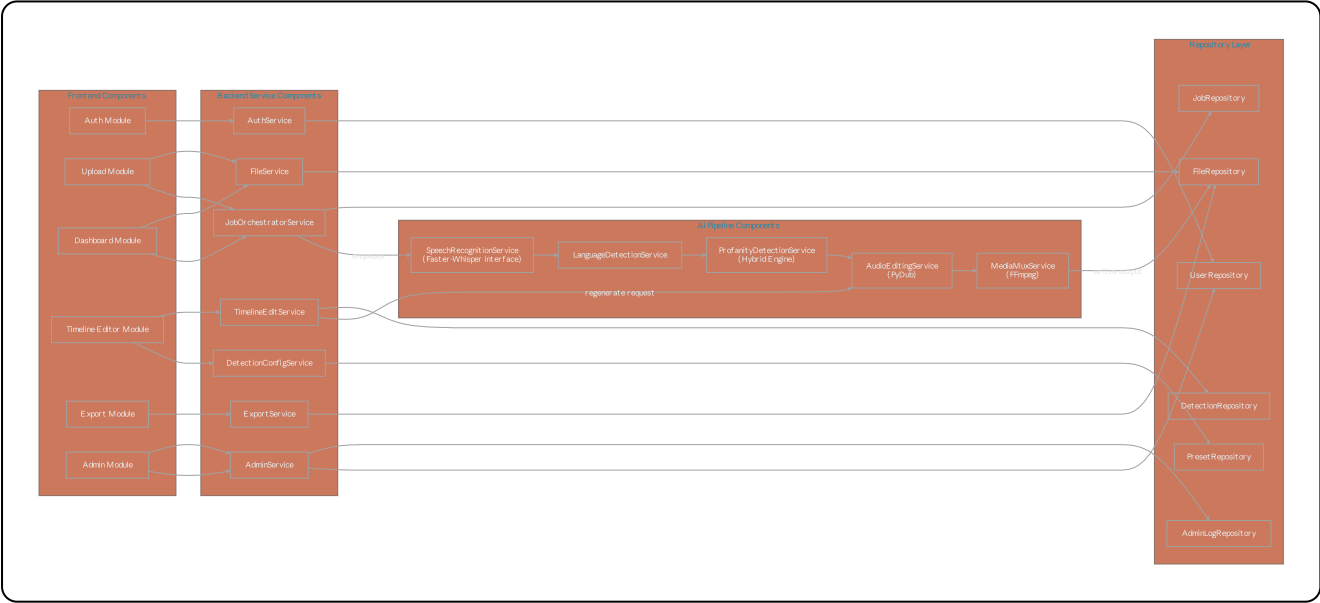
2. High-Level System Architecture



2.1 Layer Summary

Layer	Responsibility
Client Layer	Renders UI, manages local editor state, communicates via REST + WebSocket
Edge/CDN	Serves static/SSR frontend assets globally with low latency
Backend API Layer	Auth validation, request handling, job enqueueing, CRUD operations
Async Processing Layer	Decouples long-running AI work from the request/response cycle
AI Processing Pipeline	The core censorship logic — ASR, language ID, profanity detection, audio editing, muxing
Data & Storage Layer	Persists users, files, metadata, and binary media
External Services	Third-party integrations (OAuth, email)

3. Component Diagram

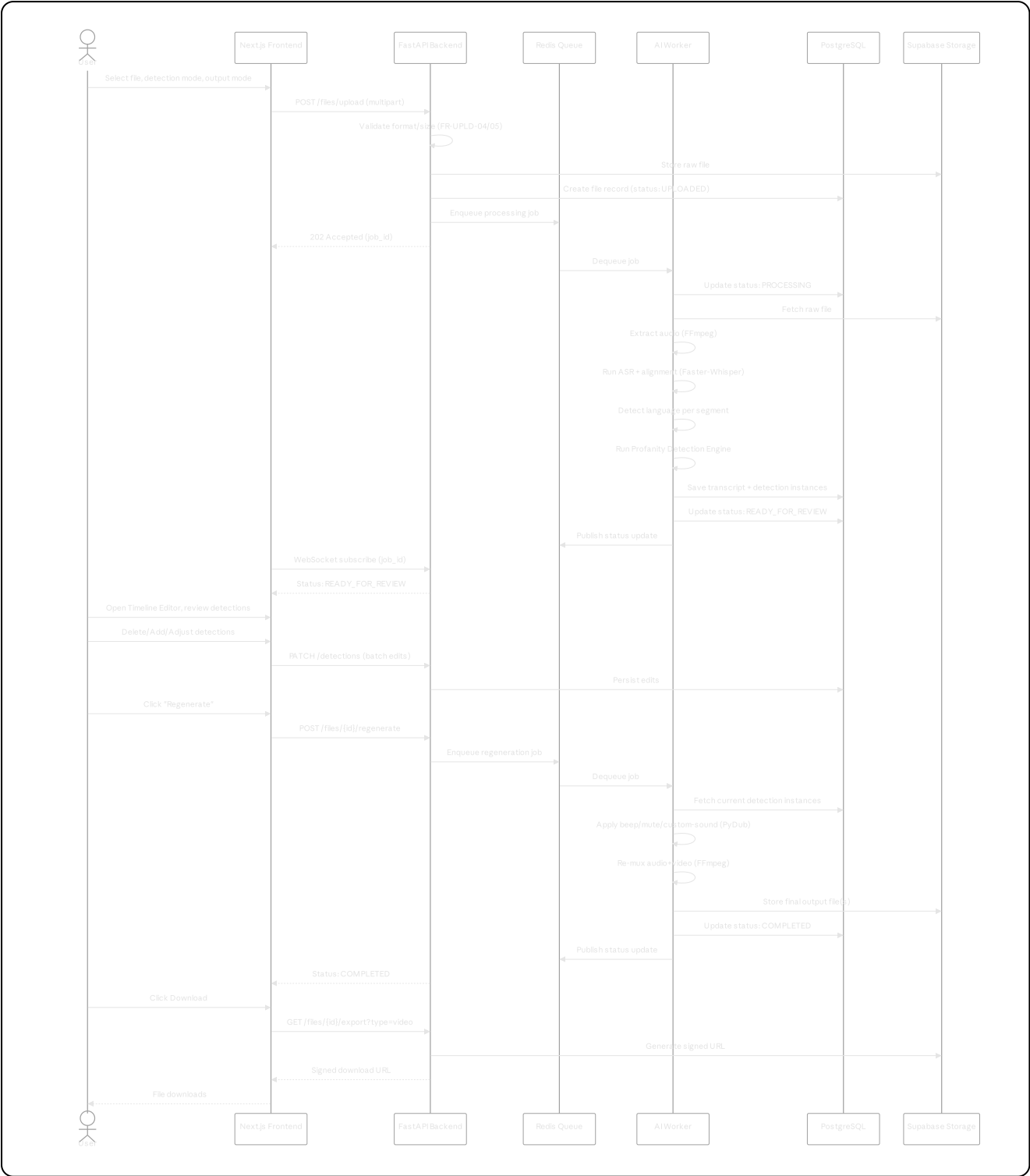


3.1 Component Responsibilities

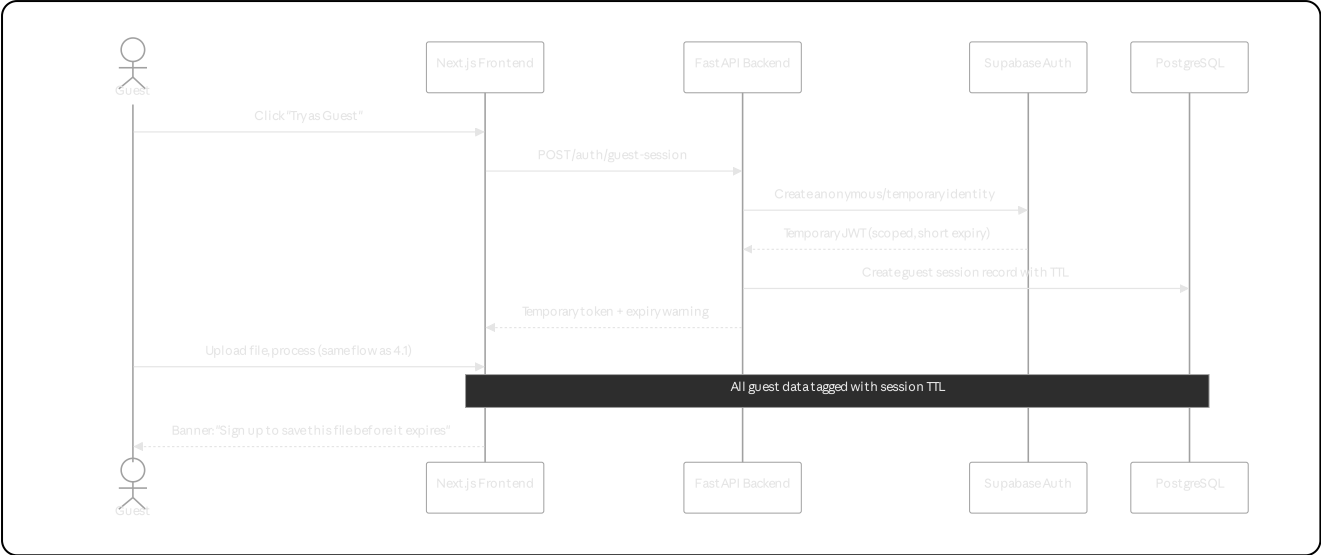
Component	Responsibility
AuthService	Signup/login, token issuance, guest session management
FileService	Upload validation, file metadata management, storage interaction
JobOrchestratorService	Creates processing jobs, enqueues to Redis, tracks job state
DetectionConfigService	Manages detection mode selection, custom word lists, presets
TimelineEditService	Handles CRUD on detection instances during user review/edit
ExportService	Assembles final downloadable artifacts (video/audio/srt/txt/json)
AdminService	User management, profanity dictionary management, analytics, logs
SpeechRecognitionService	Abstracts the ASR engine (Faster-Whisper); swappable interface
LanguageDetectionService	Determines language(s) per segment
ProfanityDetectionService	Runs the hybrid detection pipeline (normalize → dict/regex → embedding scoring)
AudioEditingService	Applies beep/mute/custom-sound edits to specific timestamp ranges
MediaMuxService	Re-combines edited audio with original video via FFmpeg

4. Sequence Diagrams

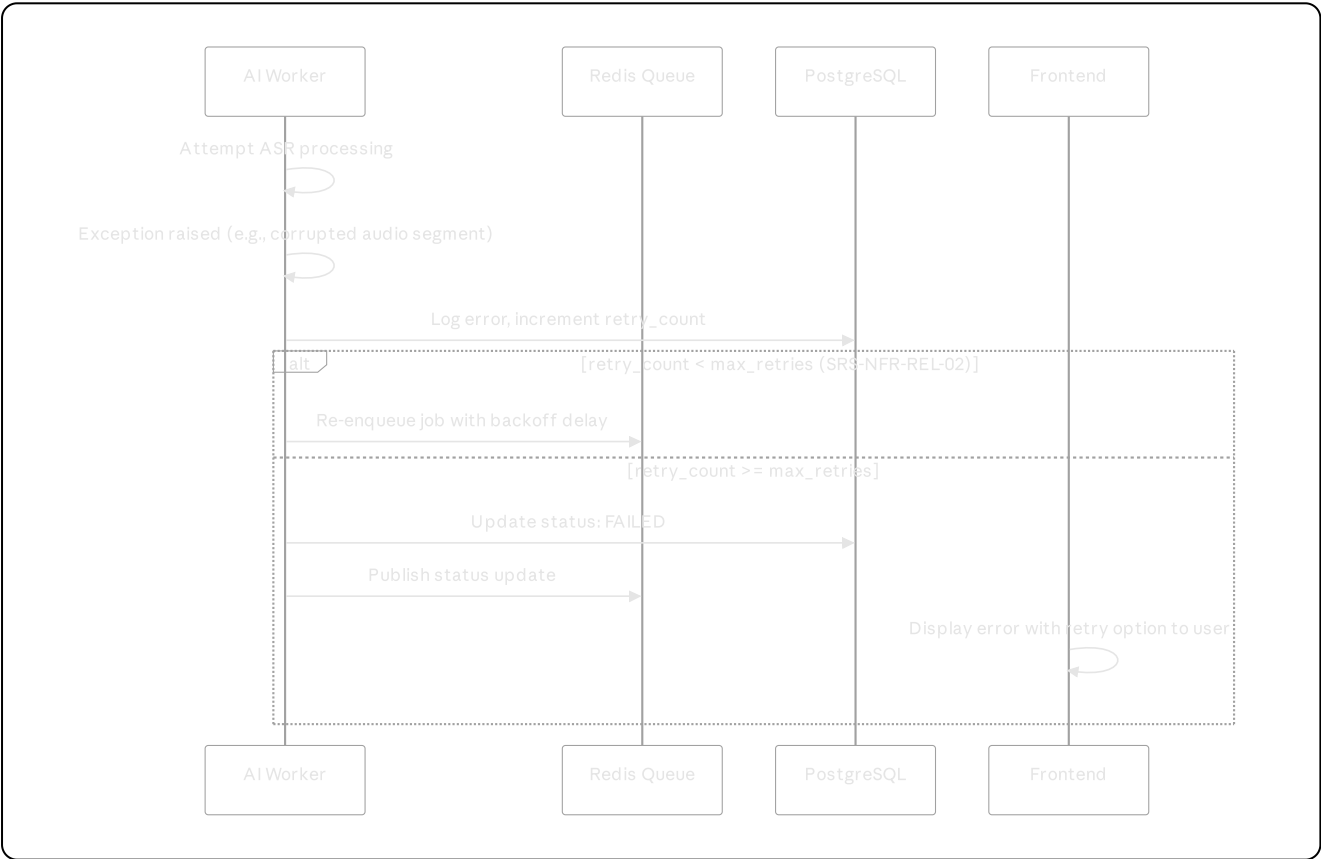
4.1 Primary Flow: Upload → Processing → Review → Export



4.2 Secondary Flow: Guest Mode Upload

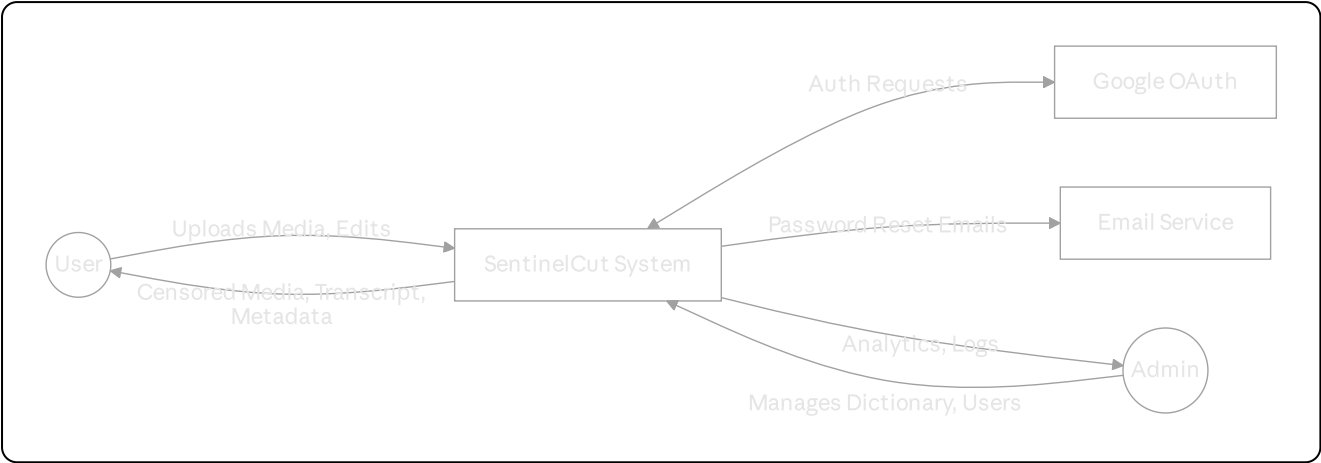


4.3 Edge Case Flow: Failed ASR / Retry

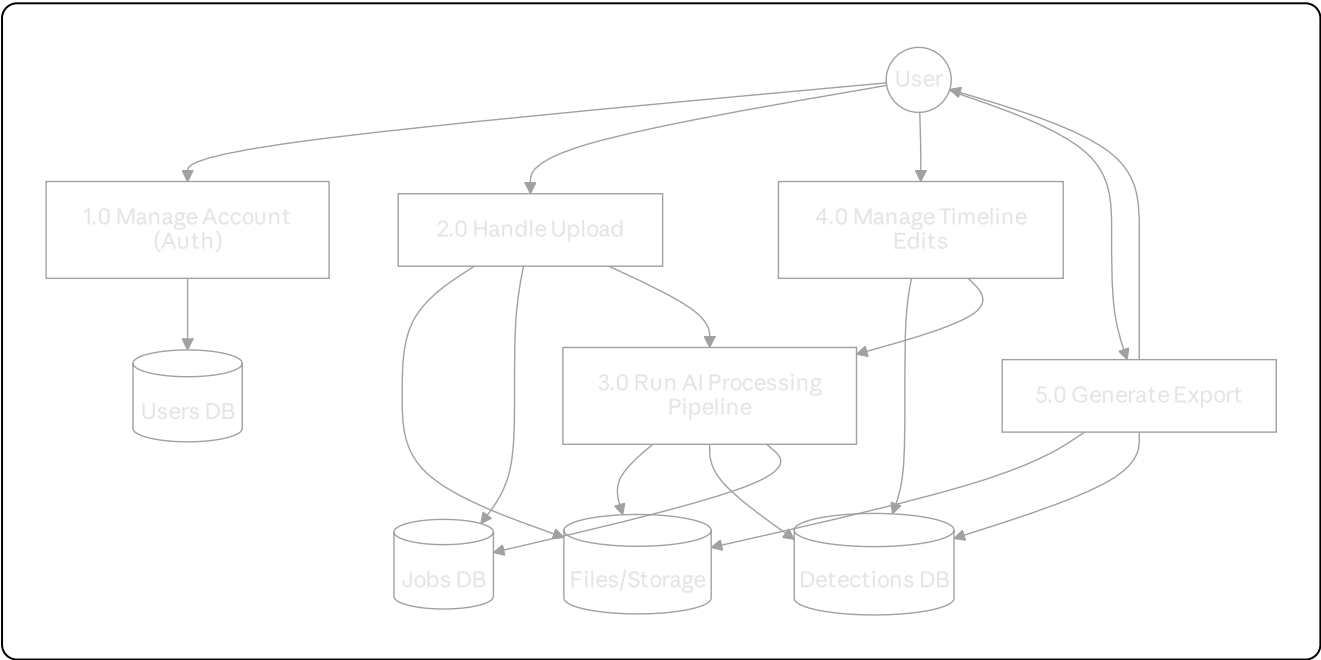


5. Data Flow Diagrams (DFD)

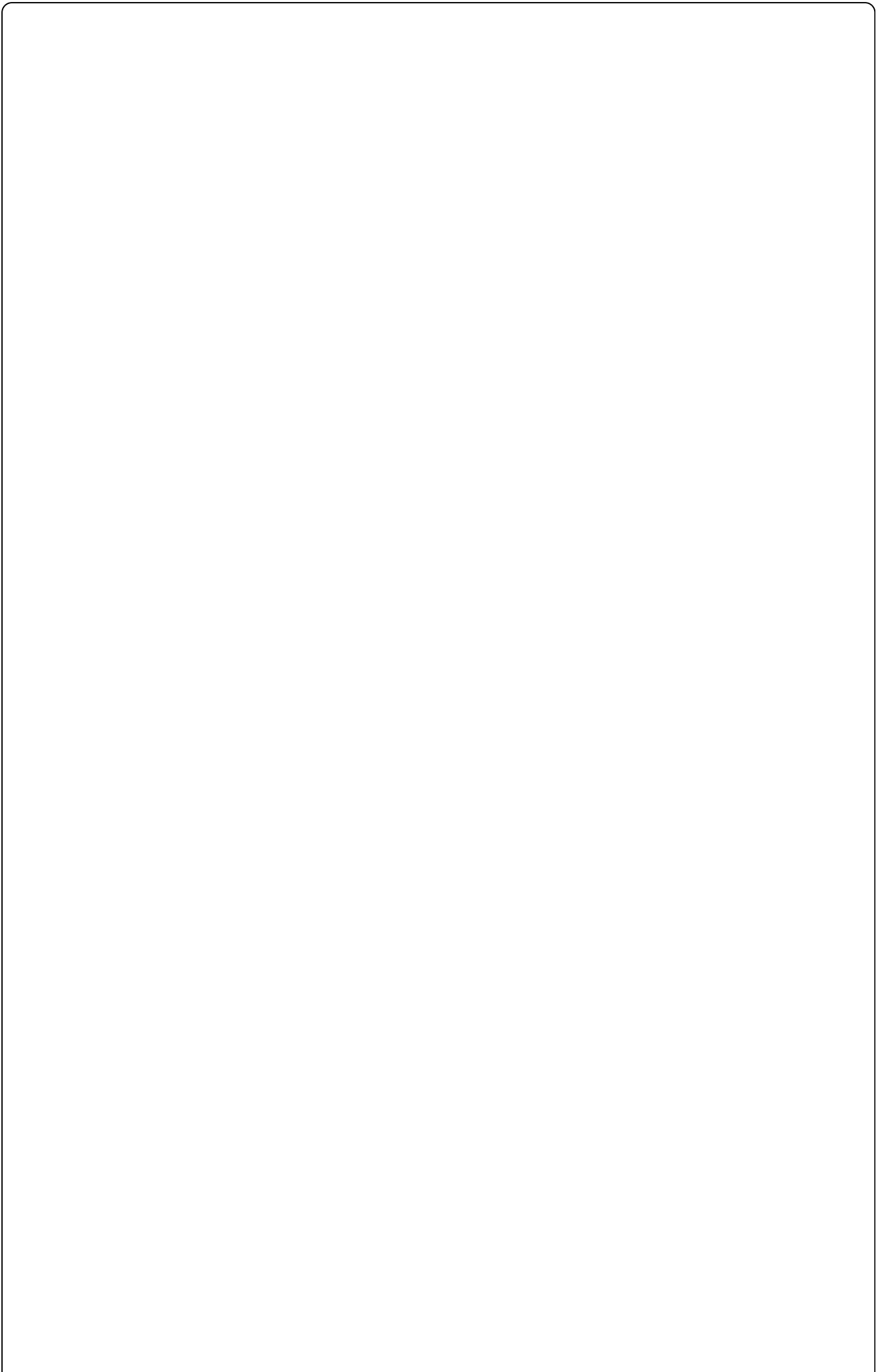
5.1 DFD Level 0 (Context Diagram)

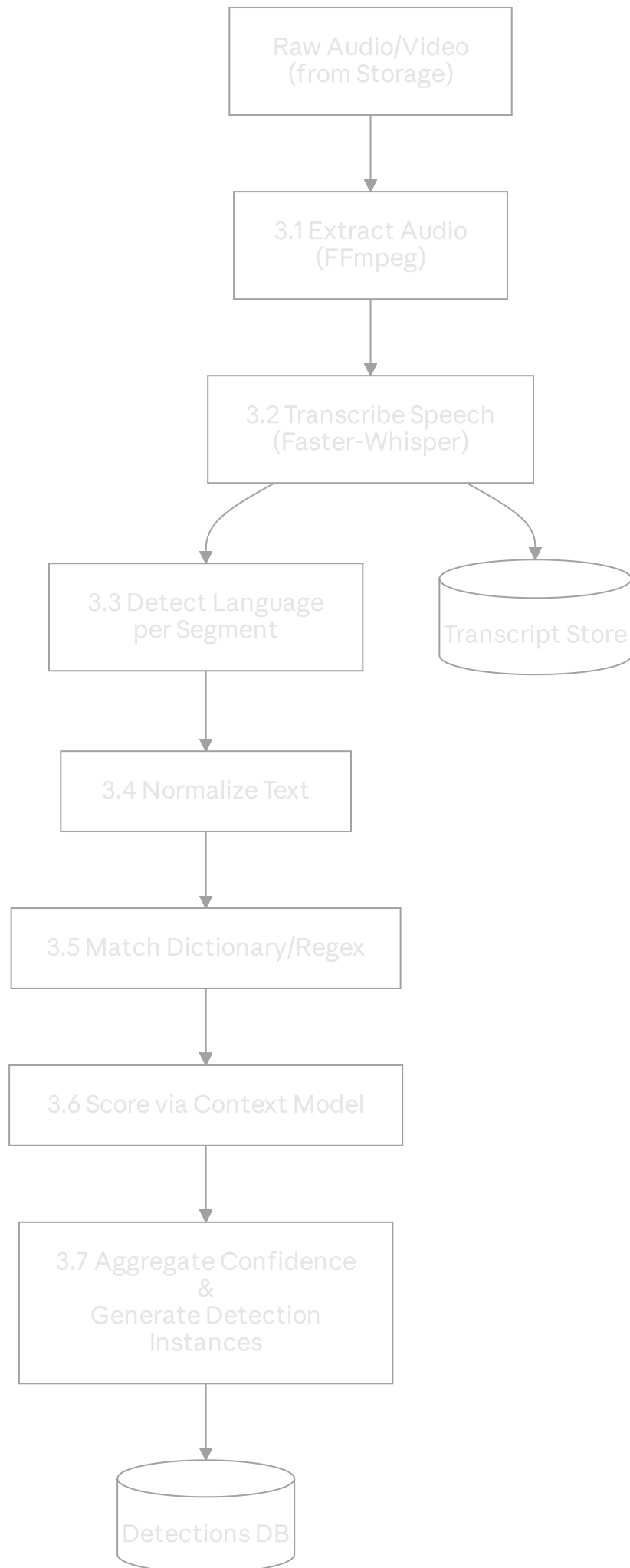


5.2 DFD Level 1 (Major Processes)



5.3 DFD Level 2 (Decomposition of Process 3.0 — AI Processing Pipeline)





6. Deployment Diagram

flowchart TB

```
    subgraph UserDevice["User Device"]
```

```
        BROWSER["Web Browser"]
```

```
    end
```

```
    subgraph VercelCloud["Vercel (Frontend Hosting)"]
```

```
        NEXTAPP["Next.js App<br/>(SSR + Static Assets)"]
```

```
    end
```

```
    subgraph RenderCloud["Render / Railway (Backend Hosting)"]
```

```
        APIINST["FastAPI Instance(s)"]
```

```
        WORKERINST["Celery/RQ Worker Instance(s)"]
```

```
    end
```

```
    subgraph UpstashCloud["Upstash (Managed Redis)"]
```

```
        REDISINST["Redis Instance"]
```

```
    end
```

```
    subgraph SupabaseCloud["Supabase (Managed Platform)"]
```

```
        AUTHINST["Supabase Auth"]
```

```
        PGINST["PostgreSQL Instance"]
```

```
        STORAGEINST["Object Storage Buckets"]
```

```
    end
```

```
    subgraph ThirdParty["Third-Party Services"]
```

```
        GOOGLEAUTH["Google Identity Platform"]
```

```
        EMAILSVC["Email API (e.g., Resend)"]
```

```
    end
```

```
BROWSER <-->|HTTPS| NEXTAPP
```

```
BROWSER <-->|HTTPS / WSS| APIINST
```

```
NEXTAPP -->|API Calls (HTTPS)| APIINST
```

```
APIINST <-->|SQL| PGINST
```

```
APIINST <-->|Auth Calls| AUTHINST
```

```
APIINST -->|Enqueue| REDISINST
```

```
WORKERINST -->|Dequeue| REDISINST
```

```
WORKERINST <-->|Read/Write Files| STORAGEINST
```

```
WORKERINST -->|Write Results| PGINST
```

```
AUTHINST <--> GOOGLEAUTH
```

```
APIINST --> EMAILSVC
```

6.1 Deployment Notes

Concern	Approach
Scaling AI Workers	Worker instances can be scaled horizontally (multiple replicas consuming the same Redis queue) as load increases — this is the primary bottleneck to monitor
Zero-downtime deploys	Frontend (Vercel) and backend (Render/Railway) both support rolling deploys independently
Environment separation	Separate <code>staging</code> and <code>production</code> environment configs (Supabase projects, Redis instances) recommended even for academic demo, to avoid corrupting demo data during development
Secrets management	API keys/DB credentials stored as environment variables in each platform's secret manager — never committed to source control

7. Cross-Cutting Concerns

7.1 Security in the Architecture

- JWT validated at the `AuthMiddleware` layer before any request reaches business logic (satisfies SRS-NFR-SEC-02).
- Signed, time-limited URLs used for all direct file upload/download to/from Supabase Storage — the backend never proxies large binary payloads through itself unnecessarily.
- RBAC enforced at the service layer: `AdminService` methods check role claims from the JWT before executing.

7.2 Observability

- Each job (upload, processing, regeneration) is logged with a `job_id`, timestamps, and status transitions in the `Jobs` table, feeding both the user-facing processing queue UI (SRS-FR-DASH-05) and the Admin analytics view (SRS-FR-ADM-04).
- Structured logging (JSON logs) at the API and Worker layers to support future integration with a log aggregation tool.

7.3 Extensibility Points (Explicitly Designed For)

- `SpeechRecognitionService` interface → swap Faster-Whisper for a cloud ASR API in a future paid tier without touching callers.
- `ProfanityDetectionService` interface → the embedding/context-scoring model can be upgraded or replaced independently of the dictionary/regex layer.
- `AudioEditingService` → designed to accept a future `VoiceReplacementStrategy` (AI voice cloning) as an additional replacement mode without restructuring the

pipeline.

8. Next Document

Document 6: Database Design — full ER diagram, table schemas, relationships, indexing strategy, and constraints for the PostgreSQL/Supabase data layer defined architecturally above.

End of System Architecture Document

Database Design Document

SentinelCut — AI-Powered Audio/Video Profanity Censorship Platform

Document Version: 1.0 **Status:** Final for Implementation **Database Engine:** PostgreSQL 15+ (via Supabase) **Based On:** SRS v1.0, System Architecture v1.0

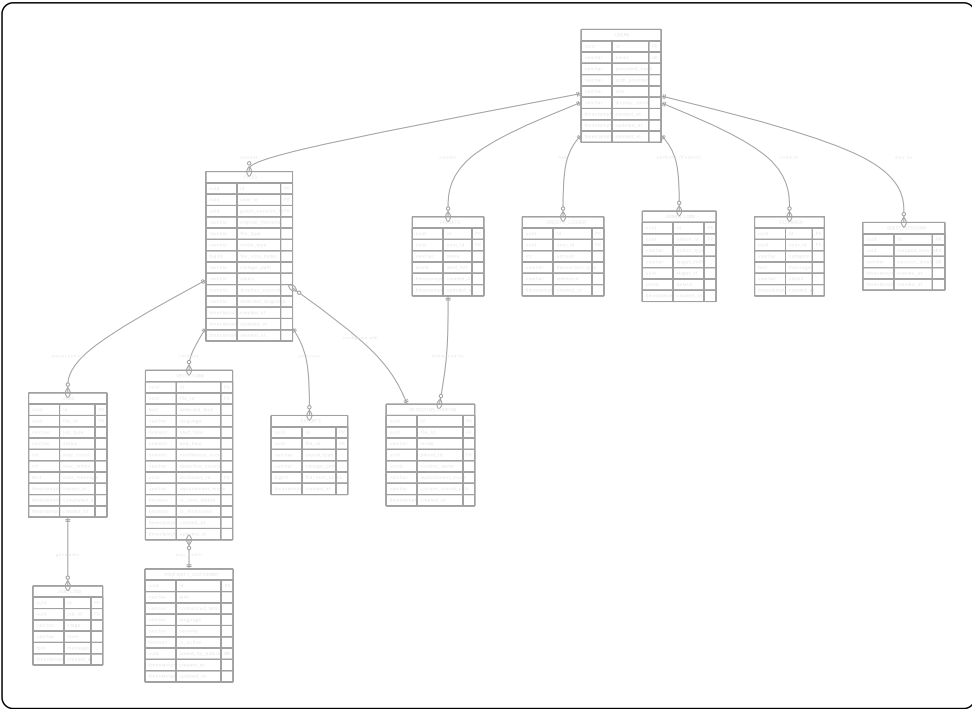
1. Purpose

This document defines the complete relational data model for SentinelCut: entity-relationship diagram, table schemas with column-level constraints, relationships, indexing strategy, and data integrity rules. It is the authoritative reference for implementing the Repository Layer defined in the System Architecture document.

2. Design Principles

- **Normalization to 3NF** for core relational entities (users, files, jobs) to avoid data anomalies, with **selective denormalization via JSONB** for flexible/semi-structured data (e.g., detection metadata snapshots, admin analytics caches) where strict normalization would add unnecessary join overhead.
 - **UUID primary keys** throughout (not auto-increment integers) — required for Supabase Row-Level Security (RLS) compatibility and to avoid exposing sequential IDs (enumeration risk) in signed URLs/API responses.
 - **Soft deletes** (`deleted_at` timestamp) on user-facing entities where audit history matters (files, users), rather than hard deletes, except where SRS-NFR-LEGAL-02 (right to delete account data) requires actual purging.
 - **Every table has** `created_at`/`updated_at` for auditability and debugging (supports SRS-NFR observability requirements).
 - **Foreign keys with explicit `ON DELETE` behavior** defined per relationship (cascade vs. restrict) to prevent orphaned records.
-

3. Entity-Relationship Diagram



4. Table Schemas (DDL-Level Detail)

4.1 `users`

Column	Type	Constraints
id	UUID	PK, default <code>gen_random_uuid()</code>
email	VARCHAR(255)	UNIQUE, NOT NULL
password_hash	VARCHAR(255)	NULL (nullable for OAuth-only accounts)
auth_provider	VARCHAR(20)	NOT NULL, CHECK IN ('email','google')
role	VARCHAR(20)	NOT NULL, DEFAULT 'user', CHECK IN ('user','admin')
display_name	VARCHAR(100)	NULL
storage_used_bytes	BIGINT	NOT NULL, DEFAULT 0
storage_limit_bytes	BIGINT	NOT NULL, DEFAULT 5368709120 (5GB default)
created_at	TIMESTAMPTZ	NOT NULL, DEFAULT now()
updated_at	TIMESTAMPTZ	NOT NULL, DEFAULT now()
deleted_at	TIMESTAMPTZ	NULL (soft delete)

Notes: `password_hash` is nullable because Google OAuth users authenticate without a local password (per FR-AUTH-02). Actual account-deletion requests (SRS-NFR-LEGAL-02) trigger a separate hard-delete routine that purges PII beyond just setting `deleted_at`.

4.2 `guest_sessions`

Column	Type	Constraints
id	UUID	PK, default <code>gen_random_uuid()</code>
mapped_user_id	UUID	FK → users.id, NULL (guests don't have a permanent user row; may later map to one on signup)
session_token	VARCHAR(255)	UNIQUE, NOT NULL
expires_at	TIMESTAMPTZ	NOT NULL
created_at	TIMESTAMPTZ	NOT NULL, DEFAULT now()

Notes: A scheduled job purges expired guest sessions and their associated `files`/`storage` records (satisfies FR-AUTH-07 and NFR-LEGAL-03).

4.3 files

Column	Type	Constraints
id	UUID	PK, default <code>gen_random_uuid()</code>
user_id	UUID	FK → users.id, NULL if guest upload, ON DELETE CASCADE
guest_session_id	UUID	FK → guest_sessions.id, NULL if registered-user upload, ON DELETE CASCADE
original_filename	VARCHAR(255)	NOT NULL
file_type	VARCHAR(10)	NOT NULL, CHECK IN ('audio','video')
mime_type	VARCHAR(100)	NOT NULL
file_size_bytes	BIGINT	NOT NULL, CHECK (file_size_bytes > 0)
storage_path	VARCHAR(500)	NOT NULL
status	VARCHAR(30)	NOT NULL, DEFAULT 'UPLOADED', CHECK IN ('UPLOADED','QUEUED','PROCESSING','READY_FOR_REVIEW','REGENERATING','COMPLETED')
duration_seconds	NUMERIC(10,2)	NULL (populated after audio extraction)
detected_language	VARCHAR(20)	NULL, CHECK IN ('english','hindi','hinglish','mixed')
created_at	TIMESTAMPTZ	NOT NULL, DEFAULT now()
updated_at	TIMESTAMPTZ	NOT NULL, DEFAULT now()
deleted_at	TIMESTAMPTZ	NULL (soft delete)

Constraint: `CHECK (user_id IS NOT NULL OR guest_session_id IS NOT NULL)` — a file must belong to either a registered user or a guest session, never neither.

4.4 detection_configs

Column	Type	Constraints
id	UUID	PK, default <code>gen_random_uuid()</code>
file_id	UUID	FK → files.id, NOT NULL, ON DELETE CASCADE
mode	VARCHAR(20)	NOT NULL, CHECK IN ('automatic','custom','combined')
preset_id	UUID	FK → presets.id, NULL
custom_words	JSONB	NULL (array of strings; used when mode is 'custom' or 'combined')
replacement_mode	VARCHAR(20)	NOT NULL, DEFAULT 'beep', CHECK IN ('beep','mute','custom_sound')
custom_sound_path	VARCHAR(500)	NULL (required if replacement_mode = 'custom_sound')
created_at	TIMESTAMPTZ	NOT NULL, DEFAULT now()

4.5 presets

Column	Type	Constraints
id	UUID	PK, default <code>gen_random_uuid()</code>
user_id	UUID	FK → users.id, NOT NULL, ON DELETE CASCADE
name	VARCHAR(100)	NOT NULL
word_list	JSONB	NOT NULL (array of strings)
created_at	TIMESTAMPTZ	NOT NULL, DEFAULT now()
updated_at	TIMESTAMPTZ	NOT NULL, DEFAULT now()

Constraint: `UNIQUE (user_id, name)` — a user cannot have two presets with the same name.

4.6 jobs

Column	Type	Constraints
id	UUID	PK, default <code>gen_random_uuid()</code>
file_id	UUID	FK → files.id, NOT NULL, ON DELETE CASCADE
job_type	VARCHAR(30)	NOT NULL, CHECK IN ('initial_processing','regeneration')
status	VARCHAR(20)	NOT NULL, DEFAULT 'PENDING', CHECK IN ('PENDING','RUNNING','COMPLETED','FAILED','RETRYING')
retry_count	INT	NOT NULL, DEFAULT 0
max_retries	INT	NOT NULL, DEFAULT 3
error_message	TEXT	NULL
started_at	TIMESTAMPTZ	NULL
completed_at	TIMESTAMPTZ	NULL
created_at	TIMESTAMPTZ	NOT NULL, DEFAULT now()

4.7 job_logs

Column	Type	Constraints
id	UUID	PK, default <code>gen_random_uuid()</code>
job_id	UUID	FK → jobs.id, NOT NULL, ON DELETE CASCADE
stage	VARCHAR(50)	NOT NULL (e.g., 'audio_extraction','asr','detection','muxing')
level	VARCHAR(10)	NOT NULL, CHECK IN ('INFO','WARN','ERROR')
message	TEXT	NOT NULL
created_at	TIMESTAMPTZ	NOT NULL, DEFAULT now()

4.8 **detections**

Column	Type	Constraints
id	UUID	PK, default <code>(gen_random_uuid())</code>
file_id	UUID	FK → files.id, NOT NULL, ON DELETE CASCADE
detected_text	TEXT	NOT NULL
language	VARCHAR(20)	NULL
start_time	NUMERIC(10,3)	NOT NULL (seconds, 3 decimal precision for ms accuracy)
end_time	NUMERIC(10,3)	NOT NULL
confidence_score	NUMERIC(4,3)	NOT NULL, CHECK (confidence_score BETWEEN 0 AND 1)
detection_source	VARCHAR(20)	NOT NULL, CHECK IN ('dictionary','custom_list','model_inferred','regex')
dictionary_id	UUID	FK → profanity_dictionary.id, NULL
replacement_mode	VARCHAR(20)	NOT NULL, DEFAULT 'beep', CHECK IN ('beep','mute','custom_sound')
is_user_added	BOOLEAN	NOT NULL, DEFAULT false
is_dismissed	BOOLEAN	NOT NULL, DEFAULT false
created_at	TIMESTAMPTZ	NOT NULL, DEFAULT now()
updated_at	TIMESTAMPTZ	NOT NULL, DEFAULT now()

Constraint: `CHECK (end_time > start_time)`.

4.9 **profanity_dictionary**

Column	Type	Constraints
id	UUID	PK, default <code>(gen_random_uuid())</code>
term	VARCHAR(100)	NOT NULL
normalized_term	VARCHAR(100)	NOT NULL (lowercased, normalized form used for matching)
language	VARCHAR(20)	NOT NULL, CHECK IN ('english','hindi','hinglish','universal')
severity	VARCHAR(10)	NOT NULL, DEFAULT 'medium', CHECK IN ('low','medium','high')
is_active	BOOLEAN	NOT NULL, DEFAULT true
added_by_admin_id	UUID	FK → users.id, NULL
created_at	TIMESTAMPTZ	NOT NULL, DEFAULT now()
updated_at	TIMESTAMPTZ	NOT NULL, DEFAULT now()

Constraint: `UNIQUE (normalized_term, language)`.

4.10 `exports`

Column	Type	Constraints
id	UUID	PK, default <code>gen_random_uuid()</code>
file_id	UUID	FK → files.id, NOT NULL, ON DELETE CASCADE
export_type	VARCHAR(20)	NOT NULL, CHECK IN ('video','audio','srt','txt','json')
storage_path	VARCHAR(500)	NOT NULL
file_size_bytes	BIGINT	NULL
created_at	TIMESTAMPTZ	NOT NULL, DEFAULT now()

4.11 `credits_ledger`

Column	Type	Constraints
id	UUID	PK, default <code>gen_random_uuid()</code>
user_id	UUID	FK → users.id, NOT NULL, ON DELETE CASCADE
amount	INT	NOT NULL (positive = credit added, negative = credit consumed)
transaction_type	VARCHAR(30)	NOT NULL, CHECK IN ('signup_bonus','file_processing','manual_adjustment','plan_purchase')
reference	VARCHAR(255)	NULL (e.g., related file_id or job_id as text)
created_at	TIMESTAMPTZ	NOT NULL, DEFAULT now()

Notes: A user's current balance is computed as `SUM(amount)` rather than stored redundantly, avoiding sync bugs; a materialized view or cached aggregate can be added later for performance if needed.

4.12 `admin_logs`

Column	Type	Constraints
id	UUID	PK, default <code>gen_random_uuid()</code>
admin_id	UUID	FK → users.id, NOT NULL
action_type	VARCHAR(50)	NOT NULL (e.g., 'dictionary_add','user_disable','dictionary_edit')
target_entity	VARCHAR(50)	NOT NULL (e.g., 'users','profanity_dictionary')
target_id	UUID	NULL
details	JSONB	NULL (before/after values or additional context)
created_at	TIMESTAMPTZ	NOT NULL, DEFAULT now()

4.13 **feedback**

Column	Type	Constraints
id	UUID	PK, default <code>gen_random_uuid()</code>
user_id	UUID	FK → users.id, NULL (feedback can be anonymous/guest)
category	VARCHAR(30)	NOT NULL, CHECK IN ('bug','feature_request','detection_accuracy','other')
message	TEXT	NOT NULL
status	VARCHAR(20)	NOT NULL, DEFAULT 'open', CHECK IN ('open','reviewed','resolved')
created_at	TIMESTAMPTZ	NOT NULL, DEFAULT now()

5. Relationships Summary

Relationship	Type	ON DELETE Behavior
users → files	1-to-many	CASCADE (deleting a user removes their files)
guest_sessions → files	1-to-many	CASCADE
files → jobs	1-to-many	CASCADE
files → detections	1-to-many	CASCADE
files → exports	1-to-many	CASCADE
files → detection_configs	1-to-1 (per processing run)	CASCADE
jobs → job_logs	1-to-many	CASCADE
users → presets	1-to-many	CASCADE
presets → detection_configs	1-to-many (referenced, optional)	SET NULL
profanity_dictionary → detections	1-to-many (optional match reference)	SET NULL
users → credits_ledger	1-to-many	CASCADE
users → admin_logs	1-to-many (as admin_id)	RESTRICT (preserve audit trail even if admin account is later modified)
users → feedback	1-to-many	SET NULL (preserve feedback even if user deletes account)

6. Indexing Strategy

Table	Index	Purpose
users	UNIQUE index on <code>email</code>	Fast login lookup; enforces uniqueness
users	Index on <code>role</code>	Fast admin-role filtering
files	Index on <code>user_id</code>	Dashboard "recent files" query (FR-DASH-01)
files	Index on <code>guest_session_id</code>	Guest cleanup job lookups
files	Index on <code>status</code>	Processing queue status queries (FR-DASH-05)
files	Composite index on <code>(user_id, created_at DESC)</code>	Efficient paginated history with sort (FR-DASH-02)
jobs	Index on <code>file_id</code>	Fetch job status per file
jobs	Index on <code>status</code>	Worker polling / admin queue monitoring
detections	Index on <code>file_id</code>	Timeline Editor load (FR-TLE-01) — most frequent query in the app
detections	Composite index on <code>(file_id, start_time)</code>	Ordered transcript rendering
profanity_dictionary	UNIQUE composite index on <code>(normalized_term, language)</code>	Fast matching lookup during detection; enforces no duplicates
profanity_dictionary	Index on <code>is_active</code>	Excludes disabled terms from matching queries
presets	Composite UNIQUE index on <code>(user_id, name)</code>	Enforce uniqueness, fast preset lookup per user
credits_ledger	Index on <code>user_id</code>	Balance calculation queries
admin_logs	Index on <code>admin_id</code>	Audit trail lookup per admin
admin_logs	Composite index on <code>(target_entity, target_id)</code>	"History of changes to this entity" queries
guest_sessions	Index on <code>expires_at</code>	Efficient cleanup job querying for expired sessions

7. Data Integrity & Constraints Summary

- **Referential integrity** enforced via foreign keys with explicit cascade rules (Section 5) — no orphaned `detections`, `jobs`, or `exports` rows can exist without a parent `files` row.
- **Check constraints** enforce valid enum-like values at the database level (e.g., `status`, `role`, `detection_source`) rather than relying solely on application-layer validation — defense in depth per SRS-NFR-SEC-03.
- **NOT NULL constraints** on all fields required for core business logic (e.g., `start_time`/`end_time` on `detections`) prevent incomplete records from corrupting

the Timeline Editor UI.

- **Row-Level Security (RLS)** policies (Supabase-native) applied on `files`, `detectors`, `presets`, `credits_ledger`, and `feedback` tables to ensure a user can only query/modify rows where `user_id` matches their authenticated JWT `sub` claim — this is the primary authorization enforcement mechanism at the data layer, complementing service-layer RBAC checks (per System Architecture Section 7.1).
 - `admin_logs` is **append-only** at the application layer (no UPDATE/DELETE exposed via API) to preserve audit trail integrity.
-

8. Migration & Versioning Strategy

- Schema changes managed via a migration tool (e.g., Alembic, given the FastAPI/Python backend) with version-controlled migration files committed to the repository.
 - Each migration is additive-first where possible (add nullable columns, backfill, then tighten constraints in a follow-up migration) to avoid downtime on the free-tier hosted database.
 - `profanity_dictionary` seed data (initial curated word list) is loaded via a dedicated seed script, separate from schema migrations, so it can be updated independently by admins without a deployment.
-

9. Next Document

Document 7: Folder Structure — the complete production-level project directory structure (frontend, backend, AI, database, configs, tests, docs) implementing the architecture and data model defined above.

End of Database Design Document

Folder Structure Document

SentinelCut — AI-Powered Audio/Video Profanity Censorship Platform

Document Version: 1.1 (Updated: Frontend switched to JavaScript, no TypeScript)

Status: Final for Implementation **Based On:** System Architecture v1.0, Database Design v1.0, Tech Stack Justification v1.0

1. Purpose

This document defines the complete, production-level repository structure for SentinelCut. It reflects the layered architecture (API layer, service layer, repository layer, AI pipeline) and the FastAPI + Next.js + PostgreSQL/Supabase stack established in prior documents. The structure is designed for a **monorepo** — a single repository containing frontend, backend, AI pipeline, and shared documentation — which is the right choice for a small academic team needing simple version control and coordinated deployments across tightly coupled layers.

2. Top-Level Repository Structure

```
sentinelcut/
├── frontend/           # Next.js web application
├── backend/           # FastAPI backend service
├── ai-pipeline/       # AI/ML processing modules (used by backend
workers)
├── database/          # Migrations, seed data, schema docs
├── infra/             # Deployment configs, Docker, CI/CD
├── docs/              # All project documentation (PRD, SRS,
etc.)
├── scripts/          # Utility/dev scripts
├── tests/             # Cross-cutting/integration/E2E tests
├── .github/           # GitHub Actions workflows
├── .env.example       # Sample environment variables (no secrets)
├── docker-compose.yml # Local dev orchestration
├── README.md
├── CHANGELOG.md
├── CONTRIBUTING.md
├── LICENSE
└── .gitignore
```

3. Frontend Structure (`frontend/`)

Update (v1.1): Per project preference, the frontend uses **Next.js with JavaScript** (not TypeScript). All `.tsx`/`.ts` files below are `.jsx`/`.js`, there is no `tsconfig.json`, and type safety is handled at runtime via **PropTypes** (for component props) and **Zod** (for API/form validation) instead of compile-time types. This is a fully supported, production-viable Next.js configuration — plenty of real-world Next.js apps ship in JS-only mode — and it removes a layer of tooling complexity, which is a reasonable trade-off for a small team optimizing for comfort and delivery speed over static type-checking.


```

UI
|   |   |— dashboard/           # Recent files, credits widget,
storage widget
|   |   |— timeline-editor/     # Transcript view, waveform,
detection cards
|   |   |   |— TranscriptView.jsx
|   |   |   |— DetectionHighlight.jsx
|   |   |   |— PlaybackControls.jsx
|   |   |   |— EditActionsToolbar.jsx
|   |   |— admin/               # Admin-specific tables/forms
|   |   |   |— shared/          # Navbar, Footer, Sidebar, Toasts
|   |   |— hooks/
|   |   |   |— useAuth.js
|   |   |   |— useFileUpload.js
|   |   |   |— useJobStatus.js   # WebSocket/polling hook
|   |   |   |— useTimelineEditor.js
|   |   |— lib/
|   |   |   |— api-client.js     # Centralized fetch/axios wrapper
|   |   |   |— supabase-client.js
|   |   |   |— constants.js
|   |   |   |— validators.js     # Client-side schema validation
(zod)
|   |— store/                   # Global state (Zustand/Redux, if
used)
|   |   |— authStore.js
|   |   |   |— editorStore.js
|   |— schemas/                 # Zod schemas (replaces TS types/
folder)
|   |   |— fileSchema.js
|   |   |— detectionSchema.js
|   |   |— userSchema.js
|   |   |   |— apiSchema.js
|   |   |— styles/
|   |   |   |— globals.css
|— tests/
|   |— unit/
|   |— component/
|   |   |— e2e/                 # Playwright/Cypress specs
|— .eslintrc.json
|— .prettierrc
|— jsconfig.json                # Path aliases (e.g.,
"@/components/*") without TS
|— next.config.js
|— tailwind.config.js

```

Rationale: Route groups ((marketing), (auth), (app), (admin)) map directly to the User Journeys defined in the PRD and allow independent layouts (e.g., admin has its own sidebar) without URL prefix pollution. The (timeline-editor/) component folder is deliberately isolated given it's flagged as the most technically demanding UI piece in the System Architecture document.

Notes on the JS-only setup:

- (jsconfig.json) replaces (tsconfig.json) — it still gives you clean import aliases (e.g., (import Button from '@components/ui/Button')) and enables editor IntelliSense/autocomplete in VS Code without requiring TypeScript itself.
 - The former (types/) folder is replaced by (schemas/), containing **Zod** schemas — these serve double duty as runtime validation (for forms and API responses) and as the de facto documentation of data shapes, which is what the TypeScript types were mainly doing anyway in a project this size.
 - Component prop validation (optional but recommended) can be added incrementally via the (prop-types) package on any component where it adds clarity, without an all-or-nothing TypeScript migration.
 - If the team later wants incremental type safety without a full rewrite, Next.js supports JSDoc-based type checking ((// @ts-check) comments) as a lightweight middle ground — noted here for future reference only, not part of the current plan.
-

4. Backend Structure (`backend/`)

```
backend/
├─ app/
│   │   └─ main.py                # FastAPI app entrypoint
│   │   └─ config.py             # Settings (env-based, Pydantic
Settings)
│   │   └─ dependencies.py        # Shared FastAPI dependencies (auth,
db session)
│   │   └─ api/
│   │       │   └─ v1/
│   │       │       │   └─ routes/
│   │       │       │       │   └─ auth_routes.py
│   │       │       │       │   └─ file_routes.py
│   │       │       │       │   └─ detection_routes.py
│   │       │       │       │   └─ timeline_routes.py
│   │       │       │       │   └─ export_routes.py
│   │       │       │       │   └─ dashboard_routes.py
│   │       │       │       │   └─ admin_routes.py
│   │       │       │       │   └─ preset_routes.py
│   │       │       │       │   └─ feedback_routes.py
│   │       │       │       │   └─ websocket_routes.py
│   │       │       │       └─ router.py    # Aggregates all v1 routers
│   │       │       └─ deps/
│   │       │           │   └─ auth_deps.py    # JWT validation dependency
│   │       │           │   └─ rbac_deps.py    # Role-check dependency
│   │       │   └─ schemas/                  # Pydantic request/response models
│   │       │       │   └─ auth_schema.py
│   │       │       │   └─ file_schema.py
│   │       │       │   └─ detection_schema.py
│   │       │       │   └─ job_schema.py
│   │       │       │   └─ preset_schema.py
│   │       │       │   └─ admin_schema.py
│   │       │   └─ models/                  # SQLAlchemy ORM models (mirrors
Database Design doc)
│   │       │       │   └─ user.py
│   │       │       │   └─ guest_session.py
│   │       │       │   └─ file.py
│   │       │       │   └─ detection.py
│   │       │       │   └─ detection_config.py
│   │       │       │   └─ job.py
│   │       │       │   └─ job_log.py
│   │       │       │   └─ preset.py
│   │       │       │   └─ profanity_dictionary.py
│   │       │       └─ export.py
```

```

| | | └─ credits_ledger.py
| | | └─ admin_log.py
| | | └─ feedback.py
| | └─ repositories/                                # Repository Pattern (data access
layer)
| | | └─ base_repository.py                        # Generic CRUD base class
| | | └─ user_repository.py
| | | └─ file_repository.py
| | | └─ detection_repository.py
| | | └─ job_repository.py
| | | └─ preset_repository.py
| | | └─ dictionary_repository.py
| | | └─ admin_log_repository.py
| | └─ services/                                    # Business logic layer
| | | └─ auth_service.py
| | | └─ file_service.py
| | | └─ job_orchestrator_service.py
| | | └─ detection_config_service.py
| | | └─ timeline_edit_service.py
| | | └─ export_service.py
| | | └─ admin_service.py
| | | └─ credits_service.py
| | └─ core/
| | | └─ security.py                                # JWT encode/decode, password
hashing
| | | └─ exceptions.py                            # Custom exception classes
| | | └─ logging_config.py
| | | └─ constants.py
| | └─ db/
| | | └─ session.py                                # SQLAlchemy engine/session setup
| | | └─ base.py                                    # Declarative base import aggregator
| | └─ workers/
| | | └─ celery_app.py                            # Celery/RQ app configuration
| | | └─ tasks/
| | | | └─ processing_task.py                    # Orchestrates the full AI pipeline
call
| | | | └─ regeneration_task.py
| | | | └─ cleanup_task.py                        # Guest session/expired file purging
| | | | └─ status_publisher.py                   # Publishes job status to Redis for
WebSocket relay
└─ tests/
| | └─ unit/
| | | └─ test_services/
| | | └─ test_repositories/
| └─ integration/

```

```
| | | └─ test_api_routes/
| | | └─ test_db/
| | └─ conftest.py
└─ alembic/                                # DB migrations (see database/ for
canonical schema docs)
| | └─ versions/
| | └─ env.py
└─ requirements.txt
└─ requirements-dev.txt
└─ Dockerfile
└─ pytest.ini
└─ .flake8
└─ README.md
```

Rationale: The `repositories/` and `services/` split directly implements the Repository Pattern and layered service architecture mandated in the Coding Standards (Product Vision brief) and reflected in the System Architecture component diagram.

`workers/tasks/` maps one-to-one to the async job types defined in the `jobs.job_type` enum in the Database Design document.

5. AI Pipeline Structure (ai-pipeline/)

```
ai-pipeline/
├── sentinelcut_ai/
│   ├── __init__.py
│   ├── asr/
│   │   ├── __init__.py
│   │   ├── speech_recognition_service.py # Interface (abstract base)
│   │   ├── faster_whisper_engine.py      # Concrete implementation
│   │   └── forced_alignment.py           # Word-level timestamp alignment
│   ├── language_detection/
│   │   ├── __init__.py
│   │   └── language_detector.py
│   ├── profanity_detection/
│   │   ├── __init__.py
│   │   ├── profanity_detection_service.py # Interface (abstract base)
│   │   └── normalizer.py                 # Repeated-char, leetspeak, Unicode
normalization
│   ├── dictionary_matcher.py             # Dictionary + regex layer
│   └── context_scorer.py                 # Embedding/IndicBERT-based scoring
layer
│   └── confidence_aggregator.py
├── audio_processing/
│   ├── __init__.py
│   ├── audio_extractor.py                # FFmpeg audio extraction wrapper
│   └── audio_editor.py                   # PyDub beep/mute/custom-sound
application
│   ├── media_muxer.py                    # FFmpeg re-mux wrapper
│   ├── pipeline/
│   │   ├── __init__.py
│   │   └── processing_pipeline.py        # Orchestrates full pipeline
(Section 5 of Architecture doc)
│   ├── pipeline_config.py
│   ├── data/
│   │   ├── dictionaries/
│   │   │   ├── english_profanity.json
│   │   │   ├── hindi_profanity.json
│   │   │   └── hinglish_slang.json
│   │   ├── embeddings/
│   │   │   └── seed_profanity_examples.json
│   └── utils/
│       ├── file_utils.py
│       ├── timing_utils.py
│       └── logging_utils.py
└── notebooks/                          # Experimentation (model
```

```
evaluation, accuracy testing)
|   ├── asr_benchmark.ipynb
|   └── profanity_detection_eval.ipynb
└── tests/
    ├── test_normalizer.py
    ├── test_dictionary_matcher.py
    ├── test_context_scorer.py
    └── test_pipeline_integration.py
└── requirements.txt
└── Dockerfile
└── README.md
```

Rationale: This module is structured to be importable directly by `backend/app/workers/tasks/processing_task.py`, keeping AI logic testable and swappable independent of the API layer — directly implementing the "Extensibility Points" defined in the System Architecture document (Section 7.3).

6. Database Structure (`database/`)

```
database/
├── migrations/                                # Mirrors backend/alembic if kept
separate for clarity
|   └── (versioned migration files)
├── seeds/
|   ├── seed_profanity_dictionary.py
|   └── seed_dev_users.py
├── schema/
|   ├── er_diagram.md                        # Reference copy of ER diagram
|   └── schema_reference.sql                 # Human-readable full DDL reference
└── README.md
```

Note: In practice, Alembic migrations physically live under `backend/alembic/` (Section 4) since they're tightly coupled to the SQLAlchemy models; `database/` serves as the canonical, framework-agnostic reference copy of the schema for documentation and onboarding purposes.

7. Infrastructure & Deployment Structure (**infra/**)

```
infra/
├── docker/
│   ├── frontend.Dockerfile
│   ├── backend.Dockerfile
│   └── worker.Dockerfile
├── docker-compose.dev.yml          # Local dev: backend + worker +
redis + postgres
├── vercel.json                    # Frontend deployment config
├── render.yaml                   # Backend/worker deployment config
(Render Blueprint)
├── supabase/
│   ├── config.toml
│   └── rls_policies.sql          # Row-Level Security policy
definitions
└── env/
    ├── .env.staging.example
    └── .env.production.example
```

8. CI/CD Structure (**.github/**)

```
.github/
├── workflows/
│   ├── frontend-ci.yml          # Lint, type-check, build, test on
PR
│   ├── backend-ci.yml          # Lint, test, coverage on PR
│   └── ai-pipeline-ci.yml       # Unit tests for detection/ASR
modules
│   └── deploy.yml              # Deploy on merge to main
├── ISSUE_TEMPLATE/
│   ├── bug_report.md
│   └── feature_request.md
└── PULL_REQUEST_TEMPLATE.md
```

9. Documentation Structure (`docs/`)

```
docs/  
├─ 01_Product_Vision.md  
├─ 02_PRD.md  
├─ 03_SRS.md  
├─ 04_Tech_Stack_Justification.md  
├─ 05_System_Architecture.md  
├─ 06_Database_Design.md  
├─ 07_Folder_Structure.md  
├─ 08_API_Design.md  
├─ 09_AI_Pipeline.md  
├─ 10_UIUX_Design.md  
├─ 11_Development_Roadmap.md  
├─ 12_Implementation_Notes.md  
├─ 13_Testing/  
│   ├─ Test_Plan.md  
│   ├─ Test_Cases.md  
│   └─ QA_Checklist.md  
├─ 14_Deployment_Guide.md  
├─ 15_Future_Scope.md  
├─ manuals/  
│   ├─ User_Manual.md  
│   ├─ Admin_Manual.md  
│   └─ Maintenance_Guide.md  
├─ academic/  
│   ├─ College_Report.md  
│   ├─ IEEE_Research_Paper.md  
│   ├─ Project_Synopsis.md  
│   ├─ Project_Proposal.md  
│   ├─ Weekly_Progress_Reports/  
│   ├─ Final_Viva_Questions.md  
│   ├─ Demo_Script.md  
│   ├─ Poster_Content.md  
│   └─ Presentation_Outline.md  
├─ standards/  
│   ├─ Coding_Standards.md  
│   ├─ Git_Workflow.md  
│   ├─ Security_Guidelines.md  
│   └─ Versioning_Strategy.md  
├─ legal/  
│   ├─ Privacy_Policy.md  
│   └─ Terms_and_Conditions.md  
└─ misc/  
    └─ Resume_Description.md
```

```
|— LinkedIn_Description.md
|— GitHub_Description.md
|— Release_Notes.md
|— Roadmap.md
```

Rationale: This maps one-to-one to the full deliverables list from the original project brief, ensuring nothing from the master scope is structurally lost or homeless.

10. Scripts (`scripts/`)

```
scripts/
|— setup_dev_env.sh          # One-command local environment
bootstrap
|— run_migrations.sh
|— seed_database.sh
|— generate_test_media.py    # Creates sample audio/video files
with known profanity for testing
|— benchmark_asr.py          # Runs ASR accuracy benchmarks
against labeled test set
|— benchmark_detection.py    # Runs profanity detection
precision/recall benchmarks
```

11. Root-Level Configuration Files

File	Purpose
<code>.env.example</code>	Documents all required environment variables without real secrets (DB URL, Supabase keys, Redis URL, JWT secret, email API key)
<code>docker-compose.yml</code>	Orchestrates local development: Postgres, Redis, backend, worker, frontend
<code>README.md</code>	Project overview, setup instructions, links to <code>docs/</code>
<code>CHANGELOG.md</code>	Version history following Keep a Changelog format
<code>CONTRIBUTING.md</code>	Contribution guide referencing <code>docs/standards/Git_Workflow.md</code>
<code>.gitignore</code>	Excludes <code>node_modules/</code> , <code>__pycache__/</code> , <code>.env</code> , media upload test fixtures, etc.

12. Directory-to-Architecture Traceability

Architecture Component (from System Architecture doc)	Folder Location
Next.js Web App	frontend/
FastAPI RESTAPI Service	backend/app/api/
Auth Middleware	backend/app/api/deps/auth_deps.py
WebSocket/Status Service	backend/app/api/v1/routes/websocket_routes.py, backend/app/workers/status_publisher.py
Celery/RQ Worker Pool	backend/app/workers/
SpeechRecognitionService	ai-pipeline/sentinelcut_ai/asr/
ProfanityDetectionService	ai-pipeline/sentinelcut_ai/profanity_detection/
AudioEditingService / MediaMuxService	ai-pipeline/sentinelcut_ai/audio_processing/
Repository Layer	backend/app/repositories/
PostgreSQL Schema	database/schema/, backend/alembic/
Deployment Configs	infra/

13. Next Document

Document 8: API Design — full REST API specification (endpoints, request/response schemas, status codes, WebSocket contract) implementing the routes scaffolded in backend/app/api/v1/routes/ above.

End of Folder Structure Document

API Design Document

SentinelCut — AI-Powered Audio/Video Profanity Censorship Platform

Document Version: 1.0 **Status:** Final for Implementation **Based On:** System Architecture v1.0, Database Design v1.0, Folder Structure v1.1

1. Purpose

This document specifies the complete REST API and WebSocket contract for SentinelCut's backend (FastAPI), implementing the routes scaffolded under `backend/app/api/v1/routes/`. It defines every endpoint's method, path, authentication requirement, request/response schema, and status codes, so frontend and backend development can proceed in parallel against a fixed contract.

2. API Conventions

2.1 Base URL & Versioning

```
Production: https://api.sentinelcut.app/api/v1
Local Dev:  http://localhost:8000/api/v1
```

All endpoints are versioned under `/api/v1`. Breaking changes will be introduced under `/api/v2` rather than mutating v1 in place.

2.2 Authentication

- Authenticated endpoints require a `Authorization: Bearer <jwt>` header.
- Guest-mode endpoints accept either a Guest Session token (same header format) or none, as noted per-endpoint.
- Admin endpoints require a valid JWT with `role: "admin"` claim; enforced via the `rbac_deps.py` dependency (per Folder Structure doc).

2.3 Standard Response Envelope

Success:

```
json
{
  "success": true,
  "data": { },
  "meta": { }
}
```

Error:

json

```
{
  "success": false,
  "error": {
    "code": "VALIDATION_ERROR",
    "message": "file_size_bytes exceeds maximum allowed size",
    "details": { "max_allowed_mb": 500 }
  }
}
```

2.4 Standard HTTP Status Codes Used

Code	Meaning	Usage
200	OK	Successful GET/PATCH/PUT
201	Created	Successful POST creating a resource
202	Accepted	Async job accepted (processing queued)
204	No Content	Successful DELETE
400	Bad Request	Validation failure
401	Unauthorized	Missing/invalid/expired token
403	Forbidden	Valid token, insufficient role/ownership
404	Not Found	Resource doesn't exist or not owned by requester
409	Conflict	Duplicate resource (e.g., preset name)
413	Payload Too Large	File exceeds max upload size
415	Unsupported Media Type	Invalid file format
422	Unprocessable Entity	Semantically invalid request (FastAPI default validation)
429	Too Many Requests	Rate limit exceeded
500	Internal Server Error	Unhandled server error
503	Service Unavailable	Processing queue/worker unavailable

2.5 Pagination

List endpoints accept `?page=1&page_size=20` and return:

json

```
"meta": {  
  "page": 1,  
  "page_size": 20,  
  "total_items": 143,  
  "total_pages": 8  
}
```

2.6 Rate Limiting

Per SRS-NFR-SEC-06, auth and upload endpoints are rate-limited (e.g., 10 requests/minute per IP for `/auth/*`, 20 requests/hour for `/files/upload` on guest sessions). Exceeding limits returns `429` with a `Retry-After` header.

3. Auth Routes (`auth_routes.py`)

3.1 `POST /auth/signup`

Auth: None **Request:**

json

```
{ "email": "user@example.com", "password": "SecurePass1!", "display_name": "Rohan" }
```

Response `201`:

json

```
{ "success": true, "data": { "user_id": "uuid", "email": "user@example.com", "acc
```

Errors: `400` (weak password), `409` (email already registered)

3.2 `POST /auth/login`

Auth: None **Request:** `{ "email": "...", "password": "..." }` **Response** `200`: `{ "access_token": "jwt", "refresh_token": "jwt", "user": {...} }` **Errors:** `401` (invalid credentials)

3.3 `POST /auth/google`

Auth: None **Request:** `{ "id_token": "google-issued-token" }` **Response** `200`: Same shape as login. Creates a new `users` record on first login (`auth_provider = 'google'`). **Errors:** `401` (invalid Google token)

3.4 POST /auth/forgot-password

Auth: None **Request:** { "email": "user@example.com" } **Response** (200): { "message": "If an account exists, a reset link has been sent." } (always 200 to avoid email enumeration)

3.5 POST /auth/reset-password

Auth: None (uses reset token from emailed link) **Request:** { "reset_token": "...", "new_password": "..." } **Response** (200): { "message": "Password updated successfully." } **Errors:** (400) (expired/invalid token)

3.6 POST /auth/guest-session

Auth: None **Response** (201): { "guest_token": "jwt", "expires_at": "2026-08-04T10:00:00Z" }

3.7 POST /auth/refresh

Auth: Refresh token in body **Request:** { "refresh_token": "jwt" } **Response** (200): { "access_token": "jwt" } **Errors:** (401) (expired/invalid refresh token)

3.8 POST /auth/logout

Auth: Required **Response** (204)

4. File Routes (file_routes.py)

4.1 POST /files/upload

Auth: Required (registered user or guest token) **Request:** multipart/form-data — field `file` (binary), `original_filename` (string) **Response** (201):

```
json
{
  "data": {
    "file_id": "uuid",
    "original_filename": "podcast_ep12.mp4",
    "file_type": "video",
    "file_size_bytes": 52428800,
    "status": "UPLOADED"
  }
}
```

Errors: (413) (too large), (415) (unsupported format), (401)

4.2 GET /files/{file_id}

Auth: Required (must own the file, or admin) **Response** (200): Full file record including current `status`, `duration_seconds`, `detected_language`. **Errors:** (404) (not found/not

owned)

4.3 GET /files

Auth: Required **Query params:** (?status=&search=&page=&page_size=&sort=-created_at

Response (200): Paginated list of the user's files (implements FR-DASH-01/02).

4.4 DELETE /files/{file_id}

Auth: Required (owner or admin) **Response** (204) (soft delete — sets deleted_at)

Errors: (404)

4.5 POST /files/{file_id}/process

Auth: Required **Request:**

```
json

{
  "detection_mode": "combined",
  "custom_words": ["badword1", "badword2"],
  "preset_id": "uuid|null",
  "replacement_mode": "beep"
}
```

Response (202): { "job_id": "uuid", "status": "QUEUED" } — enqueues the initial_processing job. **Errors:** (400) (invalid detection_mode combination), (404)

4.6 POST /files/{file_id}/regenerate

Auth: Required **Response** (202): { "job_id": "uuid", "status": "QUEUED" } — enqueues a regeneration job using current detections state. **Errors:** (404), (409) (file not in a regenerable status)

5. Detection Routes (detection_routes.py)

5.1 GET /files/{file_id}/detections

Auth: Required (owner) **Query params:** (?min_confidence=0.5) **Response** (200):

```
json
```

```
{
  "data": [
    {
      "id": "uuid",
      "detected_text": "bakchodi",
      "language": "hindi",
      "start_time": 12.450,
      "end_time": 13.100,
      "confidence_score": 0.94,
      "detection_source": "dictionary",
      "replacement_mode": "beep",
      "is_user_added": false,
      "is_dismissed": false
    }
  ]
}
```

5.2 POST /files/{file_id}/detections

Auth: Required (owner) — manually add a detection (FR-TLE-04) **Request:** `{ "detected_text": "...", "start_time": 45.2, "end_time": 45.8, "replacement_mode": "mute" }` **Response** (201): Created detection object with `is_user_added: true`. **Errors:** (400) (end_time <= start_time)

5.3 PATCH /detections/{detection_id}

Auth: Required (owner of parent file) **Request (any subset):** `{ "is_dismissed": true }` or `{ "replacement_mode": "custom_sound" }` or `{ "start_time": 45.1, "end_time": 45.9 }` **Response** (200): Updated detection object. **Errors:** (404)

5.4 PATCH /files/{file_id}/detections/batch

Auth: Required — bulk edit endpoint used by the Timeline Editor "save all changes" action **Request:**

json

```
{ "updates": [ { "id": "uuid", "is_dismissed": true }, { "id": "uuid", "replaceme
```

Response (200): `{ "updated_count": 2 }`

5.5 DELETE /detections/{detection_id}

Auth: Required (owner) **Response** (204)

6. Timeline Routes (`timeline_routes.py`)

6.1 `GET /files/{file_id}/transcript`

Auth: Required (owner) **Response** `200`:

```
json
{
  "data": {
    "full_text": "This movie is fucking amazing...",
    "segments": [
      { "text": "This movie is", "start": 0.0, "end": 1.2 },
      { "text": "fucking", "start": 1.2, "end": 1.6, "detection_id": "uuid" },
      { "text": "amazing", "start": 1.6, "end": 2.1 }
    ]
  }
}
```

6.2 `POST /files/{file_id}/editor/autosave`

Auth: Required — implements FR-TLE-08 (periodic autosave of draft edits) **Request:** `{ "draft_state": { } }` (opaque JSON blob of in-progress editor state) **Response** `200`: `{ "saved_at": "2026-08-03T10:15:00Z" }`

6.3 `GET /files/{file_id}/editor/autosave`

Auth: Required — recovers a draft after a crash/disconnect **Response** `200`: `{ "draft_state": { }, "saved_at": "..." }` or `404` if no draft exists

7. Export Routes (`export_routes.py`)

7.1 `GET /files/{file_id}/exports`

Auth: Required (owner) **Response** `200`: List of available export artifacts (`video`, `audio`, `srt`, `txt`, `json`) with generation status.

7.2 `GET /files/{file_id}/exports/{export_type}/download`

Auth: Required (owner) **Path param** `export_type`: one of `video` | `audio` | `srt` | `txt` | `json` **Response** `200`: `{ "download_url": "https://signed-url...", "expires_at": "..." }` — a short-lived signed Supabase Storage URL (per Architecture doc Section 7.1), not a proxied file stream. **Errors:** `404` (export not yet generated), `409` (file processing not complete)

8. Dashboard Routes (`dashboard_routes.py`)

8.1 `GET /dashboard/summary`

Auth: Required **Response** `200`:

```
json
{
  "data": {
    "recent_files": [ /* last 5 files */ ],
    "credits_remaining": 42,
    "storage_used_bytes": 1073741824,
    "storage_limit_bytes": 5368709120,
    "active_jobs": [ { "job_id": "uuid", "file_id": "uuid", "status": "PROCESSING" } ]
  }
}
```

9. Preset Routes (`preset_routes.py`)

9.1 `GET /presets`

Auth: Required — list the user's saved custom word-list presets.

9.2 `POST /presets`

Auth: Required **Request:** `{ "name": "My Podcast Filter", "word_list": ["word1", "word2"] }` **Response** `201` **Errors:** `409` (duplicate name for this user)

9.3 `PATCH /presets/{preset_id}`

Auth: Required (owner) **Request:** `{ "word_list": [...] }`

9.4 `DELETE /presets/{preset_id}`

Auth: Required (owner) **Response** `204`

10. Admin Routes (`admin_routes.py`)

All endpoints require `role: admin`; all mutating actions write to `admin_logs` (per Database Design Section 4.12).

10.1 `GET /admin/users`

Paginated list with filters `?status=active|disabled&search=`.

10.2 `PATCH /admin/users/{user_id}`

Request: `{ "status": "disabled" }` **Response** `200`

10.3 GET /admin/dictionary

Paginated list of (profanity_dictionary) entries, filterable by (? language=hindi&is_active=true).

10.4 POST /admin/dictionary

Request: { "term": "example", "language": "hindi", "severity": "high" } **Response** (201) **Errors:** (409) (duplicate normalized_term + language)

10.5 PATCH /admin/dictionary/{entry_id}

Request: { "is_active": false } or { "severity": "low" }

10.6 DELETE /admin/dictionary/{entry_id}

Response (204)

10.7 GET /admin/files

Paginated list of all files across all users, with filters (?status=&user_id=).

10.8 GET /admin/analytics

Response (200):

```
json
{
  "data": {
    "uploads_per_day": [ { "date": "2026-08-01", "count": 34 } ],
    "avg_processing_time_seconds": 87.4,
    "detection_stats": { "total_detections": 15234, "by_language": { "hindi": 623
  }
}
```

10.9 GET /admin/logs

Paginated (admin_logs) audit trail, filterable by (?admin_id=&target_entity=).

10.10 GET /admin/feedback

Paginated (feedback) list, filterable by (?status=open&category=bug).

10.11 PATCH /admin/feedback/{feedback_id}

Request: { "status": "resolved" }

11. Feedback Routes (`feedback_routes.py`)

11.1 `POST /feedback`

Auth: Optional (works for guest and registered users) **Request:** `{ "category": "detection_accuracy", "message": "Missed a Hinglish slang word." }` **Response** `201`

12. WebSocket Contract (`websocket_routes.py`)

12.1 Connection

```
WSS /ws/jobs/{job_id}?token=<jwt>
```

Authenticated via query-param JWT (validated on connection handshake). Rejects with close code `4401` if the token is invalid or the job doesn't belong to the requester.

12.2 Server → Client Messages

Status update:

json

```
{ "type": "status_update", "job_id": "uuid", "status": "PROCESSING", "stage": "asr" }
```

Completion:

json

```
{ "type": "job_completed", "job_id": "uuid", "file_id": "uuid", "status": "READY" }
```

Failure:

json

```
{ "type": "job_failed", "job_id": "uuid", "error_message": "ASR engine timeout", }
```

12.3 Client → Server Messages

json

```
{ "type": "ping" }
```

Server responds `{ "type": "pong" }` — used for connection keep-alive; no business logic accepted over the socket (all mutations go through REST, per the Architecture doc's separation of the WebSocket as a read-only status relay).

12.4 Fallback

If a client cannot establish a WebSocket connection (e.g., restrictive network), the frontend falls back to polling `GET /files/{file_id}` every 5 seconds (satisfies SRS-NFR-PERF-03's 5-second status latency requirement either way).

13. Error Code Reference

Code	HTTP Status	Meaning
VALIDATION_ERROR	400/422	Request body failed schema validation
AUTH_INVALID_CREDENTIALS	401	Login failed
AUTH_TOKEN_EXPIRED	401	JWT expired, refresh required
FORBIDDEN	403	Authenticated but not authorized for this resource
NOT_FOUND	404	Resource doesn't exist or isn't owned by requester
DUPLICATE_RESOURCE	409	e.g., duplicate preset name, duplicate dictionary term
FILE_TOO_LARGE	413	Exceeds configured max upload size
UNSUPPORTED_FORMAT	415	File format not in the supported list
RATE_LIMITED	429	Too many requests
PROCESSING_ERROR	500	AI pipeline failure after retries exhausted
QUEUE_UNAVAILABLE	503	Redis/worker pool unreachable

14. Endpoint Summary Table

Method	Path	Auth	Purpose
POST	/auth/signup	None	Register
POST	/auth/login	None	Login
POST	/auth/google	None	Google OAuth login
POST	/auth/forgot-password	None	Request reset link
POST	/auth/reset-password	None	Complete reset
POST	/auth/guest-session	None	Start guest session
POST	/auth/refresh	Refresh token	Refresh access token
POST	/auth/logout	Required	Logout
POST	/files/upload	Required	Upload media
GET	/files/{id}	Required	Get file detail
GET	/files	Required	List files
DELETE	/files/{id}	Required	Soft-delete file
POST	/files/{id}/process	Required	Start AI processing
POST	/files/{id}/regenerate	Required	Regenerate after edits
GET	/files/{id}/detections	Required	List detections
POST	/files/{id}/detections	Required	Add manual detection

Method	Path	Auth	Purpose
PATCH	/detections/{id}	Required	Edit a detection
PATCH	/files/{id}/detections/batch	Required	Bulk-edit detections
DELETE	/detections/{id}	Required	Remove a detection
GET	/files/{id}/transcript	Required	Get aligned transcript
POST	/files/{id}/editor/autosave	Required	Autosave draft edits
GET	/files/{id}/editor/autosave	Required	Recover draft
GET	/files/{id}/exports	Required	List export artifacts
GET	/files/{id}/exports/{type}/download	Required	Get signed download URL
GET	/dashboard/summary	Required	Dashboard data
GET/POST/PATCH/DELETE	/presets	Required	Manage presets
GET/PATCH	/admin/users	Admin	Manage users
GET/POST/PATCH/DELETE	/admin/dictionary	Admin	Manage profanity dictionary
GET	/admin/files	Admin	View all files
GET	/admin/analytics	Admin	Platform analytics
GET	/admin/logs	Admin	Audit trail

Method	Path	Auth	Purpose
GET/PATCH	/admin/feedback	Admin	Manage feedback
POST	/feedback	Optional	Submit feedback
WSS	/ws/jobs/{job_id}	Required	Real-time job status

15. Next Document

Document 9: AI Pipeline — detailed technical design of the `ai-pipeline/` module internals (ASR invocation, normalization rules, hybrid detection scoring logic, audio editing algorithms) that the `POST /files/{id}/process` and `/regenerate` endpoints trigger via the job queue.

End of API Design Document

Document Version: 1.0 Status: Final for Implementation Based On: Tech Stack Justification v1.0, System Architecture v1.0, Database Design v1.0, Folder Structure v1.1, API Design v1.0

1. Purpose

This document specifies the internal technical design of the `ai-pipeline/sentinelcut_ai/` module — the code that actually runs inside the Celery/RQ worker processes when a `POST /files/{id}/process` or `POST /files/{id}/regenerate` request is dequeued. Every prior document justified *what* to build and *why* (Tech Stack), *where it sits* in the system (Architecture), *what it persists* (Database Design), and *what triggers it* (API Design). This document defines *how each stage actually works*: exact algorithms, interfaces, data structures, and pseudocode, so a developer can implement `ai-pipeline/` directly from this spec without further design decisions.

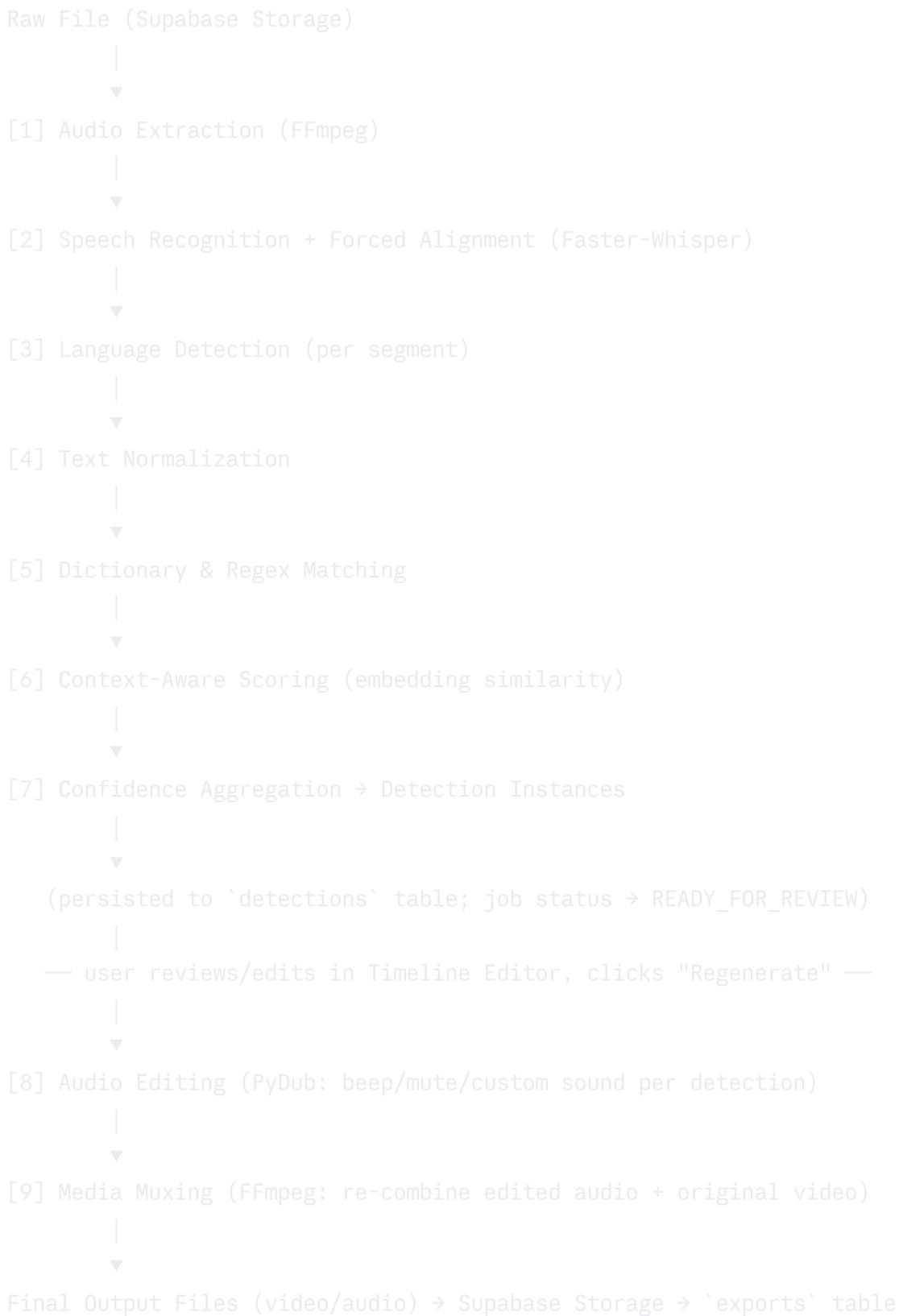
This document is the direct technical precursor to implementation and is written for the engineer(s) who will build the modules under `ai-pipeline/sentinelcut_ai/`.

2. Scope

In scope: the nine internal stages that run inside `processing_pipeline.py`, from raw file to persisted detection instances plus the audio-editing/mux stage that produces the censored output.

Out of scope (covered elsewhere): job queueing/retry orchestration at the Celery level (System Architecture §4.3/§6.1), database persistence mechanics (Database Design), and the REST/WebSocket contract that surfaces pipeline status to the frontend (API Design §12).

3. Pipeline Overview



Stages 1-7 run during `initial_processing` jobs. Stage 4-7 also re-run in isolation if the user manually adds a detection with free text (validated against the normalizer). Stages 8-9 run during both `initial_processing` (first pass, using whatever detections exist at that point) and `regeneration` jobs (using the current, user-edited detection set — this is the "Regenerate" action from PRD §4.7/FR-TLE-08).

Every stage is implemented as a class with a single well-defined entrypoint method, registered behind an interface where the System Architecture document (§7.3) already committed to swappability (`SpeechRecognitionService`, `ProfanityDetectionService`). This document defines those interfaces concretely.

4. Stage 1 — Audio Extraction

Module: `audio_processing/audio_extractor.py`

Responsibility: Given a video or audio file at `storage_path`, produce a normalized mono 16kHz WAV file suitable for ASR input, and populate `duration_seconds` on the `files` row.

Interface:

```
python

class AudioExtractor:
    def extract(self, input_path: str, output_path: str) -> AudioExtractionResult:
        """
        Runs: ffmpeg -i {input_path} -vn -ac 1 -ar 16000 -f wav {output_path}
        Returns duration_seconds, sample_rate, channel_count.
        Raises AudioExtractionError on non-zero ffmpeg exit code or unreadable stream.
        """
```

Design notes:

- If the input is already audio-only (mp3/wav/aac/flac/m4a), the `-vn` flag is a no-op; the same code path handles both audio and video uploads, avoiding a branch per file type.
- 16kHz mono is the standard input rate for Whisper-family models; extracting at this rate up front avoids a second resampling pass inside the ASR engine.
- The original video container is **never modified** at this stage — only a working-copy audio stream is pulled out. The original video file is preserved untouched for the Stage 9 re-mux, which is what guarantees resolution/frame-rate/metadata preservation (Product Vision constraint, SRS-FR-EXPT-01).
- On extraction failure (corrupted codec, empty audio track), the job is marked `FAILED` with `error_message` populated from the ffmpeg stderr tail, surfaced via the edge case in Architecture §4.3 (Failed ASR / Retry sequence — extraction failures follow the identical retry path as ASR failures).

5. Stage 2 — Speech Recognition (ASR) + Forced Alignment

Module: `asr/speech_recognition_service.py` (interface),

`asr/faster_whisper_engine.py` (implementation), `asr/forced_alignment.py`

Interface (swap point per Architecture §7.3):

```
python

class SpeechRecognitionService(ABC):
    @abstractmethod
    def transcribe(self, audio_path: str, language_hint: str | None = None) -> T:
        """Returns word-level segments: [{text, start, end, confidence}], plus de
```

Faster-Whisper implementation details:

Parameter	Value	Rationale
Model size	<code>medium</code> (default), <code>small</code> as CPU-constrained fallback	Balances Hindi/Hinglish accuracy against CPU latency budget ($\leq 2\times$ real-time, SRS-NFR-PERF-01)
Compute type	<code>int8</code> on CPU, <code>float16</code> on GPU	CTranslate2 quantization; int8 is the primary lever for meeting the CPU-first constraint from the Tech Stack doc
<code>word_timestamps</code>	<code>True</code>	Required directly by FR-TLE-01/02
<code>vad_filter</code>	<code>True</code>	Skips silence, reducing hallucinated transcriptions on music-only files (Edge Case table, PRD §7)
<code>beam_size</code>	<code>5</code>	Standard Whisper-family default; higher values traded against latency budget
Chunking	Files > 10 minutes are split into overlapping ~30s windows with 2s overlap, transcribed independently, then stitched by discarding duplicate words in the overlap region based on timestamp continuity	Bounds memory and enables the "chunk long files, show incremental progress" behavior required by the Edge Case table (3-hour lecture)

Forced alignment: Faster-Whisper's native `word_timestamps` already performs a WhisperX-style alignment internally (this is why Faster-Whisper was selected over vanilla Whisper in the Tech Stack doc — see §3.3 point 4). No separate wav2vec2 alignment pass is required for MVP; `forced_alignment.py` is kept as a thin, currently-pass-through module specifically so a dedicated alignment model (e.g., an Indic wav2vec2 checkpoint) can be inserted later without touching `faster_whisper_engine.py` or any calling code — this is the concrete realization of the "swap point" language used in the Architecture document.

Output contract (`TranscriptionResult`):

```
json
{
  "words": [
    {"text": "This", "start": 0.00, "end": 0.18, "confidence": 0.97},
    {"text": "fucking", "start": 1.22, "end": 1.61, "confidence": 0.89}
  ],
  "raw_segments": [{"text": "This movie is fucking amazing", "start": 0.0, "end": 1.61}],
  "language_guess": "en"
}
```

Word-level output feeds Stage 4–7 directly (each word/short n-gram is a matching unit); `raw_segments` feed the `transcript.full_text` field surfaced at `GET /files/{id}/transcript`.

6. Stage 3 — Language Detection

Module: `language_detection/language_detector.py`

Responsibility: Assign one of `english | hindi | hinglish | mixed` per segment (not just per file), satisfying SRS-FR-LANG-01/03.

Algorithm:

1. Take Faster-Whisper's per-segment `language_guess` (Whisper's internal detector) as a first signal — it is reliable for monolingual segments but weak on genuinely code-mixed sentences.
2. Run a lightweight script-detection pass over the transcribed text itself: classify each token as Latin-script or Devanagari-script.
3. Combine signals per segment:
 - All-Latin + Whisper says `en` → `english`
 - All-Devanagari + Whisper says `hi` → `hindi`

- Mixed Latin/Devanagari tokens within one segment, OR Latin-script tokens that match Romanized-Hindi phonetic patterns (checked against the Hinglish slang dictionary from Stage 5) → `(hinglish)`
 - Adjacent segments in the same file disagree on language → file-level `(detected_language)` is set to `(mixed)`, while each segment retains its own individual tag (this is what SRS-FR-LANG-03's code-switching requirement actually needs — segment-level tags, not a single file-level guess)
4. If confidence in step 3 is low (ambiguous script + no dictionary hit), default to `(hinglish)` rather than `(english)`, since Romanized Hindi profanity is the case most likely to be silently missed if defaulted the other way — this directly protects the recall target in SRS-FR-PDE.

The manual override endpoint behavior (SRS-FR-LANG-04, exposed via the frontend as a language dropdown) simply re-invokes Stage 2 with `(language_hint)` forced, then re-runs Stages 3-7; it does not require new pipeline logic beyond honoring the `(language_hint)` parameter already defined in the `(SpeechRecognitionService)` interface above.

7. Stage 4 — Text Normalization

Module: `(profanity_detection/normalizer.py)`

This stage exists specifically to satisfy the Product Vision's hardest requirement: catching "Fuuuuuck," "m@darchod," and similar disguised variants before any matching happens, so that Stage 5's dictionary lookup can stay a simple, explainable, admin-editable exact-match operation.

Normalization steps, applied in order:

1. **Unicode normalization (NFC)** — required for Devanagari input consistency (SRS-FR-DTCT-CFG-05); two visually-identical Devanagari strings with different combining-character encodings must normalize to the same form before comparison.
2. **Lowercasing** — Latin-script tokens only (case-folding does not apply to Devanagari).
3. **Repeated-character collapsing** — regex `(([a-z])\1{2,})` → replace runs of 3+ identical letters with a single instance, then re-check the dictionary; if no match, progressively try 2-repeat and 1-repeat collapses. This handles `(fuuuuuck → fuck)` (SRS-FR-PDE-02) without over-collapsing legitimate double letters (e.g., "book" is untouched since it only has a 2-run).
4. **Leetspeak / symbol substitution mapping** — a fixed substitution table `(@→a, 4→a, 3→e, 1→i, 0→o, $→s, 7→t)` applied to generate a normalized candidate string, e.g. `(m@darchod → madarchod)` (SRS-FR-PDE-04). Substitution is applied combinatorially only up to 2 substituted characters per word to bound the

search space and avoid false-positive explosions on legitimate words containing digits.

5. **Whitespace/punctuation stripping** at word boundaries, so that `f.u.c.k` or `f-u-c-k` still normalize to `fuck`.

Output: for every ASR word/n-gram, the normalizer emits a small ranked list of normalization candidates (original → fully normalized), each carried forward into Stage 5 with a `normalization_distance` score (0 = exact match, higher = more transformations applied). This distance later contributes negatively to the confidence score in Stage 7 — an exact dictionary hit is more trustworthy than a hit found only after three transformations.

Worked examples (traceable to Product Vision requirements):

Input	Normalization applied	Result
<code>Fuuuuuck</code>	lowercase → repeated-char collapse	<code>fuck</code>
<code>m@darchod</code>	leetspeak map (<code>@</code> → <code>a</code>)	<code>madarchod</code>
<code>bc</code> , <code>bkl</code> , <code>mc</code>	none (already normalized; matched as slang abbreviations directly in Stage 5)	<code>bc</code> , <code>bkl</code> , <code>mc</code>
<code>साले</code>	Unicode NFC normalization	<code>साले</code> (consistent encoding)

8. Stage 5 — Dictionary & Regex Matching

Module: `profanity_detection/dictionary_matcher.py`

Data source: `profanity_dictionary` table (Database Design §4.9), loaded into an in-memory hash set per language at worker startup and refreshed on a short TTL (e.g., 5 minutes) or on an explicit cache-bust signal when an admin edits the dictionary via `POST/PATCH/DELETE /admin/dictionary` — this is what makes SRS-FR-PDE-08 ("dictionary extensible without code deployment") actually true at runtime, not just at the database level.

Matching algorithm:

- For each normalized candidate from Stage 4, look up `normalized_term` in the in-memory set for the segment's detected language, and also in the `universal` language bucket (catches slang used across English/Hindi/Hinglish contexts, e.g. `bc`, `mc`, `bkl`).
- Sliding n-gram window (unigram, then bigram) is checked for multi-word entries (e.g., a curated phrase, not just single words).

3. On a hit, record: matched (`dictionary_id`), (`severity`), and (`detection_source = 'dictionary'`) (exact match) or (`'regex'`) (matched only via a pattern rule rather than an exact dictionary row — reserved for entries flagged as regex-type in the dictionary, e.g. a rule capturing any leetspeak variant of a root word without enumerating every combination).
4. Non-matches are passed to Stage 6.

This layer is intentionally the fastest and most explainable — it runs in-process against an in-memory hash set, contributing negligible latency to the overall $\leq 2\times$ -real-time budget, and every hit can be shown to the user with a precise "matched dictionary entry: X" explanation (SRS-FR-PDE-07), satisfying the explainability requirement discussed in the Tech Stack document's hybrid-architecture rationale.

9. Stage 6 — Context-Aware Scoring

Module: (`profanity_detection/context_scorer.py`)

Responsibility: Catch profanity that Stage 5 missed — novel slang, context-dependent terms, or heavily obfuscated variants beyond the normalizer's fixed rules — using semantic similarity rather than exact matching.

Approach (per Tech Stack §4.3): sentence-embedding similarity against a curated seed bank, not a fine-tuned classifier — this is the deliberate choice to avoid needing a labeled Hindi/Hinglish profanity training set that does not exist.

Algorithm:

1. Embed each transcript segment (not just the flagged word, but a small context window — the word plus 2-3 surrounding words) using a multilingual sentence-embedding model (MuRIL-based or a general multilingual sentence-transformer, per the Tech Stack recommendation).
2. Compute cosine similarity between the segment embedding and every entry in (`data/embeddings/seed_profanity_examples.json`) (curated example phrases per language, expandable by admins without retraining — this is the concrete mechanism behind the Product Vision's "support future learning" requirement).
3. If max similarity exceeds a configurable threshold (default (`0.72`), tuned empirically against the internal test set referenced in PRD §4.6/AC-1), emit a candidate detection with (`detection_source = 'model_inferred'`) and a raw similarity score.
4. This stage also **re-scores** Stage 5 dictionary hits that occurred in an ambiguous context (e.g., a word that is profane in one context but benign in another, per the PRD Edge Case table) — the embedding similarity against context-specific seed examples can *lower* an otherwise-high dictionary-match confidence, which is then finalized in Stage 7.

Because this stage runs an embedding model per segment rather than per word, and multilingual sentence-transformer models are small relative to the ASR model already loaded, this stage adds a bounded, predictable amount of CPU time and does not require a network call (no cloud LLM dependency), preserving the offline/cost-free constraint from the SRS.

10. Stage 7 — Confidence Aggregation

Module: `profanity_detection/confidence_aggregator.py`

Responsibility: Produce the final `confidence_score` (0.0–1.0) and `detection_source` persisted per row in `detections` (Database Design §4.8), satisfying SRS-FR-PDE-05.

Aggregation formula:

```
base_score =
    0.95 if detection_source == 'dictionary' (exact match,
normalization_distance == 0)
    0.85 if detection_source == 'dictionary' (matched only after
normalization, distance > 0)
    0.75 if detection_source == 'regex'
    similarity_score (0.0–1.0) if detection_source == 'model_inferred'

final_score = clamp(base_score - (0.03 * normalization_distance) +
context_adjustment, 0.0, 1.0)
```

Where `context_adjustment` is the signed delta produced by Stage 6 re-scoring (Section 9, step 4) — positive if the surrounding context reinforces the profane reading, negative if it suggests a benign homonym use.

Deduplication: if the same time range is flagged by both Stage 5 and Stage 6 independently, the two candidate detections are merged into a single `detections` row rather than persisted as duplicates — this is the same merge/dedupe behavior already specified for Combined detection mode at the user-facing level (SRS-FR-DTCT-CFG-03), applied here internally between the dictionary and model layers.

Threshold filtering: All candidate detections are persisted regardless of score (nothing is silently dropped, since the user can filter by confidence in the Timeline Editor per SRS-FR-PDE-06/FR-TLE — dropping low-confidence candidates at the pipeline level would remove information the human reviewer is specifically there to adjudicate). A configurable `MIN_PERSIST_CONFIDENCE` (default `0.3`) exists purely to discard obvious noise from the embedding layer, not to enforce the product-level precision/recall target — that target (SRS-FR-PDE AC-1: recall $\geq 85\%$, precision $\geq 80\%$) is measured and tuned against the *default-displayed* threshold in the UI, not against what's stored.

11. Detection Instance Output Contract

Each finalized detection maps directly onto the `detections` table schema (Database Design §4.8) and the API response shape already defined in API Design §5.1:

```
json
{
  "detected_text": "bakchodi",
  "language": "hindi",
  "start_time": 12.450,
  "end_time": 13.100,
  "confidence_score": 0.94,
  "detection_source": "dictionary",
  "dictionary_id": "uuid-or-null",
  "replacement_mode": "beep",
  "is_user_added": false,
  "is_dismissed": false
}
```

`replacement_mode` at creation time is inherited from the file's `detection_configs.replacement_mode` (the mode chosen at upload time); it is only overridden per-instance later if the user changes it individually in the Timeline Editor (FR-TLE-05).

12. Stage 8 — Audio Editing

Module: `audio_processing/audio_editor.py`

Responsibility: For every non-dismissed detection, replace the corresponding audio segment according to its `replacement_mode`, satisfying SRS-FR-REPL-01/02/03/05.

Interface:

```
python
class AudioEditor:
    def apply_replacements(
        self,
        audio_path: str,
        detections: list[DetectionInstance],
        replacement_assets: ReplacementAssets,
    ) -> str:
        """Returns path to the edited audio file."""
```

Algorithm (PyDub-based):

1. Load the extracted audio (`AudioSegment.from_wav(audio_path)`).
2. Sort detections by `start_time` ascending.
3. For each detection, apply a **safety padding margin** of 100–150ms on both boundaries (per the PRD Edge Case table: "profane word at very start/end of a clip"), clamped to not overlap the previous/next detection's own padded range.
4. Per `replacement_mode`:
 - **Beep**: generate a sine-wave tone (`pydub.generators.Sine(frequency=1000)`), configurable frequency per SRS-FR-REPL-01) with duration exactly matching the padded segment length, and overlay/replace it in place.
 - **Mute**: overlay `AudioSegment.silent(duration=segment_length_ms)` in place of the original segment.
 - **Custom sound**: load the user-uploaded clip from `custom_sound_path`; if shorter than the segment, loop it; if longer, trim it — always matching the original segment's duration within the ± 50 ms tolerance specified in SRS-FR-REPL-05, which is the concrete numeric target this trim/loop logic is built to satisfy.
5. Reassemble the full audio track by concatenating: [untouched audio before first detection] + [replaced segments and untouched gaps between them, in order] + [untouched audio after last detection].
6. Export the reassembled track to a working WAV file for Stage 9.

Regeneration efficiency note (ties back to PRD Open Question §11): because this stage operates purely on the already-extracted audio plus the current `detections` rows, a "Regenerate" action (FR-TLE-08) does **not** need to re-run Stages 1–7 (extraction, ASR, detection) — only Stage 8–9 re-run, using the edited detection set. This is what makes the AC-2 requirement in PRD §4.7 ("regeneration only reprocesses affected segments, not a full re-transcription") achievable, and resolves the open question raised in the PRD as "full file vs. edited segments only" — the answer, given this pipeline design, is: full audio re-assembly (cheap, since it's just PyDub segment operations) but zero re-transcription (expensive, and unnecessary since the transcript/timestamps didn't change).

13. Stage 9 — Media Muxing

Module: `audio_processing/media_muxer.py`

Responsibility: Re-combine the edited audio track with the **original, untouched video stream**, preserving resolution, frame rate, and metadata (Product Vision constraint; SRS-FR-EXPT-01/AC-1).

Algorithm (FFmpeg):

```
ffmpeg -i {original_video_path} -i {edited_audio_path} \  
-map 0:v -map 1:a \  
-c:v copy \  
-c:a aac -b:a 192k \  
-map_metadata 0 \  
-shortest \  
{output_video_path}
```

Key decisions encoded in this command:

- `-c:v copy` — the video stream is **stream-copied**, never re-encoded, which is the only way to guarantee zero quality/resolution/frame-rate loss (this is the specific technical mechanism behind the "preserve video quality/frame rate/resolution" requirement stated repeatedly across the Product Vision, PRD, and SRS).
- `-map_metadata 0` — pulls container metadata from the original file, not the edited audio, preserving original metadata as required.
- Audio is re-encoded (not stream-copied) because PyDub's edits produced a modified PCM/WAV stream that must be re-compressed into the target container's expected audio codec (typically AAC for MP4 containers).
- `-shortest` guards against a duration mismatch between video and reassembled audio causing trailing silence/frozen frames; combined with the $\pm 50\text{ms}$ tolerance enforced in Stage 8, drift stays within the $<100\text{ms}$ sync budget from the Product Vision's success metrics and SRS-FR-EXPT AC-2.

Audio-only files (mp3/wav/etc. uploads) skip this stage entirely — the Stage 8 output is the final audio export, simply transcoded to the original format via a lighter FFmpeg pass if needed.

14. Pipeline Orchestration

Module: `pipeline/processing_pipeline.py`

```
python
```

```

class ProcessingPipeline:
    def run_initial(self, file_id: str, config: DetectionConfig) -> PipelineResult:
        audio_path = self.extractor.extract(...)
        transcription = self.asr.transcribe(audio_path)
        segments = self.language_detector.tag(transcription)
        candidates = []
        for segment in segments:
            normalized = self.normalizer.normalize(segment.words)
            dict_hits = self.dictionary_matcher.match(normalized, segment.language)
            model_hits = self.context_scorer.score(segment, dict_hits)
            candidates += self.aggregator.aggregate(dict_hits, model_hits)
        detections = self.merge_with_custom_words(candidates, config)
        self.repository.save_detections(file_id, detections)
        self.audio_editor_and_mux(file_id, detections) # first-pass render
        return PipelineResult(status="READY_FOR_REVIEW", detections=detections)

    def run_regeneration(self, file_id: str) -> PipelineResult:
        detections = self.repository.get_active_detections(file_id) # excludes :
        edited_audio = self.audio_editor.apply_replacements(
            self.repository.get_extracted_audio_path(file_id), detections, ...
        )
        output_path = self.media_muxer.mux(
            self.repository.get_original_video_path(file_id), edited_audio
        )
        self.repository.save_export(file_id, output_path)
        return PipelineResult(status="COMPLETED")

```

Idempotency (SRS-NFR-REL-02, Architecture §1.2): every stage writes its output to a content-addressed or job-scoped working path (e.g., keyed by `(job_id)`), never mutating the original upload in place. A retried job simply re-runs from Stage 1 against the same immutable input and overwrites its own prior (also-immutable-to-others) intermediate outputs — no partial state from a failed attempt can corrupt a later successful attempt, satisfying the "idempotent, resumable jobs" principle stated in the Architecture document.

Custom/Combined detection modes: `(merge_with_custom_words)` (called in `(run_initial)`) is where the Detection Config's `(mode)` (`(automatic)`/`(custom)`/`(combined)`) actually takes effect — custom words from `(detection_configs.custom_words)` are matched using the *same* Stage 4–5 normalization and dictionary-matching code path (a custom word list is just an ephemeral, per-file dictionary), then merged and deduplicated against the automatic-detection candidates exactly as SRS-FR-DTCT-CFG-03 specifies, reusing infrastructure rather than introducing a parallel matching implementation.

15. Performance Considerations

Concern	Mitigation
ASR is the dominant cost (SRS-NFR-PERF-01: $\leq 2\times$ real-time)	<code>int8</code> quantized Faster-Whisper on CPU; GPU used opportunistically when available (Tech Stack §3.3, point 5) without being a hard dependency
Long files (3-hour lecture, PRD Edge Case)	Chunked transcription with incremental progress published via the WebSocket status channel (Architecture §4.1) after each chunk, not only at full completion
Concurrent jobs (SRS-NFR-PERF-04)	Each pipeline stage is stateless per invocation; horizontal scaling is achieved by running more worker replicas against the same Redis queue (Architecture §6.1), not by parallelizing within a single job
Repeated regeneration cost	As established in §12, regeneration skips Stages 1–7 entirely — this is the single largest performance lever available for the Timeline Editor's "Regenerate" action

16. Error Handling & Retry

Each stage raises a stage-specific exception (`AudioExtractionError`, `TranscriptionError`, `MuxingError`, etc.) caught at the `processing_task.py` Celery task boundary (Folder Structure §4), which:

- Writes a `job_logs` row with `stage`, `level = 'ERROR'`, and the exception message (Database Design §4.7).
- Increments `jobs.retry_count` and re-enqueues with exponential backoff if under `max_retries`, or marks the job `FAILED` and publishes a `job_failed` WebSocket event — this is precisely the sequence already diagrammed in System Architecture §4.3.
- For the specific "music only / no speech detected" edge case, an empty-but-valid transcription is **not** treated as an error — the pipeline completes normally with zero detections and the file status becomes `READY_FOR_REVIEW`, matching the PRD Edge Case table's expected behavior.

17. Extensibility Points (Concrete Realization of Architecture §7.3)

Interface	Current implementation	Future swap example
<code>SpeechRecognitionService</code>	<code>FasterWhisperEngine</code>	Cloud ASR API (Google/Azure) for a future paid tier
<code>forced_alignment.py</code>	Pass-through (Faster-Whisper's native word timestamps)	Dedicated Indic wav2vec2 alignment model
<code>ProfanityDetectionService</code>	Normalize → Dictionary/Regex → Embedding scorer	Fine-tuned IndicBERT/MuRIL classifier once labeled data exists
<code>AudioEditor</code> replacement strategy	Beep / Mute / Custom sound	<code>VoiceReplacementStrategy</code> (AI voice cloning, Future Scope) plugged in as a fourth <code>replacement_mode</code> without <code>apply_replacements</code>

18. Testing & Evaluation Strategy

- **Unit tests** (`ai-pipeline/tests/`): `test_normalizer.py` asserts the exact worked examples from Section 7 (e.g., `Fuuuuuck → fuck`, `m@darchod → madarchod`); `test_dictionary_matcher.py` and `test_context_scorer.py` test against fixture transcripts with known expected detections.
- **Benchmark scripts** (`scripts/benchmark_asr.py`, `scripts/benchmark_detection.py`, per Folder Structure §10): run against a held-out labeled test set to produce the recall/precision numbers referenced in PRD §4.6/AC-1 (recall ≥85%, precision ≥80%) and SRS §5.4 language-detection accuracy (≥85%).
- **Integration test** (`test_pipeline_integration.py`): runs the full Stage 1-9 pipeline against `scripts/generate_test_media.py` fixtures (short synthetic clips with known profanity at known timestamps) and asserts both correct detection and <100ms audio/video sync drift in the final muxed output.

19. Configuration

All tunable values in this document are externalized (per SRS-NFR-MAINT-03), not hardcoded, via `ai-pipeline/sentinelcut_ai/pipeline/pipeline_config.py` and environment variables consumed at worker startup:

Config key	Default	Section
ASR_MODEL_SIZE	medium	§5
ASR_COMPUTE_TYPE	int8	§5
CHUNK_LENGTH_SECONDS	1800 (30 min)	§5
CONTEXT_SIMILARITY_THRESHOLD	0.72	§9
MIN_PERSIST_CONFIDENCE	0.3	§10
BOUNDARY_PADDING_MS	125	§12
BEEP_FREQUENCY_HZ	1000	§12
MAX_JOB_RETRIES	3	§16 (mirrors jobs.max_retries default in Database Design)

20. Next Document

Document 10: UI/UX Design — screen-level design specifications (Landing Page, Dashboard, Upload, Timeline Editor, Admin Dashboard, Light/Dark mode) for the Next.js frontend, consuming the API contract from Document 8 and reflecting the detection/confidence data shapes defined in this AI Pipeline document (in particular, how per-word confidence scores and detection sources from Section 11 are visualized in the Timeline Editor).

End of AI Pipeline Document

1. Purpose

This document specifies the screen-level UI/UX design for the SentinelCut Next.js frontend (`frontend/src/app/`), per Folder Structure §3). It defines layout, component composition, state, and interaction behavior for every route group (`(marketing)`, `(auth)`, `(app)`, `(admin)`), and — critically — how the confidence scores and detection sources produced by the AI Pipeline (Document 9, §11) are surfaced to a non-technical user in a way that builds trust rather than confusion. Every screen below is wired to a specific endpoint from the API Design document (Document 8); no screen in this document invents data the backend doesn't already provide.

This is a design *specification*, not a visual mockup file — it defines what exists on each screen, how it behaves, and why, so implementation can proceed directly against it using Tailwind CSS and the component libraries already scaffolded in `frontend/src/components/`.

2. Design Principles

1. **Confidence is not binary.** The AI Pipeline (Document 9 §10) deliberately persists every detection with a graded confidence score rather than a hard yes/no. The UI must never collapse that back into a flat "profanity found" list — the whole value of the Timeline Editor's human-in-the-loop design (Product Vision, PRD §4.7) depends on the user being able to see which detections are certain and which are borderline.
2. **The review step is the safety net, not an afterthought.** Given the accepted MVP accuracy bar ($\geq 85\%$ recall, per PRD §8/AI Pipeline §18), the UI's job is to make correcting the AI fast and low-friction, not to pretend the AI is perfect.
3. **Guest Mode must feel complete, not crippled.** A guest user should be able to run the full upload→detect→edit→export loop; the *only* difference in the UI is persistent history and presets (per PRD §4.1/FR-1.4), and this difference must be communicated, not hidden.
4. **One interaction model for light and dark.** Every screen is designed once, against design tokens (Section 3), not designed twice — dark mode is a token swap, not a parallel UI.
5. **Startup-SaaS visual bar, not academic-project visual bar.** Per the Master Prompt's explicit framing, density, motion, and typography choices below intentionally mirror what a funded SaaS product would ship, not a bare-bones CRUD scaffold.

3. Design Tokens & Visual System

Implemented as Tailwind CSS custom theme values in `frontend/tailwind.config.js`, consumed via CSS variables so light/dark switching (SRS §4.1, "Should priority") is a single class toggle on `<html>`.

3.1 Color Tokens

Token	Light	Dark	Usage
<code>--bg-base</code>	#FAFAFA	#0B0D10	App background
<code>--bg-surface</code>	#FFFFFF	#14171C	Cards, panels
<code>--bg-surface-raised</code>	#FFFFFF (shadow)	#1B1F26	Modals, dropdowns
<code>--text-primary</code>	#111318	#F1F2F4	Body text
<code>--text-secondary</code>	#5B6270	#9AA1AC	Meta text, labels
<code>--border-subtle</code>	#E6E8EB	#262B33	Dividers, card borders
<code>--brand-primary</code>	#6D5EF6 (indigo-violet)	#8B7CFF	Primary actions, active states
<code>--accent-teal</code>	#14B8A6	#2DD4BF	Success / completed states
<code>--accent-amber</code>	#F59E0B	#FBBF24	Borderline confidence, warnings
<code>--accent-rose</code>	#F43F5E	#FB7185	Destructive actions, high-severity detections

3.2 Confidence Color Scale (used exclusively in the Timeline Editor, Section 8)

This scale is the single most important visual mapping in the product, since it is the direct UI representation of AI Pipeline §10's `confidence_score`:

Confidence range	Color token	Meaning shown to user
≥ 0.85	<code>--accent-rose</code> (solid fill)	"High confidence"
0.60–0.84	<code>--accent-amber</code> (solid fill)	"Medium confidence — review suggested"
0.30–0.59	<code>--accent-amber</code> (outline only, no fill)	"Low confidence — likely needs review"
< 0.30 (only ever reachable via <code>min_confidence</code> filter override)	<code>--text-secondary</code> (dotted outline)	"Very low confidence"

3.3 Typography

- Primary typeface: **Inter** (variable weight) — chosen for its clarity in dense data UI (Dashboard, Timeline Editor transcript) and its native support for Latin + reasonable Devanagari fallback rendering.
- Devanagari text (Hindi transcript segments) renders with a secondary stack: `'Noto Sans Devanagari', Inter, sans-serif` to guarantee correct glyph rendering regardless of the user's OS font availability — critical given Hinglish/mixed-language transcripts render both scripts inline in the same sentence (AI Pipeline §6).
- Type scale: `text-xs` (12px, meta/timestamps) → `text-sm` (14px, transcript body) → `text-base` (16px, UI body) → `text-2xl`/`text-4xl` (headings, landing page).

3.4 Spacing & Radius

- 4px base spacing unit (Tailwind default scale, unmodified).
- Radius: `rounded-lg` (8px) for cards and inputs, `rounded-full` for pills/badges (confidence badges, status chips), `rounded-2xl` for modal containers — deliberately softer than a typical enterprise admin tool, reinforcing the "startup SaaS" tone from the Master Prompt rather than a bare academic form-grid look.

3.5 Dark Mode Implementation

`next-themes`-style class-based toggling (`class="dark"` on `<html>`), persisted client-side (in-memory/React state only inside artifacts context, but in the real app via a standard cookie/localStorage preference — no backend persistence needed for this preference). All components consume only the CSS variable tokens above; no component ever hardcodes a hex value, which is what makes the "designed once" principle in Section 2 actually enforceable in code review.

4. Global Layout & Navigation

4.1 Shell Structure

- **(marketing) layout:** Full-width, no persistent sidebar. Sticky top nav (logo, "Features," "Pricing," "Login," primary CTA "Try Free").
- **(auth) layout:** Centered single-column card on a subtly branded background (soft gradient using `--brand-primary` at low opacity), no nav — minimizes distraction during signup/login per SRS-NFR-USE-01's "no external instructions needed" goal.
- **(app) layout:** Persistent left sidebar (`components/shared/Sidebar.jsx`) + top bar (`components/shared/Navbar.jsx`) — search, notifications bell tied to job status, avatar/profile menu, dark-mode toggle). Sidebar items: Dashboard, Upload, History, Settings, and (conditionally) Admin, if `role === 'admin'` from the decoded JWT.
- **(admin) layout:** Same shell shape as `(app)` but with a distinct sidebar icon set and a persistent "Admin Mode" badge in the top bar, so an admin never confuses which context they're in while managing the profanity dictionary or user records.

4.2 Responsive Behavior (SRS §4.1: desktop + tablet minimum)

- **Desktop (≥1024px):** Sidebar always expanded, Timeline Editor uses a two-pane layout (transcript left, playback/waveform + detection detail panel right).
- **Tablet (768–1023px):** Sidebar collapses to icon-only rail (click to expand as overlay); Timeline Editor stacks vertically (playback bar pinned above a full-width scrolling transcript).
- **Mobile (<768px):** Explicitly out of scope for full feature parity per SRS §4.1's minimum bar, but the Landing Page, Auth pages, and Dashboard degrade gracefully to single-column stacks so the marketing funnel and basic account checks aren't broken on a phone.

5. Landing Page (`(marketing)/page.js`)

Goal: Convert a visitor into either a Guest trial or a signup, per the Secondary Journey in PRD §6.2.

Sections (top to bottom):

1. **Hero** — headline framed around the Product Vision's core value prop ("Multilingual profanity censorship for English, Hindi & Hinglish content"), subheadline, two CTAs side by side: primary "Try as Guest" (→ calls `POST /auth/guest-session`), then redirects straight to Upload — zero friction, matching PRD §6.2 step 1) and secondary "Sign Up Free."
2. **Live demo strip** — a short, pre-recorded before/after audio waveform showing a censored word (static asset, not a live pipeline call) to make the "beep replaces

profanity" concept immediately legible without requiring the visitor to upload anything.

3. **Language support band** — three icon-labeled pills: "English," "Hindi," "Hinglish / Mixed" — directly visualizing the Main Objective from the Master Prompt/Product Vision.
 4. **How it works** — 4-step horizontal stepper mirroring the User Flow diagram (Upload → Detect → Review → Export), each step illustrated with a small static screenshot of the corresponding real screen (Upload page, Timeline Editor, Export panel) rather than generic stock art — reinforces product credibility.
 5. **Detection modes explainer** — three cards (Automatic / Custom / Combined), directly describing PRD §4.4 in plain language.
 6. **Footer** — links to Privacy Policy, Terms & Conditions (per SRS-NFR-LEGAL-01), GitHub/roadmap links, and a "Built as an academic project" credibility note (optional, configurable — a startup-facing deployment could omit this).
-

6. Auth Screens (`(auth)/*`)

6.1 Signup (`(signup/page.js)`)

Fields: email, password (inline strength meter reflecting FR-1.5's 8-char/1-digit/1-special rule live as the user types — never a rejection *after* submit), display name (optional). Secondary action: "Continue with Google" button (calls `POST /auth/google`) placed *above* the email/password form, since OAuth is typically the lower-friction path — email signup is the fallback, not vice versa.

6.2 Login (`(login/page.js)`)

Email/password + Google button, "Forgot password?" link, and a persistent small-text link "Just want to try it? Continue as Guest" — the guest path is never buried, consistent with the zero-signup-friction persona (Ankit/Evaluator) called out in PRD §3.

6.3 Forgot / Reset Password

Two-step flow matching API Design §3.4/3.5 exactly: email-entry form → generic "if an account exists..." confirmation screen (never reveals whether the email exists, per the API's anti-enumeration design) → (via emailed link) a reset-token form with the same live strength meter as signup.

6.4 Guest Session Banner

Any screen reached via Guest Mode renders a persistent, dismissible-but-reappearing top banner: "You're in Guest Mode — your files expire in [live countdown from `(expires_at)`]. [Sign up to save your work]." This directly implements the PRD Edge Case requirement ("clear warning shown before expiry") and is a shared component (`(components/shared/GuestBanner.jsx)`) mounted once in the `(app)` layout rather than duplicated per page.

7. Dashboard ((app)/dashboard/page.js)

Consumes (GET /dashboard/summary) (API Design §8.1) in a single call on mount, rendering four zones:

1. **Stat strip** (top, 3-4 cards in a row): Credits Remaining, Storage Used (progress bar: (storage_used_bytes) / (storage_limit_bytes)), Active Jobs count. Each stat card is intentionally lightweight — no charts here, just numbers — charts live only in the Admin Analytics screen (Section 12).
2. **Processing Queue panel** — live list of (active_jobs), each row showing filename, current (stage) (from the WebSocket (status_update) payload, API Design §12.2), and a progress affordance. Rows subscribe individually to (WSS /ws/jobs/{job_id}) (falling back to 5s polling per API Design §12.4) and animate from (PROCESSING) → (READY_FOR_REVIEW) with a toast + row highlight, then auto-navigate the user into the Timeline Editor if they click the row.
3. **Recent Files grid** — card-per-file (thumbnail placeholder for video, waveform placeholder for audio-only), status pill, "Open" action routing to (/files/{id}/editor) if (READY_FOR_REVIEW)+, or (/files/{id}/processing) if still in progress.
4. **Empty state** (first-time users) — replaces zones 2-3 with a single large "Upload your first file" CTA card, avoiding a dashboard that looks broken/empty on day one.

8. Upload Flow

8.1 Upload Page ((app)/upload/page.js)

- **Dropzone** (components/upload/) — large drag-and-drop target plus "Browse files" fallback button; accepted-format list (MP4/MOV/MKV/AVI/MP3/WAV/AAC/FLAC/M4A) shown as small format chips *before* a file is dropped, not only as an error after rejection.
- **Client-side validation** runs immediately on file selection (extension + size against the configured max, mirroring SRS-FR-UPLD-04) — rejected files show a specific inline error ("This file is 620MB; the maximum is 500MB") per SRS-NFR-USE-03's "specific and actionable" requirement, never a generic "upload failed."
- **Upload progress:** determinate progress bar bound to the (multipart/form-data) upload's progress event, with a Cancel button that aborts the in-flight request (FR-3.5).
- On success ((201) from (POST /files/upload)), the page transitions in place (no full navigation) into:

8.2 Configuration Panel (same page, revealed after upload)

Two side-by-side selector groups, both required before "Start Processing" activates:

- **Detection Mode** (radio card group): Automatic / Custom / Combined. Selecting Custom or Combined reveals a textarea for comma/newline-separated words plus a "Load from preset ▾" dropdown (populated via `GET /presets`) — directly implementing PRD §4.4 and FR-DTCT-CFG-04.
- **Output Mode** (radio card group): Beep / Mute / Custom Sound. Selecting Custom Sound reveals a small secondary upload dropzone for the replacement clip.

"Start Processing" calls `POST /files/{id}/process` with the assembled `detection_mode`/`custom_words`/`preset_id`/`replacement_mode` body (API Design §4.5), then routes to the Processing Status page.

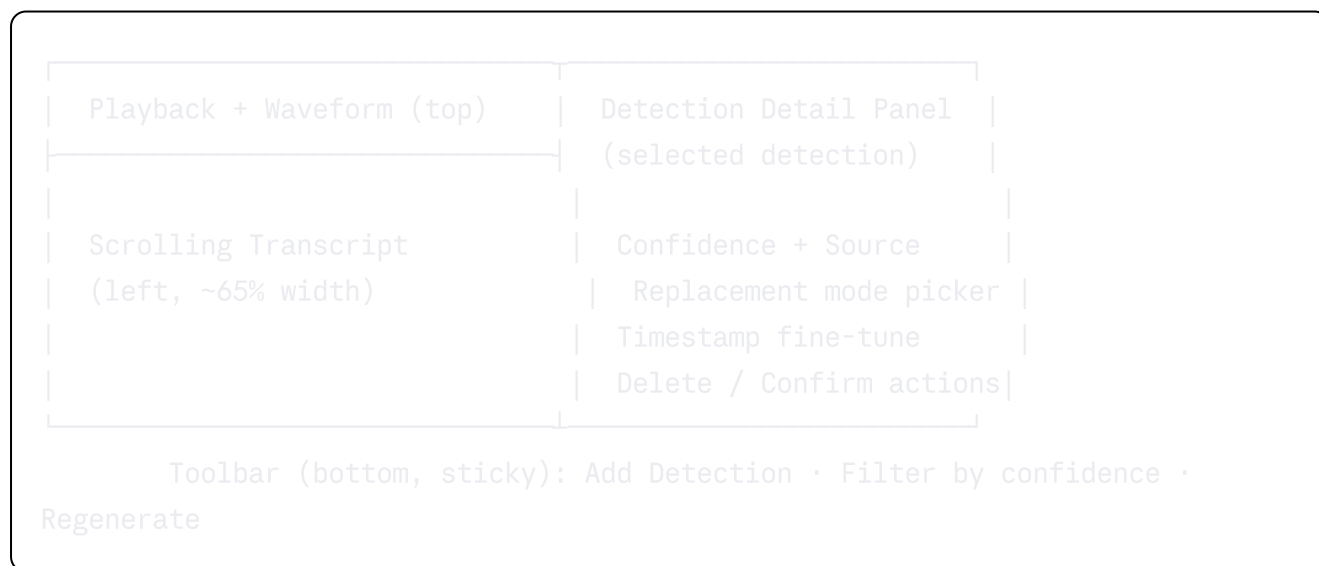
8.3 Processing Status Page (`files/[fileId]/processing/page.js`)

A focused, single-purpose screen (not the full dashboard) showing: file name, an animated stage indicator that maps directly onto the AI Pipeline stages (Document 9 §3) — "Extracting Audio → Transcribing Speech → Detecting Language → Scanning for Profanity" — driven by the `stage` field in each WebSocket `status_update` message, so the user watching this screen sees real pipeline progress, not a fake spinner. On `job_completed`, auto-navigates into the Timeline Editor.

9. Timeline Editor (`files/[fileId]/editor/page.js`)

This is the most technically demanding screen in the product (per System Architecture §3.1/Folder Structure §3 rationale) and the primary place where AI Pipeline §11's detection data must be made legible and editable. Layout described for desktop; tablet/mobile stacking per Section 4.2.

9.1 Layout



9.2 Transcript Rendering & Detection Highlighting (implements FR-TLE-01/02, consumes `GET /files/{id}/transcript` + `GET /files/{id}/detections`)

- Full transcript text renders as inline spans (`components/timeline-editor/TranscriptView.jsx`), each word positioned from `segments[].start/end` (API Design §6.1).
- Any span whose time range overlaps a `detections[]` entry (API Design §5.1) is wrapped in a `DetectionHighlight.jsx` component, background-filled or outlined per the **Confidence Color Scale** defined in Section 3.2 — this is the direct visualization of AI Pipeline §10's aggregated score, and is the single most load-bearing design decision in this document: a user must be able to glance at a paragraph and immediately see "one high-confidence hit, one borderline one" without opening anything.
- A small superscript badge on each highlighted span shows `detection_source` as a one-letter/icon tag: **D** (dictionary), **R** (regex), **M** (model-inferred) — with a hover tooltip spelling out the full source and, for dictionary hits, the matched term (surfacing AI Pipeline §8's "matched dictionary entry" explainability point directly in the UI, not buried in a details drawer).
- The currently-playing word (driven by the video/audio `<currentTime>`) gets a distinct "playhead" underline independent of the confidence color, so playback position and detection status are visually decoupled and never compete for the same color channel.

9.3 Detection Detail Panel

Clicking a highlighted span (or a row in an optional flat list view, toggle available for users who prefer scanning a list over inline transcript reading) opens the detail panel showing:

- Detected text, exact timestamp range (editable — fine-tune via small +/- stepper buttons in 50ms increments, satisfying FR-TLE-06), language tag.
- **Confidence score shown as both the color-coded badge and the literal number** (e.g., "0.94 — High confidence, dictionary match") — showing the raw number alongside the color is deliberate: power users (the Priya/Professional Editor persona from PRD §3) want the precise value, casual users get the at-a-glance color.
- Replacement mode dropdown scoped to *this detection only* (FR-TLE-05), defaulting to the file-level setting from Section 8.2 but independently overridable.
- **Dismiss** (soft "not profane" — sets `is_dismissed: true` via `PATCH /detections/{id}`) and **Delete** are presented as the same single action to the user (the API distinction between dismiss-and-keep-audit-trail vs. hard delete, API Design §5.5, is an implementation detail the UI does not need to expose as two separate buttons).

9.4 Adding a Missed Detection (FR-TLE-04)

User selects a text range directly in the transcript (native text selection, not a custom widget — minimizes learning curve) or drags a range marker directly on the waveform; either path opens a small inline form (detected text pre-filled from the selection, start/end pre-filled from the selection's word timestamps, replacement mode defaulting to the file-level setting) and calls `POST /files/{id}/detections`. The resulting span renders with a distinct visual treatment (solid `--brand-primary` outline, not the confidence color scale, since a user-added detection has no AI-generated confidence score to visualize — this is intentional: manually added detections are never confused with AI-inferred ones).

9.5 Confidence Filtering (FR-PDE-06)

A slider in the sticky toolbar ("Show detections above: [0.5] confidence") re-filters which highlights render, without re-fetching data — all detections are already loaded client-side from the single `GET /files/{id}/detections` call, and the filter is a pure client-side render concern. This lets a cautious user (Priya, bulk/precision-focused) start at a high threshold and progressively lower it to review borderline cases in a deliberate second pass, rather than being shown everything at once.

9.6 Autosave, Undo/Redo, Regenerate

- Every edit action pushes onto a local undo/redo stack (`hooks/useTimelineEditor.js`, `store/editorStore.js`) supporting at least the last 10 actions (SRS-FR-TLE-09).
- Every 30 seconds (and on tab blur), the current in-progress edit state (not yet "confirmed" — see below) is serialized and sent to `POST /files/{id}/editor/autosave` (FR-TLE-08); on load, `GET /files/{id}/editor/autosave` is checked first and, if a draft newer than the last confirmed save exists, the user is prompted: "Restore your unsaved edits from [time]?"
- The bottom toolbar's **Regenerate** button batches all pending edits into a single `PATCH /files/{id}/detections/batch` call (API Design §5.4), then calls `POST /files/{id}/regenerate` (§4.6), and transitions the screen into a compact inline "Regenerating..." state (reusing the same WebSocket-driven stage indicator pattern from Section 8.3, but scoped to Stage 8–9 only per AI Pipeline §12/§14 — the UI's copy reflects this, showing "Applying edits" and "Finalizing export," never re-showing "Transcribing speech," which correctly communicates to the user that their edits are being applied quickly rather than the whole file being reprocessed).

9.7 Inline Preview

Once regeneration completes, an inline `<video>/<audio>` player swaps in showing the *censored* output directly in the same screen (no forced navigation), satisfying PRD §4.7 AC-1 ("edits are reflected in a live preview before final export").

10. History Page ((app)/history/page.js)

Table/list view over (GET /files) (API Design §4.3) with search input, status filter chips, and sort control — directly implementing FR-DASH-02. Each row: filename, upload date, status pill, detected language tag, quick actions (Open Editor / Download / Delete). Pagination footer reflects the (meta.total_pages) envelope (API Design §2.5).

11. Settings & Profile ((app)/settings/page.js), ((app)/profile/page.js)

- **Profile:** display name, email (read-only if OAuth-provisioned), account deletion action (a clearly separated, red-accented "Danger Zone" card — satisfies SRS-NFR-LEGAL-02's right-to-delete requirement with appropriately weighted visual friction, i.e., a confirmation modal requiring the user to type the word "delete").
 - **Settings:** Preset management (list/create/edit/delete over (/presets) endpoints, §9 of API Design), theme toggle (light/dark/system), and default replacement-mode preference (a pure client-side convenience default applied when the Upload page's Configuration Panel first renders — not a new backend field).
-

12. Admin Panel ((admin)/admin/*)

Consistent shell (Section 4.1) across five sub-screens, all gated on (role: admin):

12.1 User Management (admin/users/page.js)

Paginated table (GET /admin/users) with search + status filter, row action to disable/enable a user (PATCH /admin/users/{id}) behind a confirmation modal — mutating another user's account is always a two-step action, never a single click.

12.2 Profanity Dictionary (admin/dictionary/page.js)

This screen is the direct admin-facing counterpart to AI Pipeline §8's "admin-editable, no-deployment-required" design goal. Table view (GET /admin/dictionary) filterable by language/active-status, inline "Add Term" form (term, language, severity — POST /admin/dictionary) and inline toggle for (is_active) (PATCH). A visible note near the top of this screen states plainly: *"Changes here take effect within 5 minutes for new uploads"* — setting accurate expectations given the in-memory cache TTL defined in AI Pipeline §8, rather than implying instant global effect.

12.3 Files Overview (admin/files/page.js)

Cross-user paginated file table (GET /admin/files), filterable by status/user — a monitoring view, not an editing one; no destructive actions live here beyond viewing.

12.4 Analytics (admin/analytics/page.js)

The one screen in the product that *does* use charts (justifying the earlier decision to keep the Dashboard chart-free): a bar chart of (uploads_per_day), a single stat card for

`avg_processing_time_seconds`), and a stacked/grouped chart of `detection_stats.by_language` — all sourced from `GET /admin/analytics` (API Design §10.8), giving the admin (and, for demo purposes, the evaluator persona) a visual read on system health and the multilingual detection mix the whole product is built around.

12.5 Feedback (`admin/feedback/page.js`)

Simple list/detail view over `GET`/`PATCH /admin/feedback`, status chip (`open`/`reviewed`/`resolved`) changeable inline.

13. Component Inventory Cross-Reference

Maps this document's screens directly onto the folders already scaffolded in Folder Structure §3, so implementation has no ambiguity about where each piece lives:

Screen element	Component location
Dropzone / format validation UI	<code>components/upload/</code>
Recent files, credits/storage widgets	<code>components/dashboard/</code>
Transcript rendering, highlight spans, playback controls, edit toolbar	<code>components/timeline-editor/</code> (<code>TranscriptView.jsx</code> , <code>DetectionHighlight.jsx</code> , <code>PlaybackControls.jsx</code> , <code>EditActionsToolbar.jsx</code>)
Guest banner, navbar, sidebar, toasts	<code>components/shared/</code>
Admin tables/forms	<code>components/admin/</code>
Confidence color-scale logic, format constants	<code>lib/constants.js</code>
Editor undo/redo + draft state	<code>store/editorStore.js</code> , <code>hooks/useTimelineEditor.js</code>
WebSocket/job status subscription	<code>hooks/useJobStatus.js</code>
Form/response shape validation	<code>schemas/</code> (Zod — per Folder Structure v1.1 JS-only note)

14. Accessibility Notes (SRS-NFR-USE-02)

- Confidence is never communicated by color alone: every highlighted span also carries the literal score and source in its accessible name (`(aria-label="Detected: bakchodi, confidence 0.94, dictionary match")`), satisfying WCAG's color-independence guidance while still delivering the fast visual scan described in Section 9.2.
 - All interactive elements (dropzone, timeline spans, toolbar buttons) are keyboard-reachable and operable (Enter/Space to activate, arrow keys to step between detections in the transcript), per the WCAG 2.1 AA keyboard-navigation baseline.
 - Minimum contrast ratios verified for both light and dark token sets in Section 3.1, including the amber/rose confidence colors against their respective background tokens.
-

15. Next Document

Document 11: Development Roadmap — sequencing the implementation of the frontend screens specified here against the Release Plan (PRD §10) and the backend/AI modules from Documents 5–9, into a concrete sprint-by-sprint build order for the academic project timeline.

End of UI/UX Design Document

Development Roadmap Document SentinelCut — AI-Powered Audio/Video Profanity Censorship Platform

Document Version: 1.0 Status: Final for Implementation Based On: PRD v1.0 (§9 MoSCoW, §10 Release Planning), System Architecture v1.0, Database Design v1.0, Folder Structure v1.1, API Design v1.0, AI Pipeline v1.0, UI/UX Design v1.0

1. Purpose

This document sequences everything specified in Documents 2-10 into a concrete, sprint-by-sprint build order for an academic-timeline team. It answers the question none of the prior documents answer on their own: *in what order do we actually build this, and what is true and demoable at the end of each sprint*. Every sprint below ends in a working, demoable increment — not a partial slice that only makes sense once three more sprints land — because an academic project is graded/demoed at fixed checkpoints (PRD §10: mid-semester demo, pre-final demo, final submission), not at an arbitrary release date.

2. Planning Assumptions

Assumption	Value	Rationale
Project duration	16 weeks (one academic semester)	Standard college mini-project window
Sprint length	2 weeks	Gives 8 sprints total — enough granularity for weekly progress reporting (per Master Prompt's "Weekly Progress Report" deliverable) without excessive ceremony for a small team
Team size	3-5 students	Typical mini-project team size; role split assumed in Section 8, but the sprint plan degrades gracefully to a smaller team by extending sprint length rather than cutting scope
Infra decisions	Already finalized (Tech Stack Justification v1.0)	No time is spent in this roadmap re-evaluating ASR/DB/hosting choices — that debate is closed
Definition of "demoable"	The feature works end-to-end through the real UI against the real backend, not a mocked/hardcoded state	A sprint is not "done" if only an isolated unit test passes; per PRD §10 the checkpoints are live demos

3. Build-Order Dependency Rationale

Before assigning work to sprints, the dependency chain across Documents 5-10 needs to be made explicit, since several modules *look* independent but are not:

```

Database schema (Doc 6)
└─> Repository layer (Doc 7: backend/app/repositories/)
    └─> Service layer (Doc 7: backend/app/services/)
        └─> API routes (Doc 8)
            └─> Frontend screens (Doc 10) consume API contract
└─> AI Pipeline modules (Doc 9: ai-pipeline/) – buildable in parallel,
    since Doc 9's interfaces (SpeechRecognitionService,
    ProfanityDetectionService) do not depend on the API layer at all,
    only on the DB schema for what a "detection instance" must contain
    └─> Celery task wiring (backend/app/workers/) connects
        the AI Pipeline to the Job/Detection tables
        └─> Timeline Editor (Doc 10 §9) is the
last frontend screen buildable, since it
needs real detection data, not just an
upload ack

```

Consequence for sequencing: the AI Pipeline (Document 9) and the backend CRUD/auth layer can and should be built **in parallel by different team members from Sprint 1 onward**, both against the fixed Database Design schema (Document 6) as the shared contract — this is the single highest-leverage scheduling decision in this roadmap, and is why Sprint 1-2 below split the team across those two tracks rather than building strictly top-to-bottom.

4. Milestone Alignment (PRD §10 → Sprints)

PRD Release	PRD Target Milestone	Sprints	This Document's Milestone Name
R1 — MVP	Mid-semester demo	Sprints 1-4 (Weeks 1-8)	Milestone A: Core Loop Works
R2 — Usability & Control	Pre-final demo	Sprints 5-6 (Weeks 9-12)	Milestone B: User Control
R3 — Polish & Admin	Final submission/demo	Sprints 7-8 (Weeks 13-16)	Milestone C: Production-Shaped Polish
R4+ — Future Scope	Post-graduation	Not scheduled	Documented separately in Document 15: Future Scope

5. Sprint 0 — Setup (Pre-Sprint, ~3-4 days before Sprint 1)

Not a numbered sprint against the 16-week budget, but must complete before Sprint 1 starts.

- Initialize monorepo per Folder Structure v1.1 exactly (`frontend/`, `backend/`, `ai-pipeline/`, `database/`, `infra/`, `docs/`, `scripts/`, `tests/`).
- Provision Supabase project (Postgres + Auth + Storage), Upstash Redis instance, Vercel + Render/Railway accounts (Tech Stack §11).
- Run initial Alembic migration generating all tables from Database Design v1.0 §4 verbatim.
- Seed `profanity_dictionary` with an initial curated English + Hindi + Hinglish slang list (AI Pipeline §8 data source) via `scripts/seed_database.sh`.
- Stand up CI skeleton (`.github/workflows/*-ci.yml`) running lint + placeholder test on every PR from day one — cheap to add now, expensive to retrofit in Sprint 6.
- Copy all 10 finalized documents into `docs/` per the naming convention in Folder Structure §9.

Definition of Done: `docker-compose up` brings up Postgres+Redis+backend+frontend locally with zero errors; CI is green on an empty commit.

6. Milestone A — Core Loop Works (Sprints 1-4)

Sprint 1 (Weeks 1-2): Auth + Upload Skeleton + ASR Proof-of-Concept

Two parallel tracks, per Section 3.

Backend/Frontend track:

- Implement `AuthService`/`user_repository.py`/`auth_routes.py` for email/password signup+login only (Google OAuth and guest mode deferred to Sprint 2 — they're additive, not blocking).
- Implement `FileService`/`file_repository.py`/`file_routes.py` for `POST /files/upload` and `GET /files/{id}` only, writing to Supabase Storage.
- Frontend: `(auth)` route group screens (Doc 10 §6, email/password paths only), `(app)` shell layout (Navbar/Sidebar), bare Upload page (dropzone + validation only, no configuration panel yet).

AI Pipeline track:

- Implement Stage 1 (`audio_extractor.py`) and Stage 2 (`faster_whisper_engine.py`) behind `SpeechRecognitionService`, tested standalone via a CLI script against fixture files — **no Celery/API integration yet**. Goal is proving Faster-Whisper's accuracy and latency on the team's actual target hardware before anything else is built on top of it, since this is the single highest-risk technical assumption in the whole project (per Tech Stack §3).

Definition of Done: A user can sign up, log in, and upload a file that lands in Storage with a `(files)` row; separately, a command-line script transcribes a sample Hindi/Hinglish clip with word-level timestamps and prints recognizable output.

Sprint 2 (Weeks 3-4): Job Queue + Full ASR/Detection Pipeline (Automatic Mode Only)

- Wire Celery/RQ (`(backend/app/workers/celery_app.py)`, `(tasks/processing_task.py)`) so `(POST /files/{id}/process)` actually enqueues and a worker dequeues.
- Complete AI Pipeline Stages 3-7 (language detection, normalization, dictionary matching, context scoring, confidence aggregation) for **Automatic detection mode only** — Custom/Combined modes deferred to Milestone B per the PRD's own MoSCoW (§9: Custom/Combined is a "Should Have," R2).
- Persist `(detections)` rows; implement `(GET /files/{id}/detections)` and `(GET /files/{id}/transcript)`.
- Add Guest Mode (`(POST /auth/guest-session)`) and Google OAuth (`(POST /auth/google)`) to the auth track — both are Must-Haves for R1 per PRD §9 but were reasonably deferred from Sprint 1 since they're additive to an already-working email/password path.
- Frontend: WebSocket/polling status hook (`(useJobStatus.js)`), Processing Status page (Doc 10 §8.3).

Definition of Done: Uploading a file through the real UI results in a real, worker-produced set of detection instances visible via `(GET /files/{id}/detections)` (Postman/curl-level verification is acceptable here; the Timeline Editor UI to view them lands next sprint).

Sprint 3 (Weeks 5-6): Timeline Editor (Read + Edit, No Regeneration Yet) + Beep/Mute Rendering

- Frontend: build the Timeline Editor screen (Doc 10 §9.1-9.4) against the now-real `(/detections)` and `(/transcript)` endpoints — transcript rendering, confidence-color highlighting, detail panel, dismiss/delete, manual-add. This is correctly sequenced as the last major frontend screen in Milestone A because it has the longest dependency chain (Section 3).
- Backend: `(PATCH /detections/{id})`, `(PATCH /files/{id}/detections/batch)`, `(POST /files/{id}/detections)`.
- AI Pipeline: implement Stage 8 (`(audio_editor.py)`) for Beep and Mute only (Custom Sound deferred — it's an R2 "Should Have" per PRD §9) and Stage 9 (`(media_muxer.py)`).
- Wire `(POST /files/{id}/regenerate)` end-to-end, calling only Stages 8-9 (AI Pipeline §12/§14) — this is the first point in the project where the "regeneration is

cheap" architectural decision actually gets exercised and proven, not just documented.

Definition of Done: A user can review AI-detected profanity with visible confidence colors, dismiss a false positive, add a missed one, click Regenerate, and download a video with an audible beep at the correct timestamp.

Sprint 4 (Weeks 7-8): Export Formats + Dashboard + Mid-Semester Demo Hardening

- Backend: `GET /files/{id}/exports`, signed-URL download endpoint, generating `.srt`/`.txt`/`.json` exports alongside the censored media (SRS-FR-EXPT-03/04/05).
- Frontend: Dashboard (Doc 10 §7) wired to `GET /dashboard/summary`; inline preview player in the Timeline Editor (Doc 10 §9.7).
- Bug-fix/hardening pass: run the integration test from AI Pipeline §18 against 5-10 real sample files spanning English/Hindi/Hinglish; fix any sync-drift or normalization gaps found.
- Rehearse the demo script against PRD §6.1's Primary Journey exactly (upload → automatic detect → review → regenerate → download).

Definition of Done — Milestone A / Mid-Semester Demo: Every "Must Have" row in PRD §9's MoSCoW table for R1 works live, end-to-end, through the deployed (not just local) environment.

7. Milestone B — User Control (Sprints 5-6)

Sprint 5 (Weeks 9-10): Custom/Combined Detection + Presets + Custom Sound Replacement

- AI Pipeline: extend the detection-config merge logic (AI Pipeline §14, `merge_with_custom_words`) to actually support Custom and Combined modes, reusing the Stage 4-5 normalization/matching code path as designed.
- Backend: `presets` CRUD routes (API Design §9); Custom Sound support in `audio_editor.py` (AI Pipeline §12, step 4's loop/trim logic).
- Frontend: Upload page's full Configuration Panel (Doc 10 §8.2, mode selector + preset dropdown + custom-sound sub-upload).

Definition of Done: A user can define a custom word list, save it as a named preset, reuse it on a later upload, and choose a custom audio clip as the replacement sound.

Sprint 6 (Weeks 11-12): Subtitles, History/Search, Confidence Filtering + Pre-Final Demo Hardening

- Backend: `.srt` export generation with censorship markers (SRS-FR-EXPT-03); `GET /files` search/filter/sort query params (API Design §4.3).

- Frontend: History page (Doc 10 §10); Timeline Editor confidence-threshold slider (Doc 10 §9.5); undo/redo stack (Doc 10 §9.6, at least 10 actions per SRS-FR-TLE-09).
- Begin the accuracy benchmark pass (AI Pipeline §18, `scripts/benchmark_asr.py` / `benchmark_detection.py`) against a labeled test set, producing the actual recall/precision numbers referenced in PRD §4.6 AC-1 — this needs a full sprint of lead time before the final demo, not a last-week scramble, since tuning `CONTEXT_SIMILARITY_THRESHOLD` (AI Pipeline §19) based on these results may require re-running the benchmark multiple times.

Definition of Done — Milestone B / Pre-Final Demo: Every "Should Have" row in PRD §9's MoSCoW for R2 works live; a written accuracy report exists with real recall/precision numbers against the internal test set.

8. Milestone C — Production-Shaped Polish (Sprints 7-8)

Sprint 7 (Weeks 13-14): Admin Panel + Rate Limiting/Virus Scan + Dark Mode

- Backend: all `/admin/*` routes (API Design §10) — user management, dictionary CRUD, files overview, feedback; `admin_logs` audit writes on every mutation (SRS-FR-ADM-05).
- Security hardening pass: rate limiting on `/auth/*` and `/files/upload` (SRS-NFR-SEC-06), malware scanning integration on upload (SRS-FR-UPLD-07) — both explicitly scheduled here rather than earlier, since they are R3 "Could Have" items per PRD §9 and gate nothing else in Milestones A/B.
- Frontend: all five Admin sub-screens (Doc 10 §12); global dark-mode token toggle (Doc 10 §3.5).

Definition of Done: An admin user can disable a user account, edit the profanity dictionary and see the change reflected in new uploads within the documented cache TTL (AI Pipeline §8), and view basic analytics — all while the rest of the app renders correctly in both themes.

Sprint 8 (Weeks 15-16): Credits/Storage UI, Full Regression, Documentation Finalization, Final Demo

- Frontend: Storage-usage progress bar and Credits display polish (already data-available from Sprint 4's Dashboard, this sprint adds the `credits_ledger`-backed consumption logic if not already wired); Settings/Profile screens (Doc 10 §11), including the account-deletion "Danger Zone" flow (SRS-NFR-LEGAL-02).
- Full regression pass against every AC in the PRD (§4.1-§4.10) and every SRS-NFR category (§5.1-§5.7) — this is a dedicated QA sprint, not a bug-fix-as-you-go sprint, and should follow the Test Plan/Test Cases/QA Checklist deliverables (Document 13).

- Finalize all remaining academic deliverables (College Report, IEEE-style paper, viva prep, demo script, poster — per Master Prompt's Documentation section and Folder Structure §9's `docs/academic/`).
- Final submission/demo rehearsal against the full PRD §6 User Journeys (Primary, Secondary/Guest, Edge Case).

Definition of Done — Milestone C / Final Submission: Every requirement in SRS §3 tagged Must or Should is demonstrably working; every NFR in SRS §5 has been checked at least once against its stated target (even if only via a manual/scripted spot-check rather than full load testing, given academic-scale infrastructure per SRS §5.1's own "academic/demo-scale target" framing).

9. Cross-Cutting Work Not Tied to a Single Sprint

These run continuously across all 8 sprints rather than landing in one:

Activity	Cadence	Owner
Weekly Progress Report (Master Prompt deliverable)	Every sprint, end of week 1 and week 2	Whole team, PM role compiles
CI lint/test on every PR	Continuous	Enforced by <code>.github/workflows/</code> from Sprint 0
<code>job_logs</code> /structured logging review	Continuous, spot-checked each sprint	Whoever owns the AI Pipeline track that sprint
Dictionary curation (adding new slang/profanity terms)	Ongoing, especially after Sprint 6 benchmarking	Whoever has admin access; formalized via Admin Panel from Sprint 7 onward
Coding Standards/Git Workflow adherence (Document per Master Prompt)	Continuous from Sprint 0	Enforced via PR review, documented once in <code>docs/standards/</code>

10. Risk Register

Risk	Likelihood	Impact	Mitigation
Faster-Whisper CPU latency exceeds the $\leq 2\times$ -real-time budget (SRS-NFR-PERF-01) on the team's actual free-tier hosting hardware	Medium	High — undermines the core demo	De-risked <i>first</i> , in Sprint 1, before any other code depends on it (Section 6, Sprint 1's standalone CLI proof-of-concept); fallback is dropping to the <code>small</code> model size (AI Pipeline §19)
Hindi/Hinglish detection recall falls short of the 85% target (PRD §4.6 AC-1)	Medium	Medium — a review-heavy UI (Timeline Editor) is the designed safety net, so this degrades UX rather than breaking the product	Benchmark pass explicitly scheduled in Sprint 6, not deferred to the last week, leaving time to expand the dictionary/seed bank (AI Pipeline §9)
Free-tier hosting quota exhaustion (Render/Railway/Supabase) during the final demo	Low	High — demo-day failure	Separate staging/production environments (System Architecture §6.1) so demo traffic never shares a quota with development churn; final demo rehearsed against production, not staging
Team member availability gaps (exams, other coursework) common in academic timelines	High	Medium	Sprint plan is intentionally sequenced so each sprint's Definition of Done is independently demoable — a slipped sprint delays the <i>next</i> milestone but does not invalidate work already shipped

Risk	Likelihood	Impact	Mitigation
Scope creep toward Future Scope items (voice cloning, live streaming, per Master Prompt's long features list)	Medium	Medium	PRD §9's "Won't Have (This Release)" list is treated as binding for all 8 sprints above; anything not explicitly scheduled here is out of scope until Document 15 (Future Scope)

11. Suggested Role Split (3–5 person team)

Role	Primary sprints	Primary ownership
Backend/API engineer	All	<code>backend/app/</code> — routes, services, repositories, Celery wiring
AI/ML engineer	1-2, 5-6 heaviest	<code>ai-pipeline/</code> — all 9 stages, benchmarking
Frontend engineer(s)	All, especially 3 (Timeline Editor)	<code>frontend/src/</code> — all screens per Doc 10
DevOps/QA (can be shared with another role on a 3-person team)	0, 4, 8 heaviest	CI/CD, deployment configs, regression pass, Test Plan (Doc 13)

On a 3-person team, the AI/ML and Backend roles are the most defensible to merge (both are Python, per Tech Stack §6's own "same-language pipeline" rationale) — the Frontend and DevOps/QA roles are the ones to keep distinct if forced to choose, since the Timeline Editor (Section 6, Sprint 3) and the final regression pass (Section 8, Sprint 8) are each large enough to be a full-time focus on their own sprint.

12. Next Document

Document 12: Implementation Notes — concrete coding-level guidance (naming conventions in practice, example service/repository implementations, dependency-injection wiring patterns) supporting the sprints defined above, followed by Document 13: Testing (Test Plan, Test Cases, QA Checklist) formalizing the regression pass scheduled in Sprint 8.

End of Development Roadmap Document

Implementation Notes SentinelCut — AI-Powered Audio/Video Profanity Censorship Platform

Document Version: 1.0 Status: Final for Implementation Based On: Folder Structure v1.1, Database Design v1.0, API Design v1.0, AI Pipeline v1.0, Development Roadmap v1.0

1. Purpose

Every prior document specified *what* exists (schemas, endpoints, pipeline stages) and *when* it gets built (Development Roadmap). This document specifies *how the code is actually written* — concrete naming conventions as applied to real files, worked repository/service implementations, and the dependency-injection wiring that connects FastAPI routes to services to repositories to the AI pipeline. It is the reference a developer opens while writing code in Sprint 1, not after Sprint 8.

This is deliberately narrower than a general "Coding Standards" style guide (that document — `docs/standards/Coding_Standards.md` — covers formatting, linting, and PR conventions). This document covers **architecture-in-practice**: SOLID, Repository Pattern, and dependency injection, made concrete rather than left as terminology.

2. Naming Conventions in Practice

These are not abstract rules — they are the exact conventions used consistently across every code example in this document, so copy-pasting from here into the real codebase requires no translation.

Element	Convention	Example
Python module/file	<code>snake_case.py</code>	<code>detection_repository.py</code>
Python class	<code>PascalCase</code> , suffixed by its architectural role	<code>DetectionRepository</code> , <code>ProfanityDetectionService</code> , <code>FileUploadRequest</code>
Python function/variable	<code>snake_case</code>	<code>get_active_detections</code> , <code>min_confidence</code>
Interface / abstract base class	<code>PascalCase</code> ending in the role name, no <code>I</code> / <code>Abstract</code> prefix (Pythonic, not C#-style)	<code>SpeechRecognitionService</code> (ABC), not <code>ISpeechRecognitionService</code>
Concrete implementation of an interface	<code>PascalCase</code> naming the <i>technology</i> , not a generic suffix like <code>Impl</code>	<code>FasterWhisperEngine</code> implements <code>SpeechRecognitionService</code> — the name tells you <i>which</i> engine at a glance
SQLAlchemy ORM model	Singular noun, matches table name (plural) minus the <code>s</code>	Table <code>files</code> → model class <code>File</code>
Pydantic schema	Suffixed by direction/purpose	<code>FileUploadRequest</code> , <code>FileResponse</code> , <code>DetectionUpdatePatch</code>
FastAPI route function	<code>verb_noun</code> , matches the REST action	<code>create_file</code> , <code>list_files</code> , <code>patch_detection</code>
React component file	<code>PascalCase.jsx</code>	<code>DetectionHighlight.jsx</code>
React hook	<code>camelCase</code> , <code>use</code> prefix	<code>useTimelineEditor.js</code>
Custom exception	<code>PascalCase</code> ending in <code>Error</code>	<code>AudioExtractionError</code> , <code>DuplicatePresetError</code>
Celery task	<code>snake_case</code> , verb-first, matches <code>jobs.job_type</code> value	<code>run_initial_processing</code> , <code>run_regeneration</code>

Element	Convention	Example
Environment variable	UPPER_SNAKE_CASE, module-prefixed where it could collide	ASR_MODEL_SIZE, JWT_ACCESS_TOKEN_EXPIRY_MINUTES

Branch/commit naming (feeds docs/standards/Git_Workflow.md, referenced here since it affects daily practice): feature/<sprint-number>-<short-slug> (e.g., feature/sprint3-timeline-editor-highlights), commit messages in imperative mood (Add DetectionRepository.get_active_detections), referencing the SRS-FR ID where the commit implements a specific requirement (e.g., Implement SRS-FR-TLE-04: manual detection add) — this small habit is what makes the Requirements Traceability Matrix (Document 13 companion) verifiable against real commits rather than reconstructed after the fact.

3. Layering Recap (What Calls What)



The rule that must never be violated: **a Route never imports a Repository directly, and a Service never constructs a database session itself.** Every example below exists to make that rule mechanically easy to follow, not just a convention developers have to remember under deadline pressure.

4. Repository Pattern — Worked Example

4.1 Generic Base Repository

```
python
```

```

# backend/app/repositories/base_repository.py
from typing import TypeVar, Generic, Type
from sqlalchemy.ext.asyncio import AsyncSession
from sqlalchemy import select

ModelType = TypeVar("ModelType")

class BaseRepository(Generic[ModelType]):
    """Generic CRUD so every entity-specific repository isn't reinventing
    get_by_id/list/create/delete. Entity-specific repositories subclass this
    and add only the queries that are actually specific to that entity."""

    def __init__(self, session: AsyncSession, model: Type[ModelType]):
        self.session = session
        self.model = model

    async def get_by_id(self, id: str) -> ModelType | None:
        result = await self.session.execute(
            select(self.model).where(self.model.id == id)
        )
        return result.scalar_one_or_none()

    async def create(self, **kwargs) -> ModelType:
        instance = self.model(**kwargs)
        self.session.add(instance)
        await self.session.flush()
        return instance

    async def delete(self, id: str) -> None:
        instance = await self.get_by_id(id)
        if instance:
            await self.session.delete(instance)

```

4.2 Entity-Specific Repository

python

```

# backend/app/repositories/detection_repository.py
from sqlalchemy import select
from app.models.detection import Detection
from app.repositories.base_repository import BaseRepository

class DetectionRepository(BaseRepository[Detection]):
    """Adds only the queries this app actually needs beyond generic CRUD —
    each one traces to a specific FR from the SRS."""

    def __init__(self, session):
        super().__init__(session, Detection)

    async def get_active_by_file(self, file_id: str) -> list[Detection]:
        # SRS-FR-TLE-01/02: transcript view excludes dismissed detections
        result = await self.session.execute(
            select(Detection)
            .where(Detection.file_id == file_id, Detection.is_dismissed == False)
            .order_by(Detection.start_time)
        )
        return list(result.scalars().all())

    async def bulk_patch(self, updates: list[dict]) -> int:
        # Backs PATCH /files/{id}/detections/batch (API Design §5.4)
        count = 0
        for update in updates:
            detection = await self.get_by_id(update["id"])
            if detection is None:
                continue
            for key, value in update.items():
                if key != "id":
                    setattr(detection, key, value)
            count += 1
        await self.session.flush()
        return count

```

Why this matters beyond style: because `DetectionRepository` is the *only* place `Detection` is queried, a future change (e.g., adding soft-delete semantics, or swapping to a read-replica for list queries) touches one file, not every route handler that happens to need detections — this is the concrete payoff of SRS-NFR-MAINT-01's "modular, layered architecture" requirement, not just a label on an architecture diagram.

5. Service Layer — Worked Example (with Dependency Injection)

python

```
# backend/app/services/timeline_edit_service.py
from app.repositories.detection_repository import DetectionRepository
from app.repositories.file_repository import FileRepository
from app.core.exceptions import NotFoundError, InvalidTimeRangeError

class TimelineEditService:
    """Depends only on repository INTERFACES it needs – never imports
    SQLAlchemy, never imports FastAPI. This is what makes the service
    layer independently unit-testable (Development Roadmap Sprint 8's
    regression pass runs these tests without spinning up a real DB)."""

    def __init__(
        self,
        detection_repo: DetectionRepository,
        file_repo: FileRepository,
    ):
        self.detection_repo = detection_repo
        self.file_repo = file_repo

    async def add_manual_detection(
        self, file_id: str, user_id: str, detected_text: str,
        start_time: float, end_time: float, replacement_mode: str,
    ):
        # Ownership check belongs HERE, not in the route – SRS-NFR-SEC-04 (RBAC)
        file = await self.file_repo.get_by_id(file_id)
        if file is None or file.user_id != user_id:
            raise NotFoundError("File not found")

        # SRS-FR-TLE-04's implicit validation rule (API Design §5.2 error case)
        if end_time <= start_time:
            raise InvalidTimeRangeError("end_time must be after start_time")

        return await self.detection_repo.create(
            file_id=file_id,
            detected_text=detected_text,
            start_time=start_time,
            end_time=end_time,
            replacement_mode=replacement_mode,
            is_user_added=True,
            detection_source="manual",
            confidence_score=1.0, # user-added detections are certain by definition
        )
```

Note on `confidence_score=1.0`: this is a direct, deliberate implementation of the UI/UX Design document's §9.4 rule that manually-added detections render with a distinct solid outline rather than the graded confidence color scale — setting the score to `1.0` here (rather than `null`) keeps the database schema's `NOT NULL` constraint on `confidence_score` (Database Design §4.8) intact without a special-case column, and the frontend distinguishes "manual" from "AI-inferred" via `detection_source`, not via a missing score.

6. Dependency Injection Wiring (FastAPI `Depends`)

This is the piece that actually connects Sections 4–5 to a live HTTP request, without any layer constructing its own dependencies (the concrete meaning of "use dependency injection where appropriate" from the Master Prompt's Coding Standards section).

python

```
# backend/app/dependencies.py
from fastapi import Depends
from sqlalchemy.ext.asyncio import AsyncSession
from app.db.session import get_db_session
from app.repositories.detection_repository import DetectionRepository
from app.repositories.file_repository import FileRepository
from app.services.timeline_edit_service import TimelineEditService

def get_detection_repository(session: AsyncSession = Depends(get_db_session)):
    return DetectionRepository(session)

def get_file_repository(session: AsyncSession = Depends(get_db_session)):
    return FileRepository(session)

def get_timeline_edit_service(
    detection_repo: DetectionRepository = Depends(get_detection_repository),
    file_repo: FileRepository = Depends(get_file_repository),
) -> TimelineEditService:
    return TimelineEditService(detection_repo, file_repo)
```

python

```

# backend/app/api/v1/routes/detection_routes.py
from fastapi import APIRouter, Depends
from app.services.timeline_edit_service import TimelineEditService
from app.dependencies import get_timeline_edit_service
from app.api.deps.auth_deps import get_current_user
from app.schemas.detection_schema import DetectionCreateRequest, DetectionResponse

router = APIRouter()

@router.post("/files/{file_id}/detections", response_model=DetectionResponse, status_code=201)
async def create_detection(
    file_id: str,
    body: DetectionCreateRequest,
    current_user = Depends(get_current_user),  # SRS-NFR-SEC
    service: TimelineEditService = Depends(get_timeline_edit_service),
):
    # The route's ONLY job: unwrap the request, call one service method, return the response
    # No business logic, no DB access, no ownership check lives here — see Section 7.1
    detection = await service.add_manual_detection(
        file_id=file_id,
        user_id=current_user.id,
        detected_text=body.detected_text,
        start_time=body.start_time,
        end_time=body.end_time,
        replacement_mode=body.replacement_mode,
    )
    return DetectionResponse.model_validate(detection)

```

Why this specific wiring, and not a global singleton container: FastAPI's `Depends()` graph is resolved per-request, which means `get_db_session` can yield a fresh `AsyncSession` scoped to exactly one request/transaction — this is what prevents the classic bug of two concurrent requests silently sharing (and corrupting) the same DB session, which matters directly for SRS-NFR-PERF-04's concurrent-job-handling requirement.

7. Wiring the AI Pipeline Into a Celery Task

The AI Pipeline document (Document 9) defined `ProcessingPipeline` as a plain Python class with no FastAPI or Celery awareness — by design, so it stays testable via the CLI proof-of-concept described in Development Roadmap Sprint 1. The Celery task is a thin adapter, not a place to put pipeline logic:

python

```

# backend/app/workers/tasks/processing_task.py
from app.workers.celery_app import celery_app
from sentinelcut_ai.pipeline.processing_pipeline import ProcessingPipeline
from app.repositories.job_repository import JobRepository
from app.db.session import get_sync_session # Celery tasks use sync sessions, not async

@celery_app.task(bind=True, max_retries=3)
def run_initial_processing(self, file_id: str, config: dict):
    session = get_sync_session()
    job_repo = JobRepository(session)
    try:
        pipeline = ProcessingPipeline.build_default() # composes concrete engine
        result = pipeline.run_initial(file_id, config)
        job_repo.mark_completed(file_id, result)
    except Exception as exc:
        # Matches the retry/backoff sequence diagrammed in System Architecture §4
        job_repo.log_error(file_id, stage="unknown", message=str(exc))
        raise self.retry(exc=exc, countdown=2 ** self.request.retries)

```

`ProcessingPipeline.build_default()` is the single place in the codebase where concrete engines are chosen (`FasterWhisperEngine`, the hybrid `ProfanityDetectionService` chain) — exactly one factory function stands between "the interfaces defined in Document 9" and "a real running pipeline," which is what makes the extensibility points in AI Pipeline §17 (swap ASR engine, swap detection model) a one-function change rather than a search-and-replace across the codebase.

8. Custom Exceptions & Error Mapping

```

python

# backend/app/core/exceptions.py
class SentinelCutError(Exception):
    """Base class; never raised directly."""

class NotFoundError(SentinelCutError):
    pass

class InvalidTimeRangeError(SentinelCutError):
    pass

class DuplicateResourceError(SentinelCutError):
    pass

```

python

```
# backend/app/main.py (exception handler registration)
from fastapi import Request
from fastapi.responses import JSONResponse
from app.core.exceptions import NotFoundError, InvalidTimeRangeError, DuplicateRe

@app.exception_handler(NotFoundError)
async def not_found_handler(request: Request, exc: NotFoundError):
    return JSONResponse(status_code=404, content={
        "success": False,
        "error": {"code": "NOT_FOUND", "message": str(exc)}
    })

@app.exception_handler(InvalidTimeRangeError)
async def invalid_range_handler(request: Request, exc: InvalidTimeRangeError):
    return JSONResponse(status_code=400, content={
        "success": False,
        "error": {"code": "VALIDATION_ERROR", "message": str(exc)}
    })
```

This is what makes the API Design document's error-code table (Document 8 §13) actually true in the running system: services raise domain exceptions and never know about HTTP status codes at all; exactly one place (`main.py`'s handler registry) maps a domain exception to the envelope shape and status code the frontend contract promises.

9. Configuration Pattern

python

```
# backend/app/config.py
from pydantic_settings import BaseSettings

class Settings(BaseSettings):
    database_url: str
    redis_url: str
    jwt_secret: str
    jwt_access_token_expiry_minutes: int = 60
    max_upload_size_mb: int = 500
    guest_session_ttl_hours: int = 24

    class Config:
        env_file = ".env"

settings = Settings() # instantiated once, imported everywhere – never re-read p
```

Directly implements SRS-NFR-MAINT-03 ("externalize configuration rather than hardcoding") — every tunable value named in the SRS and AI Pipeline documents (max file size, token expiry, confidence thresholds) has exactly one settings field, never a magic number inlined in a route or service.

10. SOLID Principles, Made Concrete (Not Just Named)

Principle	Where it shows up in the examples above
Single Responsibility	<code>DetectionRepository</code> only queries; <code>TimelineEditService</code> only orchestrates business rules; the route only translates HTTP↔service calls (Section 3)
Open/Closed	<code>BaseRepository</code> is extended (subclassed), never modified, when a new entity needs a repository (Section 4.1→4.2)
Liskov Substitution	Any concrete <code>SpeechRecognitionService</code> (Faster-Whisper today, a cloud API later) must be substitutable behind the same <code>transcribe()</code> signature without callers changing (AI Pipeline §5, §17)
Interface Segregation	<code>SpeechRecognitionService</code> and <code>ProfanityDetectionService</code> are separate interfaces, not one giant <code>AIEngine</code> interface — a service that only needs transcription never depends on detection-scoring methods it doesn't use
Dependency Inversion	<code>TimelineEditService</code> depends on <code>DetectionRepository</code> / <code>FileRepository</code> (concretions injected via <code>Depends()</code>), but nothing in the service layer imports SQLAlchemy directly — the direction of dependency points toward abstractions the service defines its needs against

11. Frontend Equivalent: The `api-client.js` Boundary

The same "one seam, not scattered calls" discipline applies on the frontend:

```
javascript
```

```
// frontend/src/lib/api-client.js
const BASE_URL = process.env.NEXT_PUBLIC_API_URL;

async function request(path, options = {}) {
  const token = getStoredToken(); // from authStore
  const res = await fetch(`${BASE_URL}${path}`, {
    ...options,
    headers: {
      "Content-Type": "application/json",
      ...(token ? { Authorization: `Bearer ${token}` } : {}),
      ...options.headers,
    },
  });
  const body = await res.json();
  if (!body.success) throw new ApiError(body.error);
  return body.data;
}

export const apiClient = {
  getDetections: (fileId, minConfidence) =>
    request(`/files/${fileId}/detections?min_confidence=${minConfidence ?? 0}`),
  patchDetection: (id, patch) =>
    request(`/detections/${id}`, { method: "PATCH", body: JSON.stringify(patch) })
  // ... one function per endpoint in API Design Document 8, no component calls
};
```

No component under `components/timeline-editor/` ever calls `fetch` directly — every network call goes through `apiClient`, mirroring the backend rule that routes never touch SQL directly. This is what keeps the error-envelope handling (`ApiError`, matching Document 8 §2.3's error shape) in exactly one place instead of duplicated per component.

12. Next Document

Document 13: Testing — Test Plan, Test Cases, and QA Checklist, formalizing the regression pass scheduled in Development Roadmap Sprint 8, and defining exactly how the layering and patterns established in this document are verified (unit tests against services/repositories in isolation per Section 5's note on testability, integration tests against real routes, and end-to-end tests against the full user journeys from the PRD).

End of Implementation Notes

SentinelCut — API Contract (v1, MVP)

This is the fixed contract all four team members build against. Backend implements it exactly as written here; frontend, AI, and audio/video pieces integrate against it without needing to ask each other questions. If something needs to change, update this file first, then code — not the other way around.

Base URL (local dev): `http://localhost:8000/api/v1`

Auth: all endpoints except `/auth/*` require `Authorization: Bearer <jwt>` header.

1. Auth

`POST /auth/signup`

Request

```
json
{ "email": "user@example.com", "password": "min8chars" }
```

Response 201

```
json
{ "user_id": "uuid", "email": "user@example.com", "access_token": "jwt..." }
```

`POST /auth/login`

Request

```
json
{ "email": "user@example.com", "password": "min8chars" }
```

Response 200

```
json
{ "user_id": "uuid", "email": "user@example.com", "access_token": "jwt..." }
```

Response 401 — invalid credentials

2. Upload

`POST /upload`

`multipart/form-data`

- `file`: the audio/video file (mp4, mp3, wav for MVP)
- `custom_words` (optional): JSON string array, e.g. `["badword1", "badword2"]`

Response 202 — file accepted, job queued

```
json
{
  "job_id": "uuid",
  "file_id": "uuid",
  "filename": "clip.mp4",
  "status": "queued"
}
```

Response 400 — bad format / file too large

```
json
{ "error": "unsupported_format", "message": "Only mp4, mp3, wav are supported in
```

3. Job status

`GET /jobs/{job_id}`

Response 200

```
json
{
  "job_id": "uuid",
  "status": "queued | processing | done | failed",
  "progress_pct": 0,
  "stage": "transcribing | detecting | rendering | null",
  "error": null
}
```

`status` values and meaning:

- `queued` — accepted, not started
- `processing` — worker picked it up, see `stage`
- `done` — detections + outputs ready
- `failed` — see `error`

Frontend polls this every 2–3s (per your NFR: status updates within 5s).

4. Detections

GET /jobs/{job_id}/detections

Only valid once `status = done` or `processing` (partial results allowed once transcription stage completes).

Response 200

json

```
{
  "job_id": "uuid",
  "transcript_language_hint": "hinglish",
  "detections": [
    {
      "id": "det_001",
      "word": "fuuuuck",
      "normalized": "fuck",
      "language": "english",
      "severity": "high",
      "start": 12.34,
      "end": 12.81,
      "source": "dictionary",
      "confidence": 0.95,
      "replacement": "beep",
      "status": "active"
    }
  ],
  "full_transcript": [
    { "word": "this", "start": 12.01, "end": 12.20 },
    { "word": "movie", "start": 12.21, "end": 12.33 }
  ]
}
```

See `detection-schema.json` for the authoritative schema of a single detection object.

`status` on a detection: `active` (will be censored) | `removed` (user deleted it) | `added` (user manually added it).

PATCH /jobs/{job_id}/detections/{detection_id}

User edits a detection in the timeline editor. **Request** (any subset of fields)

json

```
{ "status": "removed" }
```

or

```
json  
  
{ "replacement": "mute" }
```

Response 200 — updated detection object.

POST /jobs/{job_id}/detections

User manually adds a detection the AI missed. **Request**

```
json  
  
{ "word": "badword", "start": 45.0, "end": 45.5, "replacement": "beep" }
```

Response 201 — created detection object, `{source: "manual", status: "added"}`.

5. Regenerate / export

POST /jobs/{job_id}/regenerate

Re-renders the output using current detection edits. Triggers a new render job (does not redo transcription/detection). **Response 202**

```
json  
  
{ "job_id": "uuid", "status": "processing", "stage": "rendering" }
```

GET /jobs/{job_id}/export?type=video|audio|transcript|json

Returns a signed download URL for the requested artifact. **Response 200**

```
json  
  
{ "type": "video", "download_url": "https://storage.../signed-url", "expires_at":
```



6. Dashboard

GET /files

List current user's uploads (recent files / history — MVP version, no pagination needed yet). **Response 200**

```
json
```

```
{
  "files": [
    { "job_id": "uuid", "filename": "clip.mp4", "status": "done", "created_at": "2020-01-01T00:00:00Z" }
  ]
}
```

7. Error format (all endpoints)

json

```
{ "error": "error_code", "message": "human readable message" }
```

Common `error` codes: `unsupported_format`, `file_too_large`, `unauthorized`, `not_found`, `processing_failed`.

Notes for each owner

- **Person 1 (AI):** your script's final output must map directly into the `detections` array shape below — that's the handoff point to Person 2.
- **Person 2 (backend):** implement all endpoints above with mock/hardcoded data first (Step 4), then wire to Person 1's pipeline and Person 3's renderer.
- **Person 3 (audio/video):** your render function's input is the `detections` array (only `status: active` ones) + original file; output feeds the `/export` endpoint.
- **Person 4 (frontend):** build against this contract directly; use the example JSON blocks above as your mock data while backend is still wiring up.